

Inverse Adding-Doubling

(Version 3-16-3)

	Section	Page
iad command-line program	1	1
Handling the command-line options	6	5
The forward calculation	7	12
Calculating a grid for graphing	11	14
Monte Carlo light loss	17	19
IAD Types	34	32
IAD Public	40	36
Inverse RT	43	37
Validation	48	40
Searching Method	56	43
EZ Inverse RT	66	49
IAD Input Output	91	59
Reading the file header	93	60
Reading just one line of a data file	99	62
Formatting the header information	108	66
IAD Calculation	123	72
Initialization	125	75
Gain	126	76
Grid Routines	137	80
Calculating R and T	178	95
Reflectance measurement for one sphere	187	98
Transmittance measurement for one sphere	194	100
Dual beam case for one sphere	199	102
Double integrating spheres	201	102
IAD Find	232	111
Fixed Anisotropy	234	112
Fixed Absorption and Anisotropy	243	116
Fixed Absorption and Scattering	245	117
Fixed Optical Depth and Anisotropy	247	118
Fixed Optical Depth and Albedo	249	119
Fixed Anisotropy and Albedo	251	120
Fixed Optical Depth	254	121
Fixed Albedo	259	124
Fixed Scattering	265	127
Fixed Absorption	270	129
IAD Utilities	275	131
Finding optical thickness	277	133
Estimating R and T	283	135
Transforming properties	289	137
Guessing an inverse	310	142
Some debugging stuff	327	147

Index	340	151
--------------------	------------	-----

Copyright © 1993–2024 Scott Prahl

Permission is granted to make and distribute verbatim copies of this document provided that the copyright notice and this permission notice are preserved on all copies.

Permission is granted to copy and distribute modified versions of this document under the conditions for verbatim copying, provided that the entire resulting derived work is given a different name and distributed under the terms of a permission notice identical to this one.

1. iad command-line program.

Here is a relatively robust command-line utility that shows how the iad and ad subroutines might be called. It suffers because it is written in CWEB and I used the macro expansion feature instead of creating separate functions. Oh well.

2. All the actual output for this web file goes into iad_main.c

```

<iad_main.c 2> ≡
  <Include files for main 3>
  <print version function 21>
  <print usage function 22>
  <stringdup together function 27>
  <mystrtod function 28>
  <seconds elapsed function 29>
  <print error legend function 26>
  <what_char function 31>
  <print long error function 32>
  <print dot function 33>
  <calculate coefficients function 23>
  <parse string into array function 30>
  <print results header function 24>
  <Print results function 25>
  int main(int argc, char **argv)
  {
    <Declare variables for main 4>
    <Save the command line for use later 5>
    <Handle options 6>
    Initialize_Measure(&m);
    <Command-line changes to m 18>
    Initialize_Result(m, &r, TRUE);
    <Command-line changes to r 15>
    if (cl_forward_calc ≠ UNINITIALIZED) {
      <Calculate and Print the Forward Calculation 7>
      exit(EXIT_SUCCESS);
    }
    <prepare file for reading 12>
    if (process_command_line) {
      <Count command-line measurements 20>
      <Calculate and write optical properties 13>
      exit(EXIT_SUCCESS);
    }
    if (Read_Header(stdin, &m, &params) ≠ 0) exit(EXIT_FAILURE);
    start_time = clock();
    while (Read_Data_Line(stdin, &m, &r, params) ≡ 0) {
      <Command-line changes to m 18>
      <Calculate and write optical properties 13>
    }
    <Generate and write grid 11>
    if (cl_verbosity > 0) fprintf(stderr, "\n\n");
    if (any_error ∧ cl_verbosity > 1) print_error_legend();
    exit(EXIT_SUCCESS);
  }

```

3. The first two defines are to stop Visual C++ from silly complaints

```
<Include files for main 3> ≡  
#define _CRT_SECURE_NO_WARNINGS  
#define _CRT_NONSTDC_NO_WARNINGS  
#define NO_SLIDES 0  
#define ONE_SLIDE_ON_TOP 1  
#define TWO_IDENTICAL_SLIDES 2  
#define ONE_SLIDE_ON_BOTTOM 3  
#define ONE_SLIDE_NEAR_SPHERE 4  
#define ONE_SLIDE_NOT_NEAR_SPHERE 5  
#define MR_IS_ONLY_RD 1  
#define MT_IS_ONLY_TD 2  
#define NO_UNSCATTERED_LIGHT 3  
#include <stdio.h>  
#include <string.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <time.h>  
#include <math.h>  
#include <ctype.h>  
#include <errno.h>  
#include "ad_globl.h"  
#include "ad_prime.h"  
#include "iad_type.h"  
#include "iad_pub.h"  
#include "iad_io.h"  
#include "iad_calc.h"  
#include "iad_util.h"  
#include "version.h"  
#include "mc_lost.h"  
#include "ad_frsl.h"
```

This code is used in section 2.

4. $\langle$ Declare variables for *main* 4 $\rangle \equiv$

```

struct measure_type m;
struct invert_type r;
char *g_out_name =  $\Lambda$ ;
char *g_grid_name =  $\Lambda$ ;
int c;
long n_photons = 100000;
int MAX_MC_iterations = 19;
int any_error = 0;
int process_command_line = 0;
int params = 0;
int cl_quadrature_points = UNINITIALIZED;
int cl_verbosity = 2;
double cl_forward_calc = UNINITIALIZED;
double cl_grid_calc = UNINITIALIZED;
double cl_default_a = UNINITIALIZED;
double cl_default_g = UNINITIALIZED;
double cl_default_b = UNINITIALIZED;
double cl_default_mua = UNINITIALIZED;
double cl_default_mus = UNINITIALIZED;
double cl_default_musp = UNINITIALIZED;
double cl_tolerance = UNINITIALIZED;
double cl_slide_OD = UNINITIALIZED;
double cl_cos_angle = UNINITIALIZED;
double cl_beam_d = UNINITIALIZED;
double cl_sample_d = UNINITIALIZED;
double cl_sample_n = UNINITIALIZED;
double cl_slide_d = UNINITIALIZED;
double cl_slide_n = UNINITIALIZED;
double cl_slides = UNINITIALIZED;
double cl_default_fr = UNINITIALIZED;
double cl_rstd_t = UNINITIALIZED;
double cl_rstd_r = UNINITIALIZED;
double cl_baffle_r = UNINITIALIZED;
double cl_baffle_t = UNINITIALIZED;
double cl_ru_fraction = UNINITIALIZED;
double cl_tu_fraction = UNINITIALIZED;
double cl_lambda = UNINITIALIZED;
double cl_rwall_r = UNINITIALIZED;
double cl_rwall_t = UNINITIALIZED;
double cl_search = UNINITIALIZED;
double cl_mus0 = UNINITIALIZED;
double cl_musp0 = UNINITIALIZED;
double cl_mus0_pwr = UNINITIALIZED;
double cl_mus0_lambda = UNINITIALIZED;
double cl_UR1 = UNINITIALIZED;
double cl_UT1 = UNINITIALIZED;
double cl_Tc = UNINITIALIZED;
double cl_method = UNINITIALIZED;
int cl_num_spheres = UNINITIALIZED;
double cl_sphere_one[5] = {UNINITIALIZED, UNINITIALIZED, UNINITIALIZED, UNINITIALIZED,
    UNINITIALIZED};

```

```

double cl_sphere_two[5] = {UNINITIALIZED, UNINITIALIZED, UNINITIALIZED, UNINITIALIZED,
    UNINITIALIZED};
double cl_wave_limit[2] = {UNINITIALIZED, UNINITIALIZED};
clock_t start_time = clock();
char command_line_options[] = "1:2:a:A:b:B:c:C:d:D:e:E:f:F:g:G:h:H:i:j:Jl:L:M:n:N:o:p:q:r:R\
:s:S:t:T:u:vV:w:W:x:Xz";
char *command_line = Λ;

```

This code is used in section 2.

5. I want to include the command line to the output file. To do this, we need to save the entire thing before the options get processed. The extra +1 in the total length calculation is for the space character between options. Finally, we need to reset *optind* to 1 to start *getopt()* processing from the beginning. It should be noted that this strips any quotes from the command line.

⟨ Save the command line for use later 5 ⟩ ≡

```

{
    size_t command_line_length = 0;
    for (int i = 0; i < argc; ++i) {
        command_line_length += strlen(argv[i]) + 3;
    }
    command_line = (char *) malloc(command_line_length);
    if (command_line == Λ) {
        fprintf(stderr, "Memory allocation failed\n");
        return 1;
    }
    strcpy(command_line, "");
    for (int i = 0; i < argc; ++i) {
        if (strchr(argv[i], ' ') != Λ) {
            strcat(command_line, " ");
            strcat(command_line, argv[i]);
            strcat(command_line, " ");
        }
        else {
            strcat(command_line, argv[i]);
            strcat(command_line, " ");
        }
    }
    optind = 1;
}

```

This code is used in section 2.

6. Handling the command-line options.

⟨Handle options 6⟩ ≡

```

while ((c = getopt(argc, argv, command_line_options)) ≠ EOF) {
    int n;
    char cc;
    char *tmp_str = Λ;
    switch (c) {
    case '1': tmp_str = strdup(optarg);
        parse_string_into_array(optarg, cl_sphere_one, 5);
        if (cl_sphere_one[4] ≡ UNINITIALIZED) {
            fprintf(stderr, "Error_in_the_command-line_argument_for_1\n");
            fprintf(stderr, "the_current_argument_is_%s'but_it_must_have_5_terms:", tmp_str);
            fprintf(stderr, "d_sphere_d_sample_d_entrance_d_detector_r_wall'\n");
            exit(EXIT_FAILURE);
        }
        break;
    case '2': tmp_str = strdup(optarg);
        parse_string_into_array(optarg, cl_sphere_two, 5);
        if (cl_sphere_two[4] ≡ UNINITIALIZED) {
            fprintf(stderr, "Error_in_the_command-line_argument_for_2\n");
            fprintf(stderr, "the_current_argument_is_%s'but_it_must_have_5_terms:", tmp_str);
            fprintf(stderr, "d_sphere_d_sample_d_third_d_detector_r_wall'\n");
            exit(EXIT_FAILURE);
        }
        break;
    case 'a': cl_default_a = my_strtod(optarg);
        if (cl_default_a < 0 ∨ cl_default_a > 1) {
            fprintf(stderr, "Error_in_the_command_line\n");
            fprintf(stderr, "albedo'-a%s'\n", optarg);
            exit(EXIT_FAILURE);
        }
        break;
    case 'A': cl_default_mua = my_strtod(optarg);
        if (cl_default_mua < 0) {
            fprintf(stderr, "Error_in_the_command_line\n");
            fprintf(stderr, "absorption'-A%s'\n", optarg);
            exit(EXIT_FAILURE);
        }
        break;
    case 'b': cl_default_b = my_strtod(optarg);
        if (cl_default_b < 0) {
            fprintf(stderr, "Error_in_the_command_line\n");
            fprintf(stderr, "optical_thickness'-b%s'\n", optarg);
            exit(EXIT_FAILURE);
        }
        break;
    case 'B': cl_beam_d = my_strtod(optarg);
        if (cl_beam_d < 0) {
            fprintf(stderr, "Error_in_the_command_line\n");
            fprintf(stderr, "beam_diameter'-B%s'\n", optarg);
            exit(EXIT_FAILURE);
        }
    }
}

```

```

    break;
case 'c': cl_ru_fraction = my_strtod(optarg);
    if (cl_ru_fraction < 0.0 ∨ cl_ru_fraction > 1.0) {
        fprintf(stderr, "Error in the command line\n");
        fprintf(stderr, "unscattered_refl_fraction '-c%s'\n", optarg);
        fprintf(stderr, "must be between 0 and 1\n");
        exit(EXIT_SUCCESS);
    }
    break;
case 'C': cl_tu_fraction = my_strtod(optarg);
    if (cl_tu_fraction < 0.0 ∨ cl_tu_fraction > 1.0) {
        fprintf(stderr, "Error in the command line\n");
        fprintf(stderr, "unscattered_trans_fraction '-C%s'\n", optarg);
        fprintf(stderr, "must be between 0 and 1\n");
        exit(EXIT_SUCCESS);
    }
    break;
case 'd': cl_sample_d = my_strtod(optarg);
    if (cl_sample_d < 0) {
        fprintf(stderr, "Error in the command line\n");
        fprintf(stderr, "sample_thickness '-d%s'\n", optarg);
        exit(EXIT_FAILURE);
    }
    break;
case 'D': cl_slide_d = my_strtod(optarg);
    if (cl_slide_d < 0) {
        fprintf(stderr, "Error in the command line\n");
        fprintf(stderr, "slide_thickness '-D%s'\n", optarg);
        exit(EXIT_FAILURE);
    }
    break;
case 'e': cl_tolerance = my_strtod(optarg);
    if (cl_tolerance < 0) {
        fprintf(stderr, "Error in the command line\n");
        fprintf(stderr, "error_tolerance '-e%s'\n", optarg);
        exit(EXIT_FAILURE);
    }
    break;
case 'E': cl_slide_OD = my_strtod(optarg);
    if (cl_slide_OD < 0) {
        fprintf(stderr, "Error in the command line\n");
        fprintf(stderr, "slide_optical_depth '-E%s'\n", optarg);
        exit(EXIT_FAILURE);
    }
    break;
case 'f': cl_default_fr = my_strtod(optarg);
    if (cl_default_fr < 0.0 ∨ cl_default_fr > 1.0) {
        fprintf(stderr, "Error in the command-line argument:\n");
        fprintf(stderr, "'-f%s' The argument must be between 0 and 1.\n", optarg);
        exit(EXIT_SUCCESS);
    }
    break;

```

```

case 'F': /* initial digit means this is mus is constant */
    if (isdigit(optarg[0])) {
        cl_default_mus = my_strtod(optarg);
        if (cl_default_mus < 0) {
            fprintf(stderr, "Error in the command line\n");
            fprintf(stderr, "mus -F%s'\n", optarg);
            exit(EXIT_FAILURE);
        }
        break;
    }
    /* should be a string like 'R 1000 1.2 -1.8' */
    n = sscanf(optarg, "%c%lf%lf%lf", &cc, &cl_mus0_lambda, &cl_mus0, &cl_mus0_pwr);
    if (n != 4 || (cc != 'P' ^ cc != 'R')) {
        fprintf(stderr, "Error in the command line\n");
        fprintf(stderr, "bad -F option -F%s'\n", optarg);
        fprintf(stderr, "mus -F1.0 for mus=1.0\n");
        fprintf(stderr, "mus -F'P5001.0-1.3' for mus=1.0*(lambda/500)^(-1.3)\n");
        fprintf(stderr, "mus -F'R5001.0-1.3' for mus=1.0*(lambda/500)^(-1.3)\n");
        exit(EXIT_FAILURE);
    }
    if (cc == 'R' || cc == 'r') {
        cl_mus0 = cl_mus0;
        cl_mus0 = UNINITIALIZED;
    }
    break;
case 'g': cl_default_g = my_strtod(optarg);
    if (cl_default_g < -1 || cl_default_g > 1) {
        fprintf(stderr, "Error in the command line\n");
        fprintf(stderr, "anisotropy -g%s'\n", optarg);
        exit(EXIT_FAILURE);
    }
    if (cl_default_g == -1) cl_default_g = -MAX_ABS_G;
    if (cl_default_g == 1) cl_default_g = MAX_ABS_G;
    break;
case 'G':
    if (optarg[0] == '0') cl_slides = NO_SLIDES;
    else if (optarg[0] == '2') cl_slides = TWO_IDENTICAL_SLIDES;
    else if (optarg[0] == 't' || optarg[0] == 'T') cl_slides = ONE_SLIDE_ON_TOP;
    else if (optarg[0] == 'b' || optarg[0] == 'B') cl_slides = ONE_SLIDE_ON_BOTTOM;
    else if (optarg[0] == 'n' || optarg[0] == 'N') cl_slides = ONE_SLIDE_NEAR_SPHERE;
    else if (optarg[0] == 'f' || optarg[0] == 'F') cl_slides = ONE_SLIDE_NOT_NEAR_SPHERE;
    else {
        fprintf(stderr, "Error in the command line\n");
        fprintf(stderr, "Argument for -G%s' must be\n", optarg);
        fprintf(stderr, "t --- light always hits top slide first\n");
        fprintf(stderr, "b --- light always hits bottom slide first\n");
        fprintf(stderr, "n --- slide always closest to sphere\n");
        fprintf(stderr, "f --- slide always farthest from sphere\n");
        exit(EXIT_FAILURE);
    }
    break;
case 'H':
    if (optarg[0] == '0') {

```

```

    cl_baffle_r = 0;
    cl_baffle_t = 0;
}
else if (optarg[0] == '1') {
    cl_baffle_r = 1;
    cl_baffle_t = 0;
}
else if (optarg[0] == '2') {
    cl_baffle_r = 0;
    cl_baffle_t = 1;
}
else if (optarg[0] == '3') {
    cl_baffle_r = 1;
    cl_baffle_t = 1;
}
else {
    fprintf(stderr, "Error in the command-line_H_argument\n");
    fprintf(stderr, "argument is '%s', but", optarg);
    fprintf(stderr, "must be 0, 1, 2, or 3\n");
    exit(EXIT_FAILURE);
}
case 'i': cl_cos_angle = my_strtod(optarg);
    if (cl_cos_angle < 0 ∨ cl_cos_angle > 90) {
        fprintf(stderr, "Error in the command_line\n");
        fprintf(stderr, "incident_angle '-i%s'\n", optarg);
        fprintf(stderr, "must be between 0 and 90 degrees\n");
        exit(EXIT_FAILURE);
    }
    cl_cos_angle = cos(cl_cos_angle * M_PI / 180.0);
    break;
case 'j': cl_default_musp = my_strtod(optarg);
    if (cl_default_musp < 0) {
        fprintf(stderr, "Error in the command_line\n");
        fprintf(stderr, "musp '-j%s'\n", optarg);
        exit(EXIT_FAILURE);
    }
    break;
case 'J': cl_grid_calc = 1;
    break;
case 'l': tmp_str = strdup(optarg);
    parse_string_into_array(optarg, cl_wave_limit, 2);
    break;
case 'L': cl_lambda = my_strtod(optarg);
    break;
case 'M': MAX_MC_iterations = (int) my_strtod(optarg);
    if (MAX_MC_iterations < 0 ∨ MAX_MC_iterations > 50) {
        fprintf(stderr, "Error in the command_line\n");
        fprintf(stderr, "MC_iterations '-M%s'\n", optarg);
        exit(EXIT_FAILURE);
    }
    break;
case 'n': cl_sample_n = my_strtod(optarg);

```

```

    if ( $cl\_sample\_n < 0.1 \vee cl\_sample\_n > 10$ ) {
        fprintf(stderr, "Error_in_the_command_line\n");
        fprintf(stderr, "slab_index'-n%s'\n", optarg);
        exit(EXIT_FAILURE);
    }
    break;
case 'N':  $cl\_slide\_n = my\_strtod(optarg)$ ;
    if ( $cl\_slide\_n < 0.1 \vee cl\_slide\_n > 10$ ) {
        fprintf(stderr, "Error_in_the_command_line\n");
        fprintf(stderr, "slide_index'-N%s'\n", optarg);
        exit(EXIT_FAILURE);
    }
    break;
case 'o':  $g\_out\_name = strdup(optarg)$ ;
    break;
case 'p':  $n\_photons = (long) my\_strtod(optarg)$ ;
    break;
case 'q':  $cl\_quadrature\_points = (int) my\_strtod(optarg)$ ;
    if ( $cl\_quadrature\_points \% 4 \neq 0$ ) {
        fprintf(stderr, "Error_in_the_command_line\n");
        fprintf(stderr, "'-q%s'\n", optarg);
        fprintf(stderr, "Quadrature_points_must_be_a_multiple_of_4\n");
        exit(EXIT_FAILURE);
    }
    if ( $((cl\_cos\_angle \neq UNINITIALIZED) \wedge (cl\_quadrature\_points \% 12 \neq 0))$ ) {
        fprintf(stderr, "Error_in_the_command_line\n");
        fprintf(stderr, "'-q%s'\n", optarg);
        fprintf(stderr,
            "Quadrature_points_must_be_multiple_of_12_for_oblique_incidence\n");
        exit(EXIT_FAILURE);
    }
    break;
case 'r':  $cl\_UR1 = my\_strtod(optarg)$ ;
     $process\_command\_line = 1$ ;
    if ( $cl\_UR1 < 0 \vee cl\_UR1 > 1$ ) {
        fprintf(stderr, "Error_in_the_command_line\n");
        fprintf(stderr, "UR1_value'-r%s'\n", optarg);
        fprintf(stderr, "must_be_between_0_and_1\n");
        exit(EXIT_FAILURE);
    }
    break;
case 'R':  $cl\_rstd\_r = my\_strtod(optarg)$ ;
    if ( $cl\_rstd\_r < 0 \vee cl\_rstd\_r > 1$ ) {
        fprintf(stderr, "Error_in_the_command_line\n");
        fprintf(stderr, "Rstd_value'-R%s'\n", optarg);
        fprintf(stderr, "must_be_between_0_and_1\n");
        exit(EXIT_FAILURE);
    }
    break;
case 's':  $cl\_search = (int) my\_strtod(optarg)$ ;
    break;
case 'S':  $cl\_num\_spheres = (int) my\_strtod(optarg)$ ;

```

```

    if (cl_num_spheres  $\neq$  0  $\wedge$  cl_num_spheres  $\neq$  1  $\wedge$  cl_num_spheres  $\neq$  2) {
        fprintf(stderr, "Error_in_the_command_line\n");
        fprintf(stderr, "sphere_number'-S%s'\n", optarg);
        fprintf(stderr, "must_be_0,1,or_2\n");
        exit(EXIT_FAILURE);
    }
    break;
case 't': cl_UT1 = my_strtod(optarg);
    if (cl_UT1 < 0  $\vee$  cl_UT1 > 1) {
        fprintf(stderr, "Error_in_the_command_line\n");
        fprintf(stderr, "UT1_value'-t%s'\n", optarg);
        fprintf(stderr, "must_be_between_0_and_1\n");
        exit(EXIT_FAILURE);
    }
    process_command_line = 1;
    break;
case 'T': cl_rstd_t = my_strtod(optarg);
    if (cl_rstd_t < 0  $\vee$  cl_rstd_t > 1) {
        fprintf(stderr, "Error_in_the_command_line\n");
        fprintf(stderr, "transmission_standard'-T%s'\n", optarg);
        fprintf(stderr, "must_be_between_0_and_1\n");
        exit(EXIT_FAILURE);
    }
    break;
case 'u': cl_Tc = my_strtod(optarg);
    if (cl_Tc < 0  $\vee$  cl_Tc > 1) {
        fprintf(stderr, "Error_in_the_command_line\n");
        fprintf(stderr, "unscattered_transmission'-u%s'\n", optarg);
        fprintf(stderr, "must_be_between_0_and_1\n");
        exit(EXIT_FAILURE);
    }
    process_command_line = 1;
    break;
case 'v': print_version(cl_verbosity);
    exit(EXIT_SUCCESS);
    break;
case 'V': cl_verbosity = my_strtod(optarg);
    break;
case 'w': cl_rwall_r = my_strtod(optarg);
    if (cl_rwall_r < 0  $\vee$  cl_rwall_r > 1) {
        fprintf(stderr, "Error_in_the_command_line\n");
        fprintf(stderr, "refl_sphere_wall'-w%s'\n", optarg);
        fprintf(stderr, "must_be_between_0_and_1\n");
        exit(EXIT_FAILURE);
    }
    break;
case 'W': cl_rwall_t = my_strtod(optarg);
    if (cl_rwall_t < 0  $\vee$  cl_rwall_r > 1) {
        fprintf(stderr, "Error_in_the_command_line\n");
        fprintf(stderr, "trans_sphere_wall'-W%s'\n", optarg);
        fprintf(stderr, "must_be_between_0_and_1\n");
        exit(EXIT_FAILURE);
    }

```

```
    }  
    break;  
case 'x': Set_Debugging((int) my_strtod(optarg));  
    break;  
case 'X': cl_method = COMPARISON;  
    break;  
case 'z': cl_forward_calc = 1;  
    process_command_line = 1;  
    break;  
default: fprintf(stderr, "unknown option '%c'\n", c); /* fall through */  
case 'h': print_usage();  
    exit(EXIT_SUCCESS);  
}  
}  
argc -= optind;  
argv += optind;
```

This code is used in section [2](#).

7. The forward calculation.

We are doing a forward calculation. This takes care of setting up the sample scattering and absorption coefficients when they are specified on the command line. It also deals with the case when the reduced scattering coefficient is specified. We carefully set up the default values (scattering is 1/mm, absorption is 0/mm, and anisotropy is 0.0).

```

⟨ Calculate and Print the Forward Calculation 7 ⟩ ≡
  double temp_mus = 1;
  double temp_mua = 0;

  r.g = 0;
  if (cl_default_mus ≠ UNINITIALIZED) temp_mus = cl_default_mus;
  if (cl_default_mua ≠ UNINITIALIZED) temp_mua = cl_default_mua;
  if (cl_default_g ≠ UNINITIALIZED) r.g = cl_default_g;
  if (cl_default_musp ≠ UNINITIALIZED) temp_mus = cl_default_musp/(1 - r.g);

```

See also sections 8, 9, and 10.

This code is used in section 2.

8. We still need to set the albedo and optical depth appropriately. Obviously when the $-a$ switch is used then the albedo must be set to equal to $cl_default_a$. It is also important to adjust the scattering and absorption coefficients in this case so that they match the albedo.

This is easy when both the optical thickness and the physical thickness of the sample are both given. If this not the case, then we just assume the scattering coefficient is correct and adjust the absorption coefficient to be whatever it needs to be. However in the case of $a = 0$ then we set the scattering coefficient to zero and the absorption coefficient to 1/mm.

```

⟨ Calculate and Print the Forward Calculation 7 ⟩ +≡
  if (cl_default_a ≠ UNINITIALIZED) {
    r.a = cl_default_a;
    if (cl_default_b ≠ UNINITIALIZED ∧ cl_sample_d ≠ UNINITIALIZED) {
      temp_mus = cl_default_a * cl_default_b / cl_sample_d;
      temp_mua = cl_default_b / cl_sample_d - temp_mus;
    }
    else {
      if (cl_default_a ≡ 0) {
        temp_mus = 0;
        temp_mua = 1;
      }
      else temp_mua = temp_mus / cl_default_a - temp_mus;
    }
  }
  else r.a = temp_mus / (temp_mus + temp_mua);

```

9. By this point the scattering coefficient and scattering coefficient are set correctly. These values will be used unless $-b$ has been used to set the optical thickness — $cl_default_b$. If the sample thickness is not specified, then the thickness is assumed infinite.

```

⟨ Calculate and Print the Forward Calculation 7 ⟩ +≡
  if (cl_default_b ≠ UNINITIALIZED) {
    r.b = cl_default_b;
  }
  else {
    if (cl_sample_d ≡ UNINITIALIZED) r.b = HUGE_VAL;
    else r.b = (temp_mus + temp_mua) * cl_sample_d;
  }

```

```

10.  ⟨ Calculate and Print the Forward Calculation 7 ⟩ +=
    r.slabs.a = r.a;
    r.slabs.b = r.b;
    r.slabs.g = r.g;
    {
        double mu_s, mu_sp, mu_a, m_r, m_t;
        if (MAX_MC_iterations == 0 ∨ m.num_spheres == 0) {
            Calculate_MR_MT(m, r, MC_NONE, TRUE, &m_r, &m_t);
        }
        else {
            Calculate_MR_MT(m, r, MC_REDO, TRUE, &m_r, &m_t);
        }
        Calculate_Mua_Musp(m, r, &mu_s, &mu_sp, &mu_a);
        if (cl_verbosity > 0) {
            Write_Header(m, r, -1, command_line);
            print_results_header(stdout);
        }
        print_optical_property_result(stdout, m, r, m_r, m_t, mu_a, mu_sp, 0);
    }

```

11. Calculating a grid for graphing.

We will start simple. Just vary a' and b' .

⟨Generate and write grid 11⟩ ≡

```

if (cl_grid_calc ≠ UNINITIALIZED) {
    double m_r, m_t, aprime, bprime, g;
    double aa[] = {0, 0.8, 0.9, 0.95, 0.98, 0.99, 1.0};
    double bb[] = {0, 0.2, 0.5, 1.0, 3.0, 10.0, 100};
    int i, j;
    int count = 0;
    FILE *grid;

    grid = fopen(g_grid_name, "w");
    if (grid ≡ Λ) {
        fprintf(stderr, "Could not open grid file '%s' for output\n", g_out_name);
        exit(EXIT_FAILURE);
    }
    m_ur1_lost = 0;
    m_uru_lost = 0;
    m_ut1_lost = 0;
    m_utu_lost = 0;
    if (r.default_g ≠ UNINITIALIZED) {
        g = r.default_g;
    }
    else if (r.found) {
        g = r.slabs.g;
    }
    else {
        g = 0;
    }
    fprintf(grid, "#_#s_(g=%6.4f)\n", command_line, g);
    fprintf(grid, "#_a' _b' _g _M_R _M_T\n");
    fprintf(stderr, "\ndoing grid calculation\n");
    for (i = 0; i < 7; i++) {
        aprime = aa[i];
        for (j = 0; j < 7; j++) {
            fprintf(stderr, "*");
            bprime = bb[j];
            r.a = aprime / (1 - g + aprime * g);
            r.b = bprime / (1 - r.slabs.a * g);
            r.g = g;
            r.slabs.a = r.a;
            r.slabs.b = r.b;
            r.slabs.g = r.g;
            if (MAX_MC_iterations ≡ 0) {
                Calculate_MR_MT(m, r, MC_NONE, TRUE, &m_r, &m_t);
            }
            else {
                Calculate_MR_MT(m, r, MC_REDO, TRUE, &m_r, &m_t);
            }
            count++;
            if (count % 10 ≡ 0) fprintf(stderr, "\n");
            if (count % 50 ≡ 0) fprintf(stderr, "\n");
            fprintf(grid, "%10.5f, _%10.5f, _%10.5f, _%10.5f, _%10.5f\n", 39aprime, bprime, g, m_r, m_t);
        }
    }
}

```

```

    }
}
fclose(grid);
fprintf(stderr, "\n");
}

```

This code is used in section 2.

12. Make sure that the file is not named '-' and warn about too many files

⟨prepare file for reading 12⟩ ≡

```

if (argc > 1) {
    fprintf(stderr, "Only a single file can be processed at a time\n");
    fprintf(stderr, "try 'applyiad file1 file2 ... fileN'\n");
    exit(EXIT_FAILURE);
}
if (argc == 1 ∧ strcmp(argv[0], "-") ≠ 0) { /* filename exists and != "-" */
    int n;
    char *base_name, *rt_name;
    base_name = strdup(argv[0]);
    n = (int)(strlen(base_name) - strlen(".rxt"));
    if (n > 0 ∧ strstr(base_name + n, ".rxt") ≠ Λ) base_name[n] = '\0';
    rt_name = strdup_together(base_name, ".rxt");
    if (freopen(argv[0], "r", stdin) ≡ Λ ∧ freopen(rt_name, "r", stdin) ≡ Λ) {
        fprintf(stderr, "Could not open either '%s' or '%s'\n", argv[0], rt_name);
        exit(EXIT_FAILURE);
    }
    if (g_out_name ≡ Λ) g_out_name = strdup_together(base_name, ".txt");
    if (g_grid_name ≡ Λ) g_grid_name = strdup_together(base_name, ".grid");
    free(rt_name);
    free(base_name);
    process_command_line = 0;
}
if (g_out_name ≠ Λ) {
    if (freopen(g_out_name, "w", stdout) ≡ Λ) {
        fprintf(stderr, "Could not open file '%s' for output\n", g_out_name);
        exit(EXIT_FAILURE);
    }
}
}

```

This code is used in section 2.

13. Need to explicitly reset *r.search* each time through the loop, because it will get altered by the calculation process. This also allows the command line to overwrite the reflection or transmission value from the command line with something like `-r 0` or `-t 0`.

We also want to be able to let different lines have different constraints. In particular consider the file `newton.tst`. In that file the first two rows contain three real measurements and the last two have the collimated transmission explicitly set to zero — in other words there are really only two measurements.

```

< Calculate and write optical properties 13 > ≡
{
  < Local Variables for Calculation 14 >
  if (cl_wave_limit[0] ≠ UNINITIALIZED) {
    if (m.lambda ≠ 0) {
      if (m.lambda < cl_wave_limit[0]) skip = TRUE;
      if (m.lambda > cl_wave_limit[1]) skip = TRUE;
    }
  }
  if (Debug(DEBUG_ANY) ∧ ¬skip) {
    fprintf(stderr, "\n-----NEXT_DATA_POINT-----\n");
    if (m.lambda ≠ 0) fprintf(stderr, "lambda=%6.1f", m.lambda);
    fprintf(stderr, "MR=%8.5f MT=%8.5f\n\n", m.m_r, m.m_t);
    if (skip) fprintf(stderr, "skipping, wavelength out of range.\n");
  }
  if (¬skip) {
    rt_total++;
    Initialize_Result(m, &r, FALSE);
    < Command-line changes to r 15 >
    < Warn and quit for bad options 19 >
    < Write Header 16 >
    m.ur1_lost = 0;
    m.uru_lost = 0;
    m.ut1_lost = 0;
    m.utu_lost = 0;
    Inverse_RT(m, &r);
    < Improve result using Monte Carlo 17 >
    calculate_coefficients(m, r, &LR, &LT, &mu_sp, &mu_a);
    print_optical_property_result(stdout, m, r, LR, LT, mu_a, mu_sp, rt_total);
    if (r.error ≠ IAD_NO_ERROR) any_error = 1;
    if (Debug(DEBUG_ANY)) print_long_error(r.error);
    else print_dot(start_time, r.error, mc_total, TRUE, cl_verbosity);
  }
}

```

This code is used in section 2.

14.

〈Local Variables for Calculation 14〉≡

```

static int rt_total = 0;
static int mc_total = 0;
double ur1 = 0;
double ut1 = 0;
double uru = 0;
double utu = 0;
double mu_a = 0;
double mu_sp = 0;
double LR = 0;
double LT = 0;
int skip = FALSE;

```

This code is used in section 13.

15. 〈Command-line changes to r 15〉≡

```

if (cl_quadrature_points ≠ UNINITIALIZED) r.method.quad_pts = cl_quadrature_points;
else r.method.quad_pts = 8;
if (cl_default_a ≠ UNINITIALIZED) r.default_a = cl_default_a;
if (cl_default_mua ≠ UNINITIALIZED) {
    r.default_mua = cl_default_mua;
    if (cl_sample_d ≠ UNINITIALIZED) r.default_ba = cl_default_mua * cl_sample_d;
    else r.default_ba = cl_default_mua * m.slab_thickness;
}
if (cl_default_b ≠ UNINITIALIZED) r.default_b = cl_default_b;
if (cl_default_g ≠ UNINITIALIZED) r.default_g = cl_default_g;
if (cl_tolerance ≠ UNINITIALIZED) {
    r.tolerance = cl_tolerance;
    r.MC_tolerance = cl_tolerance;
}
if (cl_musp0 ≠ UNINITIALIZED)
    cl_mus0 = (r.default_g ≠ UNINITIALIZED) ? cl_musp0 / (1 - r.default_g) : cl_musp0;
if (cl_mus0 ≠ UNINITIALIZED ∧ m.lambda ≠ 0)
    cl_default_mus = cl_mus0 * pow(m.lambda / cl_mus0_lambda, cl_mus0_pwr);
if (cl_default_mus ≠ UNINITIALIZED) {
    r.default_mus = cl_default_mus;
    if (cl_sample_d ≠ UNINITIALIZED) r.default_bs = cl_default_mus * cl_sample_d;
    else r.default_bs = cl_default_mus * m.slab_thickness;
}
if (cl_default_musp ≠ UNINITIALIZED) {
    if (cl_default_g ≠ UNINITIALIZED) r.default_mus = cl_default_musp / (1.0 - cl_default_g);
    else r.default_mus = cl_default_musp;
    if (cl_sample_d ≠ UNINITIALIZED) r.default_bs = r.default_mus * cl_sample_d;
    else r.default_bs = r.default_mus * m.slab_thickness;
}
if (cl_search ≠ UNINITIALIZED) r.search = cl_search;

```

This code is used in sections 2 and 13.

16. $\langle \text{Write Header } 16 \rangle \equiv$

```

if ( $rt\_total \equiv 1 \wedge cl\_verbosity > 0$ ) {
  Write_Header( $m, r, params, command\_line$ );
  if ( $MAX\_MC\_iterations > 0$ ) {
    if ( $n\_photons \geq 0$ )
      fprintf(stdout, "#_Photons_used_to_estimate_lost_light=_%ld\n",  $n\_photons$ );
    else fprintf(stdout, "#_Time_used_to_estimate_lost_light=_%ld_ms\n",  $-n\_photons$ );
  }
  else fprintf(stdout, "#_Photons_used_to_estimate_lost_light=_%ld\n", 0);
  fprintf(stdout, "#\n");
  print_results_header(stdout);
}

```

This code is used in section 13.

17. Monte Carlo light loss. Use Monte Carlo to figure out how much light leaks out. We use the sphere corrected values as the starting values and only do try Monte Carlo when spheres are used, the albedo unknown or non-zero, and there has been no error. The sphere parameters must be known because otherwise the beam size and the port size are unknown.

⟨Improve result using Monte Carlo 17⟩ ≡

```

if (m.num_spheres > 0) {
  if (Debug(DEBUG_LOST_LIGHT)) {
    print_results_header(stderr);
    print_optical_property_result(stderr, m, r, LR, LT, mu_a, mu_sp, rt_total);
  }
  while (r.MC_iterations < MAX_MC_iterations) {
    double last_mu_sp, last_mu_a, last_final_distance;
    calculate_coefficients(m, r, &LR, &LT, &mu_sp, &mu_a);
    last_mu_sp = mu_sp;
    last_mu_a = mu_a;
    last_final_distance = r.final_distance;
    if (Debug(DEBUG_ITERATIONS) ∨ Debug(DEBUG_A_LITTLE)) {
      fprintf(stderr, "\n-----Monte Carlo Iteration %d-----\n",
        r.MC_iterations + 1);
    }
    MC_Lost(m, r, n_photons, &ur1, &ut1, &uru, &utu, &m.ur1_lost, &m.ut1_lost, &m.uru_lost,
      &m.utu_lost);
    mc_total++;
    r.MC_iterations++;
    Inverse_RT(m, &r);
    calculate_coefficients(m, r, &LR, &LT, &mu_sp, &mu_a);
    if (r.found) {
      if (fabs(last_mu_a - mu_a) < r.MC_tolerance ∧ fabs(last_mu_sp - mu_sp) < r.MC_tolerance) {
        break;
      }
      else {
        if (Debug(DEBUG_ITERATIONS))
          fprintf(stderr, "Repeat MC because mua and musp are still changing\n");
      }
    }
    else {
      if (last_final_distance - r.final_distance < r.MC_tolerance) {
        if (Debug(DEBUG_ITERATIONS)) fprintf(stderr, "MC does not make things better\n");
        break;
      }
      else {
        if (Debug(DEBUG_ITERATIONS))
          fprintf(stderr, "Repeat MC because distance is reduced\n");
      }
    }
  }
  if (Debug(DEBUG_LOST_LIGHT))
    print_optical_property_result(stderr, m, r, LR, LT, mu_a, mu_sp, rt_total);
  else print_dot(start_time, r.error, mc_total, FALSE, cl_verbosity);
}

```

This code is used in section 13.

18. Stuff the command-line arguments that should be constant over the entire inversion process into the measurement record and set up the result record to handle the arguments properly so that the optical properties can be determined.

⟨ Command-line changes to *m* 18 ⟩ ≡

```

if (cl_cos_angle ≠ UNINITIALIZED) {
    m_slab_cos_angle = cl_cos_angle;
    if (cl_quadrature_points ≡ UNINITIALIZED) cl_quadrature_points = 12;
    if (cl_quadrature_points ≠ 12 * (cl_quadrature_points/12)) {
        fprintf(stderr,
            "If you use the -i option to specify an oblique incidence angle, then\n");
        fprintf(stderr, "the number of quadrature points must be a multiple of 12\n");
        exit(EXIT_SUCCESS);
    }
}
if (cl_sample_n ≠ UNINITIALIZED) m_slab_index = cl_sample_n;
if (cl_slide_n ≠ UNINITIALIZED) {
    m_slab_bottom_slide_index = cl_slide_n;
    m_slab_top_slide_index = cl_slide_n;
}
if (cl_slide_OD ≠ UNINITIALIZED) {
    m_slab_bottom_slide_b = cl_slide_OD;
    m_slab_top_slide_b = cl_slide_OD;
}
if (cl_sample_d ≠ UNINITIALIZED) m_slab_thickness = cl_sample_d;
if (cl_beam_d ≠ UNINITIALIZED) m_d_beam = cl_beam_d;
if (cl_slide_d ≠ UNINITIALIZED) {
    m_slab_bottom_slide_thickness = cl_slide_d;
    m_slab_top_slide_thickness = cl_slide_d;
}
if (cl_slides ≡ NO_SLIDES) {
    m_slab_bottom_slide_index = 1.0;
    m_slab_bottom_slide_thickness = 0.0;
    m_slab_top_slide_index = 1.0;
    m_slab_top_slide_thickness = 0.0;
}
if (cl_slides ≡ ONE_SLIDE_ON_TOP ∨ cl_slides ≡ ONE_SLIDE_NEAR_SPHERE) {
    m_slab_bottom_slide_index = 1.0;
    m_slab_bottom_slide_thickness = 0.0;
}
if (cl_slides ≡ ONE_SLIDE_ON_BOTTOM ∨ cl_slides ≡ ONE_SLIDE_NOT_NEAR_SPHERE) {
    m_slab_top_slide_index = 1.0;
    m_slab_top_slide_thickness = 0.0;
}
if (cl_slides ≡ ONE_SLIDE_NEAR_SPHERE ∨ cl_slides ≡ ONE_SLIDE_NOT_NEAR_SPHERE) m_flip_sample = 1;
else m_flip_sample = 0;
if (cl_method ≠ UNINITIALIZED) m_method = (int) cl_method;
if (cl_rstd_t ≠ UNINITIALIZED) m_rstd_t = cl_rstd_t;
if (cl_rstd_r ≠ UNINITIALIZED) m_rstd_r = cl_rstd_r;
if (cl_rwall_r ≠ UNINITIALIZED) m_rw_r = cl_rwall_r;
if (cl_rwall_t ≠ UNINITIALIZED) m_rw_t = cl_rwall_t;
if (cl_sphere_one[0] ≠ UNINITIALIZED) {
    double d_sample_r, d_third_r, d_detector_r;

```

```

    m.d_sphere_r = cl_sphere_one[0];
    d_sample_r = cl_sphere_one[1];
    d_third_r = cl_sphere_one[2];
    d_detector_r = cl_sphere_one[3];
    m.rw_r = cl_sphere_one[4];
    m.as_r = (d_sample_r/m.d_sphere_r/2) * (d_sample_r/m.d_sphere_r/2);
    m.at_r = (d_third_r/m.d_sphere_r/2) * (d_third_r/m.d_sphere_r/2);
    m.ad_r = (d_detector_r/m.d_sphere_r/2) * (d_detector_r/m.d_sphere_r/2);
    m.aw_r = 1.0 - m.as_r - m.at_r - m.ad_r;
    m.d_sphere_t = m.d_sphere_r;
    m.as_t = m.as_r;
    m.at_t = m.at_r;
    m.ad_t = m.ad_r;
    m.aw_t = m.aw_r;
    m.rw_t = m.rw_r;
    if (cl_num_spheres == UNINITIALIZED) m.num_spheres = 1;
}
if (cl_sphere_two[0] != UNINITIALIZED) {
    double d_sample_t, d_third_t, d_detector_t;
    m.d_sphere_t = cl_sphere_two[0];
    d_sample_t = cl_sphere_two[1];
    d_third_t = cl_sphere_two[2];
    d_detector_t = cl_sphere_two[3];
    m.rw_t = cl_sphere_two[4];
    m.as_t = (d_sample_t/m.d_sphere_t/2) * (d_sample_t/m.d_sphere_t/2);
    m.at_t = (d_third_t/m.d_sphere_t/2) * (d_third_t/m.d_sphere_t/2);
    m.ad_t = (d_detector_t/m.d_sphere_t/2) * (d_detector_t/m.d_sphere_t/2);
    m.aw_t = 1.0 - m.as_t - m.at_t - m.ad_t;
    if (cl_num_spheres == UNINITIALIZED) m.num_spheres = 2;
}
if (cl_num_spheres != UNINITIALIZED) {
    m.num_spheres = (int) cl_num_spheres;
    if (m.num_spheres > 0 & m.method == UNKNOWN) m.method = SUBSTITUTION;
}
if (cl_ru_fraction != UNINITIALIZED) m.fraction_of_ru_in_mr = cl_ru_fraction;
if (cl_tu_fraction != UNINITIALIZED) m.fraction_of_tu_in_mt = cl_tu_fraction;
if (cl_UR1 != UNINITIALIZED) m.m_r = cl_UR1;
if (cl_UT1 != UNINITIALIZED) m.m_t = cl_UT1;
if (cl_Tc != UNINITIALIZED) m.m_u = cl_Tc;
if (cl_default_fr != UNINITIALIZED) m.f_r = cl_default_fr;
if (cl_baffle_r != UNINITIALIZED) m.baffle_r = cl_baffle_r;
if (cl_baffle_t != UNINITIALIZED) m.baffle_t = cl_baffle_t;
if (cl_lambda != UNINITIALIZED) m.lambda = cl_lambda;

```

This code is used in section 2.

19. $\langle$ Warn and quit for bad options 19 $\rangle \equiv$

```

if (cl_method  $\equiv$  COMPARISON  $\wedge$  m.d_sphere_r  $\neq$  0  $\wedge$  m.as_r  $\equiv$  0) {
    fprintf(stderr, "A_dual-beam_measurement_is_specified,_but_no_port_sizes.\n");
    fprintf(stderr, "You_might_forsake_the_X_option_and_use_zero_spheres_(which_gives\n");
    fprintf(stderr, "the_same_result_except_lost_light_is_not_taken_into_account).\n");
    fprintf(stderr, "Alternatively,_bite_the_bullet_and_enter_your_sphere_parameters,\n");
    fprintf(stderr, "with_the_knowledge_that_only_the_beam_diameter_and_sample_port\n");
    fprintf(stderr, "diameter_will_be_used_to_estimate_lost_light_from_the_edges.\n");
    exit(EXIT_SUCCESS);
}
if (cl_method  $\equiv$  COMPARISON  $\wedge$  m.num_spheres  $\equiv$  2) {
    fprintf(stderr, "A_dual-beam_measurement_is_specified,_but_a_two_sphere_experiment\n");
    fprintf(stderr, "is_specified._Since_this_seems_impossible,_I_will_make_it\n");
    fprintf(stderr, "impossible_for_you_unless_you_specify_0_or_1_sphere.\n");
    exit(EXIT_SUCCESS);
}
if (cl_method  $\equiv$  COMPARISON  $\wedge$  m.f_r  $\neq$  0) {
    fprintf(stderr, "A_dual-beam_measurement_is_specified,_but_a_fraction_of_light\n");
    fprintf(stderr, "is_specified_to_hit_the_sphere_wall_first._This_situation\n");
    fprintf(stderr, "is_not_supported_by_iad._Sorry.\n");
    exit(EXIT_SUCCESS);
}

```

This code is used in section 13.

20. Put the command-line values for reflection and transmission into the measurement record.

 $\langle$ Count command-line measurements 20 $\rangle \equiv$

```

m.num_measures = 3;
if (m.m_r  $\equiv$  0) m.num_measures--;
if (m.m_t  $\equiv$  0) m.num_measures--;
if (m.m_u  $\equiv$  0) m.num_measures--;
params = m.num_measures;

```

This code is used in section 2.

21. $\langle$ print version function 21 $\rangle \equiv$

```

static void print_version(int verbosity)
{
    if (verbosity  $\equiv$  0) {
        fprintf(stdout, "%s", VersionShort);
    }
    else {
        fprintf(stdout, "iad%s\n", Version);
        fprintf(stdout, "Copyright_1993-2024_Scott_Prahl,_scott.prahl@oit.edu\n");
        fprintf(stdout, "((((((((((((see_Applied_Optics,_32:559-568,_1993))\n\n");
        fprintf(stdout, "This_is_free_software;_see_the_source_for_copying_conditions.\n");
        fprintf(stdout, "There_is_no_warranty;_not_even_for_MERCHANTABILITY_or_FITNESS.\n");
        fprintf(stdout, "FOR_A_PARTICULAR_PURPOSE.\n");
    }
}

```

This code is used in section 2.

22. `<print usage function 22> ≡`

```
static void print_usage(void)
{
    fprintf(stdout, "iad_%s\n\n", Version);
    fprintf(stdout, "iad_finds_optical_properties_from_measurements\n\n");
    fprintf(stdout, "Usage: iad [options] input\n\n");
    fprintf(stdout, "Options:\n");
    fprintf(stdout, "uu-1_#_#_#_#_# reflection_sphere_parameters\n");
    fprintf(stdout, "          'd_sphere_d_sample_port_d_entrance_por\
t_d_detector_port_r_wall'\n");
    fprintf(stdout, "uu-2_#_#_#_#_# transmission_sphere_parameters\n");
    fprintf(stdout, "          'd_sphere_d_sample_port_d_third_port_d\
_detector_port_r_wall'\n");
    fprintf(stdout, "uu-a_# use_this_albedo\n");
    fprintf(stdout, "uu-A_# use_this_absorption_coefficient\n");
    fprintf(stdout, "uu-b_# use_this_optical_thickness\n");
    fprintf(stdout, "uu-B_# beam_diameter\n");
    fprintf(stdout, "uu-c_# fraction_of_unscattered_refl_in_MR\n");
    fprintf(stdout, "uu-C_# fraction_of_unscattered_trans_in_MT\n");
    fprintf(stdout, "uu-d_# thickness_of_sample\n");
    fprintf(stdout, "uu-D_# thickness_of_slide\n");
    fprintf(stdout, "uu-e_# error_tolerance(default_0.0001)\n");
    fprintf(stdout, "uu-E_# optical_depth(=mua*D)for_slides\n");
    fprintf(stdout,
        "uu-f_# allow_a_fraction_0.0-1.0_of_light_to_hit_sphere_wall_first\n");
    fprintf(stdout, "uu-F_# constrain_scattering_coefficient\n");
    fprintf(stdout, "          #_=constant: use_constant_scattering_coefficient\n");
    fprintf(stdout, "          #_=P_lambda0_mus0_gamma' then_mus=mus0\
*(lambda/lambda0)^gamma\n");
    fprintf(stdout, "          #_=R_lambda0_musp0_gamma' musp=musp0*\
(lambda/lambda0)^gamma\n");
    fprintf(stdout, "uu-g_# scattering_anisotropy(default_0)\n");
    fprintf(stdout, "uu-G_# type_of_boundary_0',_2',_t',_b',_n',_f'\n");
    fprintf(stdout, "          '0'or_'2'-----number_of_slides\n");
    fprintf(stdout, "          't'(top)or_'b'(bottom)---one_slide\
that_is_hit_by_light_first\n");
    fprintf(stdout, "          'n'(near)or_'f'(far)---one_slide\
position_relative_to_sphere\n");
    fprintf(stdout, "uu-h display_help\n");
    fprintf(stdout, "uu-H_# #_=0, no baffles_for_R_or_T_spheres\n");
    fprintf(stdout, "          #_=1, baffle_for_R_but_not_for_T_sphere\n");
    fprintf(stdout, "          #_=2, baffle_for_T_but_not_for_R_sphere\n");
    fprintf(stdout, "          #_=3, baffle_for_both_R_and_T_spheres(default)\n");
    fprintf(stdout, "uu-i_# incident_angle_in_degrees\n");
    fprintf(stdout, "uu-j_# constrain_reduced_scattering_coefficient\n");
    fprintf(stdout, "uu-J generate_grid_after_inverse_calculation\n");
    fprintf(stdout, "uu-l_# wavelength_limits\n");
    fprintf(stdout, "uu-L_# specify_the_wavelength_lambda\n");
    fprintf(stdout, "uu-M_# limit_number_of_MonteCarlo_iterations\n");
    fprintf(stdout, "uu-n_# specify_index_of_refraction_of_slab\n");
    fprintf(stdout, "uu-N_# specify_index_of_refraction_of_slides\n");
    fprintf(stdout, "uu-o filename_explicitly_specify_filename_for_output\n");
}
```

```

fprintf(stdout, "p# of Monte Carlo photons (default 100000)\n");
fprintf(stdout, "a negative number is max MC time in milliseconds\n");
fprintf(stdout, "q# number of quadrature points (default=8)\n");
fprintf(stdout, "r# total reflection measurement\n");
fprintf(stdout, "R# actual reflectance for 100% measurement\n");
fprintf(stdout, "s# specify type of search to do\n");
fprintf(stdout, "S# number of spheres used\n");
fprintf(stdout, "t# total transmission measurement\n");
fprintf(stdout, "T# actual transmission for 100% measurement\n");
fprintf(stdout, "u# unscattered transmission measurement\n");
fprintf(stdout, "v# version information\n");
fprintf(stdout, "V0# verbosity low --- no output to stdout\n");
fprintf(stdout, "V1# verbosity moderate\n");
fprintf(stdout, "V2# verbosity high\n");
fprintf(stdout, "w# wall reflectivity for reflection sphere\n");
fprintf(stdout, "W# wall reflectivity for transmission sphere\n");
fprintf(stdout, "x# set debugging level\n");
fprintf(stdout, "X# dual beam configuration\n");
fprintf(stdout, "z# do forward calculation\n");
fprintf(stdout, "Examples:\n");
fprintf(stdout, "iadiad file.rxt Results will be put in file.txt\n");
fprintf(stdout, "iadiad file Same as above\n");
fprintf(stdout, "iadiad-c0.9 file.rxt Assume M_R includes 90% of unscattered reflectance\n");
fprintf(stdout, "iadiad-C0.8 file.rxt Assume M_T includes 80% of unscattered transmittance\n");
fprintf(stdout, "iadiad-e0.0001 file.rxt Better convergence to R & T values\n");
fprintf(stdout, "iadiad-f1.0 file.rxt All light hits reflectance sphere wall first\n");
fprintf(stdout, "iadiad-l'500'600' file.rxt Only do wavelengths between 500 and 600\n");
fprintf(stdout, "iadiad-o out file.rxt Calculated values in/out\n");
fprintf(stdout, "iadiad-r0.3 R_total=0.3, b=inf, find albedo\n");
fprintf(stdout, "iadiad-r0.3-t0.4 R_total=0.3, T_total=0.4, find a,b,g\n");
fprintf(stdout, "iadiad-r0.3-t0.4-n1.5 R_total=0.3, T_total=0.4, n=1.5, find a,b\n");
fprintf(stdout, "iadiad-r0.3-t0.4 R_total=0.3, T_total=0.4, find a,b\n");
fprintf(stdout, "iadiad-p1000 file.rxt Only 1000 photons\n");
fprintf(stdout, "iadiad-p-100 file.rxt Allow only 100ms per iteration\n");
fprintf(stdout, "iadiad-q4 file.rxt Four quadrature points\n");
fprintf(stdout, "iadiad-M0 file.rxt No MC (iad)\n");
fprintf(stdout, "iadiad-M1 file.rxt MC once (iad->MC->iad)\n");
fprintf(stdout, "iadiad-M2 file.rxt MC twice (iad->MC->iad->MC->iad)\n");
fprintf(stdout, "iadiad-M0-q4 file.rxt Fast and crude conversion\n");
fprintf(stdout, "iadiad-Gut file.rxt One top slide with properties from file.rxt\n");
fprintf(stdout, "iadiad-Gb-N1.5-D1 file Use 1 bottom slide with n=1.5 and thickness=1\n");
fprintf(stdout, "iadiad-x1 file.rxt Show sphere and MC effects\n");
fprintf(stdout, "iadiad-x2 file.rxt Show grid decisions\n");
fprintf(stdout, "iadiad-x4 file.rxt Show iterations\n");
fprintf(stdout, "iadiad-x8 file.rxt Show lost light effects\n");
fprintf(stdout, "iadiad-x16 file.rxt Show best grid points\n");
fprintf(stdout, "iadiad-x32 file.rxt Show decisions for type of search\n");

```

```

fprintf(stdout, "iadx64.file.rxtShowallgridcalculations\n");
fprintf(stdout, "iadx128.file.rxtShowspherecalculations\n");
fprintf(stdout, "iadx256.file.rxtDEBUG_EVERY_CALC\n");
fprintf(stdout, "iadx511.file.rxtShowalldebuggingoutput\n");
fprintf(stdout,
        "iadx-X-i8.file.rxtDualbeam_spectrometer_with_8_degree_incidence\n\n");
fprintf(stdout,
        "iadx-z-a0.9-b1-i45Forward_calc_assuming_45_degree_incidence\n\n");
fprintf(stdout, "applyiadx.rxt.y.rxtProcessmultiplefiles\n\n");
fprintf(stdout, "Report_bugs_to<scott.prahl@oit.edu>\n\n");
}

```

This code is used in section 2.

23. This can only be called immediately after *Inverse_RT* You have been warned! Notice that *Calculate_Distance* does not pass any slab properties.

⟨calculate coefficients function 23⟩ ≡

```

static void calculate_coefficients(struct measure_type m, struct invert_type r, double *LR, double
        *LT, double *musp, double *mua)
{
    double delta, mus;
    *LR = 0;
    *LT = 0;
    if (r.found ∨ (¬r.found ∧ r.error ≡ IAD_TOO_MANY_ITERATIONS)) {
        Calculate_Distance(LR, LT, &delta);
        Calculate_Mua_Musp(m, r, &mus, musp, mua);
    }
    else {
        *musp = 0;
        *mua = 0;
    }
}

```

This code is used in section 2.

```
static void print_results_header(FILE *fp)
{
```

```

if (Debug(DEBUG_LOST_LIGHT)) {
    fprintf(fp, "#_Meas_M_R_Meas_M_T_calccalccalcc|");
    fprintf(fp, "_LostLostLostLost_MC_IAD_Error\n");
    fprintf(fp, "#_wave_M_R_fit_M_T_fit_mu_a_mu_s'gg|");
    fprintf(fp, "UR1URUUT1UTU|#_#Type\n");
    fprintf(fp, "#_nm_---_---_---_---_1/mm_---_---_");
    fprintf(fp, "_____n");
    fprintf(fp, "#-----");
    fprintf(fp, "-----\n");
}
else {
    fprintf(fp, "#_Measured_tMeasured_tMeasured_tEstimat\
ed_tEstimated_tEstimated");
    if (Debug(DEBUG_LOST_LIGHT)) fprintf(fp, "\tLostLostLostLostLo\
st_MC_IAD_Error");
    fprintf(fp, "\n");
    fprintf(fp, "##wave_tM_R_tfit_tM_T_tfit_tmu_a\
_mu_a_mu_s't_gg");
    if (Debug(DEBUG_LOST_LIGHT)) fprintf(fp, "\tUR1URUUT1UTU\
T_#_#_State");
    fprintf(fp, "\n");
    fprintf(fp, "#_[nm]_t_---_---_---_---_t_---_---_t_1/mm\
_---_---_");
    if (Debug(DEBUG_LOST_LIGHT)) fprintf(fp, "\t_---_---_---_---_-\
_---_---_---_---_");
    fprintf(fp, "\n");
}
}

```

This code is used in section 2.

25. When debugging lost light, it is handy to see how each iteration changes the calculated values for the optical properties. We do that here if we are debugging, otherwise we just print a number or something to keep the user from wondering what is going on.

⟨Print results function 25⟩ ≡

```

void print_optical_property_result(FILE *fp, struct measure_type m, struct invert_type r, double
    LR, double LT, double mu_a, double mu_sp, int line)
{
    if (Debug(DEBUG_LOST_LIGHT)) {
        if (m.lambda ≠ 0) fprintf(fp, "%6.1f\_\_\_\", m.lambda);
        else fprintf(fp, "%6d\_\_\_\", line);
        if (mu_a ≥ 200) mu_a = 199.9999;
        if (mu_sp ≥ 1000) mu_sp = 999.9999;
        fprintf(fp, "%6.4f\_\%6.4f\_\|_\", m.m_r, LR);
        fprintf(fp, "%6.4f\_\%6.4f\_\|_\", m.m_t, LT);
        fprintf(fp, "%6.3f\_\", mu_a);
        fprintf(fp, "%6.3f\_\", mu_sp);
        fprintf(fp, "%6.3f\_|", r.g);
        fprintf(fp, "\_6.4f\_\%6.4f\_\", m.ur1_lost, m.uru_lost);
        fprintf(fp, "%6.4f\_\%6.4f\_\|_\", m.ut1_lost, m.utu_lost);
        fprintf(fp, "%2d\_\_\", r.MC_iterations);
        fprintf(fp, "%3d", r.AD_iterations);
        fprintf(fp, "\_\_\_\_c\_\n", what_char(r.error));
    }
    else {
        if (m.lambda ≠ 0) fprintf(fp, "%6.1f\t", m.lambda);
        else fprintf(fp, "%6d\t", line);
        if (mu_a ≥ 200) mu_a = 199.9999;
        if (mu_sp ≥ 1000) mu_sp = 999.9999;
        fprintf(fp, "\_9.4f\t\_%9.4f\t", m.m_r, LR);
        fprintf(fp, "\_9.4f\t\_%9.4f\t", m.m_t, LT);
        fprintf(fp, "\_9.4f\t", mu_a);
        fprintf(fp, "\_9.4f\t", mu_sp);
        fprintf(fp, "\_9.4f\t", r.g);
        fprintf(fp, "\_%c\_\n", what_char(r.error));
    }
    fflush(fp);
}

```

This code is used in section 2.

26. $\langle$ print error legend function 26 $\rangle \equiv$

```
static void print_error_legend(void)
{
    if (Debug(DEBUG_ANY)) return;
    fprintf(stderr, "-----Sorry, but... errors encountered-----\n");
    fprintf(stderr, "****=>Success*****");
    fprintf(stderr, "0-9=>Monte Carlo Iteration\n");
    fprintf(stderr, "R=>M_Ris too big");
    fprintf(stderr, "r=>M_Ris too small\n");
    fprintf(stderr, "T=>M_Tis too big");
    fprintf(stderr, "t=>M_Tis too small\n");
    fprintf(stderr, "U=>M_Uis too big");
    fprintf(stderr, "u=>M_Uis too small\n");
    fprintf(stderr, "!=>M_R+M_T>1");
    fprintf(stderr, "+=>Did not converge\n\n");
}
```

This code is used in section 2.

27. returns a new string consisting of s+t

$\langle$ stringdup together function 27 $\rangle \equiv$

```
static char *strdup_together(char *s, char *t)
{
    char *both;
    if (s == 0) {
        if (t == 0) return 0;
        return strdup(t);
    }
    if (t == 0) return strdup(s);
    both = malloc(strlen(s) + strlen(t) + 1);
    if (both == 0) fprintf(stderr, "Could not allocate memory for both strings.\n");
    strcpy(both, s);
    strcat(both, t);
    return both;
}
```

This code is used in section 2.

28. catch parsing errors in strtod

⟨mystrtod function 28⟩ ≡

```
static double my_strtod(const char *str)
{
    char *endptr;
    errno = 0;
    double val = strtod(str, &endptr);
    if (endptr == str) { /* No digits were found */
        fprintf(stderr, "Error in the command line\n");
        fprintf(stderr, "No conversion could be performed for '%s'.\n", str);
        exit(EXIT_FAILURE);
    }
    if (*endptr != '\0') { /* String contains extra characters after the number */
        fprintf(stderr, "Error in the command line\n");
        fprintf(stderr, "Partial conversion of string = '%s'\n", str);
        exit(EXIT_FAILURE);
    }
    if (errno == ERANGE) { /* The converted value is out of range of representable values by a double */
        fprintf(stderr, "Error in the command line\n");
        printf("The value '%s' is out of range of double.\n", str);
        exit(EXIT_FAILURE);
    }
    return val;
}
```

This code is used in section 2.

29. assume that start time has already been set

⟨seconds elapsed function 29⟩ ≡

```
static double seconds_elapsed(clock_t start_time)
{
    clock_t finish_time = clock();
    return (double)(finish_time - start_time)/CLOCKS_PER_SEC;
}
```

This code is used in section 2.

30. given a string and an array, this fills the array with numbers from the string. The numbers should be separated by spaces.

Returns 0 upon successfully filling n entries, returns 1 for any error.

⟨parse string into array function 30⟩ ≡

```
static int parse_string_into_array(char *s, double *a, int n)
{
    char *t, *last, *r;
    int i = 0;

    t = s;
    last = s + strlen(s);
    while (t < last) { /* a space should mark the end of number */
        r = t;
        while (*r ≠ ' ' ∧ *r ≠ '\0') r++;
        *r = '\0'; /* parse the number and save it */
        if (sscanf(t, "%lf", &a[i]) ≠ 0) return 1;
        i++; /* are we done? */
        if (i ≡ n) {
            if (i ≡ 5) { /* sphere parameters case */
                if (a[i - 1] ≤ 0 ∨ a[i - 1] > 1) {
                    fprintf(stderr,
                        "Sphere wall reflectivity (r_w=%g) must be a fraction less than one.\n",
                        a[i - 1]);
                    exit(EXIT_FAILURE);
                }
            }
        }
        return 0;
    } /* move pointer just after last number */
    t = r + 1;
}
return 1;
```

This code is used in section 2.

31. ⟨what_char function 31⟩ ≡

```
static char what_char(int err)
{
    if (err ≡ IAD_NO_ERROR) return '*';
    if (err ≡ IAD_TOO_MANY_ITERATIONS) return '+';
    if (err ≡ IAD_MR_TOO_BIG) return 'R';
    if (err ≡ IAD_MR_TOO_SMALL) return 'r';
    if (err ≡ IAD_MT_TOO_BIG) return 'T';
    if (err ≡ IAD_MT_TOO_SMALL) return 't';
    if (err ≡ IAD_MU_TOO_BIG) return 'U';
    if (err ≡ IAD_MU_TOO_SMALL) return 'u';
    if (err ≡ IAD_TOO_MUCH_LIGHT) return '!';
    return '?';
}
```

This code is used in section 2.

32. $\langle$ print long error function 32 $\rangle \equiv$

```
static void print_long_error(int err)
{
    if (err == IAD_TOO_MANY_ITERATIONS) fprintf(stderr, "Failed_Search, too_many_iterations\n");
    if (err == IAD_MR_TOO_BIG) fprintf(stderr, "Failed_Search, M_R_is_too_big\n");
    if (err == IAD_MR_TOO_SMALL) fprintf(stderr, "Failed_Search, M_R_is_too_small\n");
    if (err == IAD_MT_TOO_BIG) fprintf(stderr, "Failed_Search, M_T_is_too_big\n");
    if (err == IAD_MT_TOO_SMALL) fprintf(stderr, "Failed_Search, M_T_is_too_small\n");
    if (err == IAD_MU_TOO_BIG) fprintf(stderr, "Failed_Search, M_U_is_too_big\n");
    if (err == IAD_MU_TOO_SMALL) fprintf(stderr, "Failed_Search, M_U_is_too_small\n");
    if (err == IAD_TOO_MUCH_LIGHT) fprintf(stderr, "Failed_Search, Total_light_bigger_than_1\n");
    if (err == IAD_NO_ERROR) fprintf(stderr, "Successful_Search\n");
    fprintf(stderr, "\n");
}
```

This code is used in section 2.

33. The idea here is to show some intermediate output while a file is being processed.

$\langle$ print dot function 33 $\rangle \equiv$

```
static void print_dot(clock_t start_time, int err, int points, int final, int verbosity)
{
    static int counter = 0;
    counter++;
    if (verbosity == 0 || Debug(DEBUG_ANY)) return;
    if (final) fprintf(stderr, "%c", what_char(err));
    else {
        counter--;
        fprintf(stderr, "%1d\b", points % 10);
    }
    if (final) {
        if (counter % 50 == 0) {
            double rate = (seconds_elapsed(start_time) / counter);
            fprintf(stderr, "%3d_done (%5.2f_s/pt)\n", counter, rate);
        }
        else if (counter % 10 == 0) fprintf(stderr, "\n");
    }
    fflush(stderr);
}
```

This code is used in section 2.

34. IAD Types. This file has no routines. It is responsible for creating the header file `iad_type.h` and nothing else.

```
<iad_type.h 34> ≡
  <Preprocessor definitions>
  <Structs to export from IAD Types 37>
```

35.

```
#define FIND_A 0
#define FIND_B 1
#define FIND_AB 2
#define FIND_AG 3
#define FIND_AUTO 4
#define FIND_BG 5
#define FIND_BaG 6
#define FIND_BsG 7
#define FIND_Ba 8
#define FIND_Bs 9
#define FIND_G 10
#define FIND_B_WITH_NO_ABSORPTION 11
#define FIND_B_WITH_NO_SCATTERING 12
#define RELATIVE 0
#define ABSOLUTE 1
#define COLLIMATED 1.0
#define DIFFUSE 0.0
#define FALSE 0
#define TRUE 1
#define IAD_MAX_ITERATIONS 200
#define BIG_A_CALC_VALUE 999999.0
#define SMALL_A_CALC_VALUE 0.00001
#define MAX_ABS_G 0.999999
#define GRID_SIZE 101
```

36. Need error codes for this silly program

```
#define IAD_NO_ERROR 0
#define IAD_TOO_MANY_ITERATIONS 1
#define IAD_AS_NOT_VALID 16
#define IAD_AE_NOT_VALID 17
#define IAD_AD_NOT_VALID 18
#define IAD_RW_NOT_VALID 19
#define IAD_RD_NOT_VALID 20
#define IAD_RSTD_NOT_VALID 21
#define IAD_GAMMA_NOT_VALID 22
#define IAD_F_NOT_VALID 23
#define IAD_BAD_PHASE_FUNCTION 24
#define IAD_QUAD_PTS_NOT_VALID 25
#define IAD_BAD_G_VALUE 26
#define IAD_TOO_MANY_LAYERS 27
#define IAD_MEMORY_ERROR 28
#define IAD_FILE_ERROR 29
#define IAD_EXCESSIVE_LIGHT_LOSS 30
#define IAD_RT_LT_MINIMUM 31
#define IAD_MR_TOO_SMALL 32
#define IAD_MR_TOO_BIG 33
#define IAD_MT_TOO_SMALL 34
#define IAD_MT_TOO_BIG 35
#define IAD_MU_TOO_SMALL 36
#define IAD_MU_TOO_BIG 37
#define IAD_TOO_MUCH_LIGHT 38
#define IAD_TSTD_NOT_VALID 39
#define UNINITIALIZED -99
#define DEBUG_A_LITTLE 1
#define DEBUG_GRID 2
#define DEBUG_ITERATIONS 4
#define DEBUG_LOST_LIGHT 8
#define DEBUG_BEST_GUESS 16
#define DEBUG_SEARCH 32
#define DEBUG_GRID_CALC 64
#define DEBUG_SPHERE_GAIN 128
#define DEBUG_EVERY_CALC 256
#define DEBUG_ANY #FFFFFFF
#define UNKNOWN 0
#define COMPARISON 1
#define SUBSTITUTION 2
#define MC_NONE 0
#define MC_USE_EXISTING 1
#define MC_REDO 2
```

37. The idea of the structure **measure_type** is collect all the information regarding a single measurement together in one spot. No information regarding how the inversion procedure is supposed to be done is contained in this structure, unlike in previous incarnations of this program.

⟨Structs to export from IAD Types 37⟩ =

```
typedef struct measure_type {
    double slab_index;
    double slab_thickness;
    double slab_top_slide_index;
    double slab_top_slide_b;
    double slab_top_slide_thickness;
    double slab_bottom_slide_index;
    double slab_bottom_slide_b;
    double slab_bottom_slide_thickness;
    double slab_cos_angle;
    int num_spheres;
    int num_measures;
    int method;
    int flip_sample;
    int baffle_r, baffle_t;
    double d_beam;
    double fraction_of_ru_in_mr;
    double fraction_of_tu_in_mt;
    double m_r, m_t, m_u;
    double lambda;
    double as_r, ad_r, at_r, aw_r, rd_r, rw_r, rstd_r, f_r;
    double as_t, ad_t, at_t, aw_t, rd_t, rw_t, rstd_t;
    double ur1_lost, uru_lost, ut1_lost, utu_lost;
    double d_sphere_r, d_sphere_t;
} IAD_measure_type;
```

See also sections 38 and 39.

This code is used in section 34.

38. This describes how the inversion process should proceed and also contains the results of that inversion process.

⟨Structs to export from IAD Types 37⟩ +≡

```
typedef struct invert_type {
    double a;    /* the calculated albedo */
    double b;    /* the calculated optical depth */
    double g;    /* the calculated anisotropy */
    int found;
    int search;
    int metric;
    double tolerance;
    double MC_tolerance;
    double final_distance;
    int error;
    struct AD_slab_type slab;
    struct AD_method_type method;
    int AD_iterations;
    int MC_iterations;
    double default_a;
    double default_b;
    double default_g;
    double default_ba;
    double default_bs;
    double default_mua;
    double default_mus;
} IAD_invert_type;
```

39. A few types that used to be enum's are now int's.

⟨Structs to export from IAD Types 37⟩ +≡

```
typedef int search_type;
typedef int boolean_type;
typedef int illumination_type;
typedef struct guess_t {
    double distance;
    double a;
    double b;
    double g;
} guess_type;
extern double FRACTION;
```

40. IAD Public.

This contains the routine *Inverse_RT* that should generally be the basic entry point into this whole mess. Call this routine with the proper values and true happiness is bound to be yours.

Altered accuracy of the standard method of root finding from 0.001 to 0.00001. Note, it really doesn't help to change the method from ABSOLUTE to RELATIVE, but I did anyway. (3/3/95)

```
<iad_pub.c 40> ≡
#include <stdio.h>
#include <math.h>
#include "nr_util.h"
#include "ad_globl.h"
#include "ad_frsnl.h"
#include "iad_type.h"
#include "iad_util.h"
#include "iad_calc.h"
#include "iad_find.h"
#include "iad_pub.h"
#include "iad_io.h"
#include "stdlib.h"
#include "mc_lost.h"
  <Definition for Inverse_RT 44>
  <Definition for measure_OK 50>
  <Definition for determine_search 57>
  <Definition for Initialize_Result 61>
  <Definition for Initialize_Measure 69>
  <Definition for ez_Inverse_RT 67>
  <Definition for Spheres_Inverse_RT 71>
  <Definition for Spheres_Inverse_RT2 84>
  <Definition for Calculate_MR_MT 78>
  <Definition for MinMax_MR_MT 82>
  <Definition for Calculate_Minimum_MR 80>
```

41. All the information that needs to be written to the header file *iad_pub.h*. This eliminates the need to maintain a set of header files as well.

```
<iad_pub.h 41> ≡
  <Prototype for Inverse_RT 43>;
  <Prototype for measure_OK 49>;
  <Prototype for determine_search 56>;
  <Prototype for Initialize_Result 60>;
  <Prototype for ez_Inverse_RT 66>;
  <Prototype for Initialize_Measure 68>;
  <Prototype for Calculate_MR_MT 77>;
  <Prototype for MinMax_MR_MT 81>;
  <Prototype for Calculate_Minimum_MR 79>;
  <Prototype for Spheres_Inverse_RT2 83>;
```

42. Here is the header file needed to access one interesting routine in the *libiad.so* library.

```
<lib_iad.h 42> ≡
  <Prototype for ez_Inverse_RT 66>;
  <Prototype for Spheres_Inverse_RT 70>;
  <Prototype for Spheres_Inverse_RT2 83>;
```

43. Inverse RT. *Inverse_RT* is the main function in this whole package. You pass the variable *m* containing your experimentally measured values to the function *Inverse_RT*. It hopefully returns the optical properties in *r* that are appropriate for your experiment.

⟨Prototype for *Inverse_RT* 43⟩ ≡

```
void Inverse_RT(struct measure_type m, struct invert_type *r)
```

This code is used in sections 41 and 44.

44. ⟨Definition for *Inverse_RT* 44⟩ ≡

⟨Prototype for *Inverse_RT* 43⟩

```
{
    if (r→search ≡ FIND_AUTO) {
        r→search = determine_search(m, *r);
    }
    if (r→search ≡ FIND_B_WITH_NO_ABSORPTION) {
        r→default_a = 1;
        r→search = FIND_B;
    }
    if (r→search ≡ FIND_B_WITH_NO_SCATTERING) {
        r→default_a = 0;
        r→search = FIND_B;
    }
    ⟨Exit with bad input data 45⟩
    ⟨Find the optical properties 46⟩
    ⟨Print basic sphere and MC effects 47⟩
}
```

This code is used in section 40.

45. There is no sense going to all the trouble to try a multivariable minimization if the input data is bogus. So I wrote a single routine *measure_OK* to do just this.

⟨Exit with bad input data 45⟩ ≡

```
r→error = measure_OK(m, *r);
if (r→method.quad_pts < 4) r→error = IAD_QUAD_PTS_NOT_VALID;
if (r→error ≠ IAD_NO_ERROR) return;
```

This code is used in section 44.

46. Now I fob the real work off to the unconstrained minimization routines. Ultimately, I would like to replace all these by constrained minimization routines. Actually the first five already are constrained. The real work will be improving the last five because these are 2-D minimization routines.

```

⟨Find the optical properties 46⟩ ≡
  if (Debug(DEBUG_ITERATIONS)) {
    fprintf(stderr, "-----Beginning New Search-----\n");
    fprintf(stderr, "aaaaaaaaa");
    fprintf(stderr, "aaaaaaaaaabbbaaaaggggll");
    fprintf(stderr, "aaaaaM_Raaaaaaacalcuuu|");
    fprintf(stderr, "aaaaaM_Taaaaaaacalcuuu|");
    if (r-metric ≡ RELATIVE) fprintf(stderr, "relative distance\n");
    else fprintf(stderr, "absolute distance\n");
  }
  switch (r-search) {
  case FIND_A: U_Find_A(m, r);
    break;
  case FIND_B: U_Find_B(m, r);
    break;
  case FIND_G: U_Find_G(m, r);
    break;
  case FIND_Ba: U_Find_Ba(m, r);
    break;
  case FIND_Bs: U_Find_Bs(m, r);
    break;
  case FIND_AB: U_Find_AB(m, r);
    break;
  case FIND_AG: U_Find_AG(m, r);
    break;
  case FIND_BG: U_Find_BG(m, r);
    break;
  case FIND_BsG: U_Find_BsG(m, r);
    break;
  case FIND_BaG: U_Find_BaG(m, r);
    break;
  }
  if (Debug(DEBUG_ITERATIONS))
    fprintf(stderr, "Final amoeba/brent result after %d iterations\n", r-AD_iterations);
  if (r-AD_iterations ≥ IAD_MAX_ITERATIONS) {
    r-error = IAD_TOO_MANY_ITERATIONS;
    r-found = FALSE;
  }
}

```

This code is used in section 44.

47. This is to support -x 1

⟨Print basic sphere and MC effects 47⟩ ≡

```

if (Debug(DEBUG_A_LITTLE)) {
    double M_R, M_T, mua, mus, musp;
    Calculate_Mua_Musp(m, *r, &mus, &musp, &mua);
    fprintf(stderr, "AD_iterations=%3dMC_iterations=%3d", r->AD_iterations, r->MC_iterations);
    fprintf(stderr, "          a=%6.4f, b=%6.4f, g=%6.4f, ", r->slab.a, r->slab.b, r->slab.g);
    fprintf(stderr, "mu_a=%6.4f, mu_s=%6.4f, mus'=%6.4f\n", mua, mus, musp);
    fprintf(stderr, "          M_R_loss%8.5fM_T_loss%8.5f", m->ur1_lost,
        m->ut1_lost);
    if (r->MC_iterations == 0) fprintf(stderr, "(none_yet)\n");
    else fprintf(stderr, "\n");
    Calculate_MR_MT(m, *r, MC_NONE, FALSE, &M_R, &M_T);
    fprintf(stderr, "          M_R_no_corrections%8.5fM_T_no_corrections%8.5f\n", M_R, M_T);
    Calculate_MR_MT(m, *r, MC_NONE, TRUE, &M_R, &M_T);
    fprintf(stderr, "          M_R+osphere%8.5fM_T+osphere%8.5f\n", M_R, M_T);
    Calculate_MR_MT(m, *r, MC_USE_EXISTING, FALSE, &M_R, &M_T);
    fprintf(stderr, "          M_R+mc%8.5fM_T+mc%8.5f\n", M_R, M_T);
    Calculate_MR_MT(m, *r, MC_USE_EXISTING, TRUE, &M_R, &M_T);
    fprintf(stderr, "          M_R+osphere+mc%8.5fM_T+osphere+mc%8.5f\n", M_R, M_T);
    fprintf(stderr, "          M_R_measured%8.5fM_T_measured%8.5f", m->m_r, m->m_t);
    fprintf(stderr, "\n");
    fprintf(stderr, "Final_distance%8.5f\n\n", r->final_distance);
}

```

This code is used in section 44.

48. Validation.

49. Now the question is — just what is bad data? Here's the prototype.

⟨Prototype for *measure_OK* 49⟩ ≡

```
int measure_OK(struct measure_type m, struct invert_type r)
```

This code is used in sections 41 and 50.

50. It would just be nice to stop computing with bad data. This does not work in practice because it turns out that there is often bogus data in a full wavelength scan. Often the reflectance is too low for short wavelengths and at long wavelengths the detector (photomultiplier tube) does not work worth a damn.

The two sphere checks are more complicated. For example, we can no longer categorically state that the transmittance is less than one or that the sum of the reflectance and transmittance is less than one. Instead we use the transmittance to bound the values for the reflectance — see the routine *MinMax_MR_MT* below.

⟨Definition for *measure_OK* 50⟩ ≡

⟨Prototype for *measure_OK* 49⟩

```
{
  double ru, tu;
  if (m.num_spheres ≠ 2) {
    ⟨Check MT for zero or one spheres 52⟩
    ⟨Check MR for zero or one spheres 51⟩
  }
  else {
    int error = MinMax_MR_MT(m, r);
    if (error ≠ IAD_NO_ERROR) return error;
  }
  ⟨Check MU 53⟩
  if (m.num_spheres ≠ 0) {
    ⟨Check sphere parameters 54⟩
  }
  return IAD_NO_ERROR;
}
```

This code is used in section 40.

51. The reflectance is constrained by the index of refraction of the material and the transmission. The upper bound for the reflectance is just one minus the transmittance. The specular (unscattered) reflectance from the boundaries imposes minimum for the reflectance. Obviously, the reflected light cannot be less than that from the first boundary. This might be calculated by assuming an infinite layer thickness. But we can do better.

There is a definite bound on the minimum reflectance from a sample. If you have a sample with a given transmittance m_t , the minimum reflectance possible is found by assuming that the sample does not scatter any light.

Knowledge of the indices of refraction makes it a relatively simple matter to determine the optical thickness $b = \mu_a * d$ of the slab. The minimum reflection is obtained by including all the specular reflectances from all the surfaces.

If the default albedo has been specified as zero, then there is really no need to check MR because it is ignored.

⟨ Check MR for zero or one spheres 51 ⟩ $\equiv$

```
{
  double mr, mt;
  Calculate_Minimum_MR(m, r, &mr, &mt);
  if (m.m_r < 0) return IAD_MR_TOO_SMALL;
  if (m.m_r > 1) return IAD_MR_TOO_BIG;
  /* one parameter search only needs one good measurement */
  if (r.search  $\equiv$  FIND_A  $\vee$  r.search  $\equiv$  FIND_G  $\vee$  r.search  $\equiv$  FIND_B  $\vee$  r.search  $\equiv$  FIND_Bs  $\vee$  r.search  $\equiv$ 
      FIND_Ba) {
    if (m.m_r < mr  $\wedge$  m.m_t  $\leq$  0) return IAD_MR_TOO_SMALL;
  }
  else {
    if (r.default_a  $\equiv$  UNINITIALIZED  $\vee$  r.default_a > 0) {
      if (m.m_r < mr) return IAD_MR_TOO_SMALL;
    }
  }
}
```

This code is used in section 50.

52. The transmittance is also constrained by the index of refraction of the material. The minimum transmittance is zero, but the maximum transmittance cannot exceed the total light passing through the sample when there is no scattering or absorption. This is calculated by assuming an infinitely thin (to eliminate any scattering or absorption effects).

There is a problem when spheres are present. The estimated values for the transmittance using Sp_mu_RT are not actually limiting cases. This will require a bit of fixing, but for now that test is omitted if the number of spheres is more than zero.

⟨ Check MT for zero or one spheres 52 ⟩ $\equiv$

```
if (m.m_t < 0) return IAD_MT_TOO_SMALL;
if (m.m_t > 1) return IAD_MR_TOO_BIG;
Sp_mu_RT_Flip(m.flip_sample, r.slab.n_top_slide, r.slab.n_slab, r.slab.n_bottom_slide, r.slab.b_top_slide, 0,
  r.slab.b_bottom_slide, r.slab.cos_angle, &ru, &tu);
if (m.num_spheres  $\equiv$  0  $\wedge$  m.m_t > tu) {
  fprintf(stderr, "ntop=%7.5f, nslab=%7.5f, nbottom=%7.5f\n", r.slab.n_top_slide, r.slab.n_slab,
    r.slab.n_bottom_slide);
  fprintf(stderr, "tu_max=%7.5f, m_t=%7.5f, t_std=%7.5f\n", tu, m.m_t, m.rstd_t);
  return IAD_MT_TOO_BIG;
}
```

This code is used in section 50.

53. The unscattered transmission is now always included in the total transmittance. Therefore the unscattered transmittance must fall between zero and M_T

⟨ Check MU 53 ⟩ $\equiv$

```

    if ( $m.m_u < 0$ ) return IAD_MU_TOO_SMALL;
    if ( $m.m_t > 0 \wedge m.m_u > m.m_t$ ) return IAD_MU_TOO_BIG;

```

This code is used in section 50.

54. Make sure that reflection sphere parameters are reasonable

⟨ Check sphere parameters 54 ⟩ $\equiv$

```

    if ( $m.as_r < 0 \vee m.as_r \geq 0.2$ ) return IAD_AS_NOT_VALID;
    if ( $m.ad_r < 0 \vee m.ad_r \geq 0.2$ ) return IAD_AD_NOT_VALID;
    if ( $m.at_r < 0 \vee m.at_r \geq 0.2$ ) return IAD_AE_NOT_VALID;
    if ( $m.rw_r < 0 \vee m.rw_r > 1.0$ ) return IAD_RW_NOT_VALID;
    if ( $m.rd_r < 0 \vee m.rd_r > 1.0$ ) return IAD_RD_NOT_VALID;
    if ( $m.rstd_r < 0 \vee m.rstd_r > 1.0$ ) return IAD_RSTD_NOT_VALID;
    if ( $m.rstd_t < 0 \vee m.rstd_t > 1.0$ ) return IAD_TSTD_NOT_VALID;
    if ( $m.f_r < 0 \vee m.f_r > 1$ ) return IAD_F_NOT_VALID;

```

See also section 55.

This code is used in section 50.

55. Make sure that transmission sphere parameters are reasonable

⟨ Check sphere parameters 54 ⟩ $+ \equiv$

```

    if ( $m.as_t < 0 \vee m.as_t \geq 0.2$ ) return IAD_AS_NOT_VALID;
    if ( $m.ad_t < 0 \vee m.ad_t \geq 0.2$ ) return IAD_AD_NOT_VALID;
    if ( $m.at_t < 0 \vee m.at_t \geq 0.2$ ) return IAD_AE_NOT_VALID;
    if ( $m.rw_t < 0 \vee m.rw_r > 1.0$ ) return IAD_RW_NOT_VALID;
    if ( $m.rd_t < 0 \vee m.rd_t > 1.0$ ) return IAD_RD_NOT_VALID;
    if ( $m.rstd_t < 0 \vee m.rstd_t > 1.0$ ) return IAD_TSTD_NOT_VALID;

```

56. Searching Method.

The original idea was that this routine would automatically determine what optical parameters could be figured out from the input data. This worked fine for a long while, but I discovered that often it was convenient to constrain the optical properties in various ways. Consequently, this routine got more and more complicated.

What should be done is to figure out whether the search will be 1D or 2D and split this routine into two parts.

It would be nice to enable the user to constrain two parameters, but the infrastructure is missing at this point.

⟨Prototype for *determine_search* 56⟩ ≡

search_type *determine_search*(**struct measure_type** *m*, **struct invert_type** *r*)

This code is used in sections 41 and 57.

57. This routine is responsible for selecting the appropriate optical properties to determine.

⟨Definition for *determine_search* 57⟩ ≡

⟨Prototype for *determine_search* 56⟩

```
{
    double rt, tt, rd, td, tc, rc;
    int search = 0;
    int independent = 0;
    int constraints = 0;
    if (m.m_r > 0) independent++;
    if (m.m_t > 0) independent++;
    if (m.m_u > 0) independent++;
    if (r.default_a ≠ UNINITIALIZED) constraints++;
    if (r.default_b ≠ UNINITIALIZED) constraints++;
    if (r.default_g ≠ UNINITIALIZED) constraints++;
    if (r.default_mua ≠ UNINITIALIZED) constraints++;
    if (r.default_mus ≠ UNINITIALIZED) constraints++;
    Estimate_RT(m, r, &rt, &tt, &rd, &rc, &td, &tc);
    if (Debug(DEBUG_SEARCH)) {
        fprintf(stderr, "SEARCH: starting with %d measurement(s)\n", independent);
        fprintf(stderr, "SEARCH: and with %d constraint(s)\n", constraints);
        if (r.default_a ≠ UNINITIALIZED) fprintf(stderr, "albedo constrained\n");
        if (r.default_b ≠ UNINITIALIZED)
            fprintf(stderr, "optical thickness constrained\n");
        if (r.default_g ≠ UNINITIALIZED) fprintf(stderr, "anisotropy constrained\n");
        if (r.default_mua ≠ UNINITIALIZED) fprintf(stderr, "mua constrained\n");
        if (r.default_mus ≠ UNINITIALIZED) fprintf(stderr, "mus constrained\n");
        fprintf(stderr, "SEARCH: m_r = %8.5f", m.m_r);
        fprintf(stderr, "m_t = %8.5f", m.m_t);
        fprintf(stderr, "m_u = %8.5f\n", m.m_u);
        fprintf(stderr, "SEARCH: r_t = %8.5f", rt);
        fprintf(stderr, "rd = %8.5f", rd);
        fprintf(stderr, "ru = %8.5f\n", rc);
        fprintf(stderr, "SEARCH: tt = %8.5f", tt);
        fprintf(stderr, "td = %8.5f", td);
        fprintf(stderr, "tu = %8.5f\n", tc);
    }
    if (rd ≡ 0 ∧ independent ≥ 2) {
        if (Debug(DEBUG_SEARCH)) fprintf(stderr, "SEARCH: no information in rd\n");
        independent--;
    }
    if (td ≡ 0 ∧ independent ≥ 2) {
        if (Debug(DEBUG_SEARCH)) fprintf(stderr, "SEARCH: no information in td\n");
        independent--;
    }
    if (constraints + independent > 3) {
        fprintf(stderr, "Too many constraints!\n");
    }
    if (independent ≡ 0) {
        search = FIND_A;
    }
    else if (independent ≡ 1) {
        ⟨One parameter search 58⟩
    }
}
```

```

}
else if (independent  $\equiv$  2) {
  ⟨ Two parameter search 59 ⟩
} /* three real parameters with information! */
else {
  search = FIND_AG;
}
if (search  $\equiv$  FIND_BG  $\wedge$  m.m_u > 0) search = FIND_G;
if (Debug(DEBUG_SEARCH)) {
  fprintf(stderr, "SEARCH: ending with %d measurement(s)\n", independent);
  fprintf(stderr, "SEARCH: and with %d constraint(s)\n", constraints);
  fprintf(stderr, "SEARCH: final choice for search = ");
  if (search  $\equiv$  FIND_A) fprintf(stderr, "FIND_A\n");
  if (search  $\equiv$  FIND_B) fprintf(stderr, "FIND_B\n");
  if (search  $\equiv$  FIND_AB) fprintf(stderr, "FIND_AB\n");
  if (search  $\equiv$  FIND_AG) fprintf(stderr, "FIND_AG\n");
  if (search  $\equiv$  FIND_AUTO) fprintf(stderr, "FIND_AUTO\n");
  if (search  $\equiv$  FIND_BG) fprintf(stderr, "FIND_BG\n");
  if (search  $\equiv$  FIND_BaG) fprintf(stderr, "FIND_BaG\n");
  if (search  $\equiv$  FIND_BsG) fprintf(stderr, "FIND_BsG\n");
  if (search  $\equiv$  FIND_Ba) fprintf(stderr, "FIND_Ba\n");
  if (search  $\equiv$  FIND_Bs) fprintf(stderr, "FIND_Bs\n");
  if (search  $\equiv$  FIND_G) fprintf(stderr, "FIND_G\n");
  if (search  $\equiv$  FIND_B_WITH_NO_ABSORPTION) fprintf(stderr, "FIND_B_WITH_NO_ABSORPTION\n");
  if (search  $\equiv$  FIND_B_WITH_NO_SCATTERING) fprintf(stderr, "FIND_B_WITH_NO_SCATTERING\n");
}
return search;
}

```

This code is used in section 40.

58. The fastest inverse problems are those in which just one measurement is known. This corresponds to a simple one-dimensional minimization problem. The only complexity is deciding exactly what should be allowed to vary. The basic assumption is that the anisotropy has been specified or will be assumed to be zero.

If the anisotropy is assumed known, then one other assumption will allow us to figure out the last parameter to solve for.

Ultimately, if no default values are given, then we look at the value of the total transmittance. If this is zero, then we assume that the optical thickness is infinite and solve for the albedo. Otherwise we will just make a stab at solving for the optical thickness assuming the albedo is one.

⟨ One parameter search 58 ⟩ ≡

```

if (r.default_a ≠ UNINITIALIZED) {
  if (r.default_a ≡ 0) search = FIND_B_WITH_NO_SCATTERING;
  else if (r.default_a ≡ 1) search = FIND_B_WITH_NO_ABSORPTION;
  else if (tt ≡ 0) search = FIND_G;
  else search = FIND_B;
}
else if (r.default_b ≠ UNINITIALIZED) search = FIND_A;
else if (r.default_bs ≠ UNINITIALIZED) search = FIND_Ba;
else if (r.default_ba ≠ UNINITIALIZED) search = FIND_Bs;
else if (m.m_t ≡ 0) search = FIND_A;
else if (rd ≡ 0) search = FIND_B_WITH_NO_SCATTERING;
else search = FIND_B_WITH_NO_ABSORPTION;

```

This code is used in section 57.

59. If the absorption depth $\mu_a d$ is constrained return *FIND_BsG*. Recall that I use the bizarre mnemonic $bs = \mu_s d$ here and so this means that the program will search over various values of $\mu_s d$ and g .

If there are just two measurements then I assume that the anisotropy is not of interest and the only thing to calculate is the reduced albedo and optical thickness based on an assumed anisotropy.

```

⟨Two parameter search 59⟩ ≡
  if (r.default_a ≠ UNINITIALIZED) {
    if (r.default_a ≡ 0) search = FIND_B;
    else if (r.default_g ≠ UNINITIALIZED) search = FIND_B;
    else if (r.default_b ≠ UNINITIALIZED) search = FIND_G;
    else search = FIND_BG;
  }
  else if (r.default_b ≠ UNINITIALIZED) {
    if (r.default_g ≠ UNINITIALIZED) search = FIND_A;
    else search = FIND_AG;
  }
  else if (r.default_g ≠ UNINITIALIZED) {
    search = FIND_AB;
  }
  else if (r.default_ba ≠ UNINITIALIZED) {
    if (r.default_g ≠ UNINITIALIZED) search = FIND_Bs;
    else search = FIND_BsG;
  }
  else if (r.default_bs ≠ UNINITIALIZED) {
    if (r.default_g ≠ UNINITIALIZED) search = FIND_Ba;
    else search = FIND_BaG;
  }
  else if (rt + tt > 1 ∧ 0 ∧ m.num_spheres ≠ 2) search = FIND_B_WITH_NO_ABSORPTION;
  else search = FIND_AB;

```

This code is used in section 57.

60. This little routine just stuffs reasonable values into the structure we use to return the solution. This does not replace the values for *r.default_g* nor for *r.method.quad_pts*. Presumably these have been set correctly elsewhere.

```

⟨Prototype for Initialize_Result 60⟩ ≡
  void Initialize_Result(struct measure_type m, struct invert_type *r, int overwrite_defaults)

```

This code is used in sections 41 and 61.

```

61.  ⟨Definition for Initialize_Result 61⟩ ≡
  ⟨Prototype for Initialize_Result 60⟩
  {
    ⟨Fill r with reasonable values 62⟩
  }

```

This code is used in section 40.

62. Start with the optical properties.

```

⟨Fill r with reasonable values 62⟩ ≡
  r→a = 0.0;
  r→b = 0.0;
  r→g = 0.0;

```

See also sections 63, 64, and 65.

This code is used in section 61.

63. Continue with other useful stuff.

```

⟨ Fill r with reasonable values 62 ⟩ +=
  r→found = FALSE;
  r→tolerance = 0.0001;
  r→MC_tolerance = 0.01;    /* percent */
  r→search = FIND_AUTO;
  r→metric = ABSOLUTE;
  r→final_distance = 10;
  r→AD_iterations = 4;
  r→MC_iterations = 0;
  r→error = IAD_NO_ERROR;

```

64. The defaults might be handy

```

⟨ Fill r with reasonable values 62 ⟩ +=
  if (overwrite_defaults) {
    r→default_a = UNINITIALIZED;
    r→default_b = UNINITIALIZED;
    r→default_g = UNINITIALIZED;
    r→default_ba = UNINITIALIZED;
    r→default_bs = UNINITIALIZED;
    r→default_mua = UNINITIALIZED;
    r→default_mus = UNINITIALIZED;
  }

```

65. It is necessary to set up the slab correctly so, I stuff reasonable values into this record as well.

```

⟨ Fill r with reasonable values 62 ⟩ +=
  r→slab.a = 0.5;
  r→slab.b = 1.0;
  r→slab.g = 0;
  r→slab.phase_function = HENYEY_GREENSTEIN;
  r→slab.n_slab = m.slab_index;
  r→slab.n_top_slide = m.slab_top_slide_index;
  r→slab.n_bottom_slide = m.slab_bottom_slide_index;
  r→slab.b_top_slide = m.slab_top_slide_b;
  r→slab.b_bottom_slide = m.slab_bottom_slide_b;
  r→slab.cos_angle = m.slab_cos_angle;
  r→method.a_calc = 0.5;
  r→method.b_calc = 1;
  r→method.g_calc = 0.5;
  r→method.quad_pts = 8;
  r→method.b_thinnest = 1.0/32.0;

```

66. EZ Inverse RT. *ez_Inverse_RT* is a simple interface to the main function *Inverse_RT* in this package. It eliminates the need for complicated data structures so that the command line interface (as well as those to Perl and Mathematica) will be simpler. This function assumes that the reflection and transmission include specular reflection and that the transmission also include unscattered transmission.

Other assumptions are that the top and bottom slides have the same index of refraction, that the illumination is collimated. Of course no sphere parameters are included.

⟨Prototype for *ez_Inverse_RT* 66⟩ ≡

```
void ez_Inverse_RT(double n,double nslide,double UR1,double UT1,double Tu,double *a,double
    *b,double *g,int *error)
```

This code is used in sections 41, 42, and 67.

67. ⟨Definition for *ez_Inverse_RT* 67⟩ ≡

⟨Prototype for *ez_Inverse_RT* 66⟩

```
{
    struct measure_type m;
    struct invert_type r;

    *a = 0;
    *b = HUGE_VAL;
    *g = 0;
    Initialize_Measure(&m);
    m.slab_index = n;
    m.slab_top_slide_index = nslide;
    m.slab_bottom_slide_index = nslide;
    m.slab_cos_angle = 1.0;
    m.num_measures = 3;
    if (UT1 ≡ 0) m.num_measures--;
    if (Tu ≡ 0) m.num_measures--;
    m.m_r = UR1;
    m.m_t = UT1;
    m.m_u = Tu;
    Initialize_Result(m, &r, TRUE);
    r.method.quad_pts = 8;
    Inverse_RT(m, &r);
    *error = r.error;
    if (r.error ≡ IAD_NO_ERROR) {
        *a = r.a;
        *b = r.b;
        *g = r.g;
    }
}
```

This code is used in section 40.

68. ⟨Prototype for *Initialize_Measure* 68⟩ ≡

```
void Initialize_Measure(struct measure_type *m)
```

This code is used in sections 41 and 69.

69. $\langle$ Definition for *Initialize_Measure* 69 $\rangle \equiv$

$\langle$ Prototype for *Initialize_Measure* 68 $\rangle$

```
{
  double default_sphere_d = 8.0 * 25.4;
  double default_sample_d = 0.0 * 25.4;
  double default_detector_d = 0.1 * 25.4;
  double default_entrance_d = 0.5 * 25.4;
  double sphere_area = M_PI * default_sphere_d * default_sphere_d;
  m_slab_index = 1.0;
  m_slab_top_slide_index = 1.0;
  m_slab_top_slide_b = 0.0;
  m_slab_top_slide_thickness = 0.0;
  m_slab_bottom_slide_index = 1.0;
  m_slab_bottom_slide_b = 0.0;
  m_slab_bottom_slide_thickness = 0.0;
  m_slab_thickness = 1.0;
  m_slab_cos_angle = 1.0;
  m_num_spheres = 0;
  m_num_measures = 1;
  m_method = UNKNOWN;
  m_fraction_of_ru_in_mr = 1.0;
  m_fraction_of_tu_in_mt = 1.0;
  m_baffle_r = 1;
  m_baffle_t = 1;
  m_flip_sample = 0;
  m_m_r = 0.0;
  m_m_t = 0.0;
  m_m_u = 0.0;
  m_d_sphere_r = default_sphere_d;
  m_as_r = (M_PI * default_sample_d * default_sample_d / 4.0) / sphere_area;
  m_ad_r = (M_PI * default_detector_d * default_detector_d / 4.0) / sphere_area;
  m_at_r = (M_PI * default_entrance_d * default_entrance_d / 4.0) / sphere_area;
  m_aw_r = 1.0 - m_as_r - m_ad_r - m_at_r;
  m_rd_r = 0.0;
  m_rw_r = 1.0;
  m_rstd_r = 1.0;
  m_f_r = 0.0;
  m_d_sphere_t = default_sphere_d;
  m_as_t = m_as_r;
  m_ad_t = m_ad_r;
  m_at_t = 0;
  m_aw_t = 1.0 - m_as_t - m_ad_t - m_at_t;
  m_rd_t = 0.0;
  m_rw_t = 1.0;
  m_rstd_t = 1.0;
  m_lambda = 0.0;
  m_d_beam = 0.0;
  m_ur1_lost = 0;
  m_u ru_lost = 0;
  m_ut1_lost = 0;
  m_utu_lost = 0;
}
```

This code is used in section 40.

70. To avoid interfacing with C-structures it is necessary to pass the information as arrays. Here I have divided the experiment into (1) setup, (2) reflection sphere coefficients, (3) transmission sphere coefficients, (4) measurements, and (5) results.

⟨Prototype for *Spheres_Inverse_RT* 70⟩ ≡

```
void Spheres_Inverse_RT(double *setup, double *analysis, double *sphere_r, double *sphere_t, double
    *measurements, double *results)
```

This code is used in sections 42 and 71.

71. ⟨Definition for *Spheres_Inverse_RT* 71⟩ ≡

⟨Prototype for *Spheres_Inverse_RT* 70⟩

```
{
    struct measure_type m;
    struct invert_type r;
    long num_photons;
    double ur1, ut1, uru, utu;
    int i, mc_runs = 1;
    Initialize_Measure(&m);
    ⟨handle setup 72⟩
    ⟨handle reflection sphere 75⟩
    ⟨handle transmission sphere 76⟩
    ⟨handle measurement 74⟩
    Initialize_Result(m, &r, TRUE);
    results[0] = 0;
    results[1] = 0;
    results[2] = 0;
    ⟨handle analysis 73⟩
    Inverse_RT(m, &r);
    for (i = 0; i < mc_runs; i++) {
        MC_Lost(m, r, num_photons, &ur1, &ut1, &uru, &utu, &m.ur1_lost, &m.ut1_lost, &m.uru_lost,
            &m.utu_lost);
        Inverse_RT(m, &r);
    }
    if (r.error ≡ IAD_NO_ERROR) {
        results[0] = (1 - r.a) * r.b / m.slab_thickness;
        results[1] = (r.a) * r.b / m.slab_thickness;
        results[2] = r.g;
    }
    results[3] = r.error;
}
```

This code is used in section 40.

72. These are in exactly the same order as the parameters in the .rxt header

⟨handle setup 72⟩ ≡

```
{
  double d_sample_r, d_entrance_r, d_detector_r;
  double d_sample_t, d_entrance_t, d_detector_t;

  m.slab_index = setup[0];
  m.slab_top_slide_index = setup[1];
  m.slab_thickness = setup[2];
  m.slab_top_slide_thickness = setup[3];
  m.d_beam = setup[4];
  m.rstd_r = setup[5];
  m.num_spheres = (int) setup[6];
  m.d_sphere_r = setup[7];
  d_sample_r = setup[8];
  d_entrance_r = setup[9];
  d_detector_r = setup[10];
  m.rw_r = setup[11];
  m.d_sphere_t = setup[12];
  d_sample_t = setup[13];
  d_entrance_t = setup[14];
  d_detector_t = setup[15];
  m.rw_t = setup[16];
  r.default_g = setup[17];
  num_photons = (long) setup[18];
  m.as_r = (d_sample_r/m.d_sphere_r/2.0) * (d_sample_r/m.d_sphere_r/2.0);
  m.at_r = (d_entrance_r/m.d_sphere_r/2.0) * (d_entrance_r/m.d_sphere_r/2.0);
  m.ad_r = (d_detector_r/m.d_sphere_r/2.0) * (d_detector_r/m.d_sphere_r/2.0);
  m.aw_r = 1.0 - m.as_r - m.at_r - m.ad_r;
  m.as_t = (d_sample_t/m.d_sphere_t/2.0) * (d_sample_t/m.d_sphere_t/2.0);
  m.at_t = (d_entrance_t/m.d_sphere_t/2.0) * (d_entrance_t/m.d_sphere_t/2.0);
  m.ad_t = (d_detector_t/m.d_sphere_t/2.0) * (d_detector_t/m.d_sphere_t/2.0);
  m.aw_t = 1.0 - m.as_t - m.at_t - m.ad_t;
  m.slab_bottom_slide_index = m.slab_top_slide_index;
  m.slab_bottom_slide_thickness = m.slab_top_slide_thickness;
  fprintf(stderr, "****_executing_FIXME_****/n");
  m.slab_cos_angle = 1.0; /* FIXME */
}
```

This code is used in section 71.

73. ⟨handle analysis 73⟩ ≡

```
r.method.quad_pts = (int) analysis[0];
mc_runs = (int) analysis[1];
```

This code is used in section 71.

74.

```

⟨ handle measurement 74 ⟩ ≡
    m.m_r = measurements[0];
    m.m_t = measurements[1];
    m.m_u = measurements[2];
    m.num_measures = 3;
    if (m.m_t ≡ 0) m.num_measures--;
    if (m.m_u ≡ 0) m.num_measures--;

```

This code is used in section 71.

75.

```

⟨ handle reflection sphere 75 ⟩ ≡
    m.as_r = sphere_r[0];
    m.at_r = sphere_r[1];
    m.ad_r = sphere_r[2];
    m.rw_r = sphere_r[3];
    m.rd_r = sphere_r[4];
    m.rstd_r = sphere_r[5];
    m.f_r = sphere_r[7];

```

This code is used in section 71.

76.

```

⟨ handle transmission sphere 76 ⟩ ≡
    m.as_t = sphere_t[0];
    m.at_t = sphere_t[1];
    m.ad_t = sphere_t[2];
    m.rw_t = sphere_t[3];
    m.rd_t = sphere_t[4];
    m.rstd_t = sphere_t[5];

```

This code is used in section 71.

77. I needed a routine that would calculate the values of M_R and M_T without doing the whole inversion process. It seems odd that this does not exist yet.

The values for the lost light *m.uru_lost* etc., should be calculated before calling this routine.

⟨ Prototype for *Calculate_MR_MT* 77 ⟩ ≡

```

void Calculate_MR_MT(struct measure_type m, struct invert_type r, int include_MC, int
    include_spheres, double *M_R, double *M_T)

```

This code is used in sections 41 and 78.

78. $\langle$ Definition for *Calculate_MR_MT* 78 $\rangle \equiv$

$\langle$ Prototype for *Calculate_MR_MT* 77 $\rangle$

```
{
    double distance;
    struct measure_type old_mm;
    struct invert_type old_rr;
    if (include_MC  $\equiv$  MC_NONE) {
        m.ur1_lost = 0;
        m.ut1_lost = 0;
        m.uru_lost = 0;
        m.utu_lost = 0;
    }
    if (include_MC  $\equiv$  MC_REDO) {
        double ur1, ut1, uru, utu;
        long n_photons = 100000;
        MC_Lost(m, r, n_photons, &ur1, &ut1, &uru, &utu, &m.ur1_lost, &m.ut1_lost, &m.uru_lost,
            &m.utu_lost);
    }
    if ( $\neg$ include_spheres) {
        m.num_spheres = 0;
    }
    Get_Calc_State(&old_mm, &old_rr);
    Set_Calc_State(m, r);
    Calculate_Distance(M_R, M_T, &distance);
    Set_Calc_State(old_mm, old_rr);
}
```

This code is used in section 40.

79. So, it turns out that the minimum measured M_R can be less than four percent for black glass! This is because the sphere efficiency is much worse for the glass than for the white standard.

If a *default_b* is specified then that determines the collimated transmission even when a *m_u* measurement is available.

$\langle$ Prototype for *Calculate_Minimum_MR* 79 $\rangle \equiv$

```
void Calculate_Minimum_MR(struct measure_type m, struct invert_type r, double *mr, double
    *mt)
```

This code is used in sections 41 and 80.

80. $\langle$ Definition for *Calculate_Minimum_MR* 80 $\rangle \equiv$
 $\langle$ Prototype for *Calculate_Minimum_MR* 79 $\rangle$
 $\{$
 if ($r.default_b \neq \text{UNINITIALIZED}$) $r.slabs.b = r.default_b$;
 else if ($m.m_u > 0$) $r.slabs.b = \text{What_Is_B}(r.slabs, m.m_u)$;
 else $r.slabs.b = \text{HUGE_VAL}$;
 $r.slabs.a = 0$;
 if ($r.default_g \equiv \text{UNINITIALIZED}$) $r.slabs.g = 0.0$;
 else $r.slabs.g = r.default_g$;
 $r.a = r.slabs.a$;
 $r.b = r.slabs.b$;
 $r.g = r.slabs.g$;
 $\text{Calculate_MR_MT}(m, r, \text{FALSE}, \text{TRUE}, mr, mt)$;
 $\}$

This code is used in section 40.

81. The minimum possible value of MR for a given MT will be when the albedo is zero and the maximum value will be when the albedo is one. In the first case there will be no light loss and in the second we will assume that any light loss is negligible (to maximize MR).

The second case is perhaps over-simplified. Obviously for a fixed thickness as the albedo increases, the reflectance will increase. So how does $U_Find_B()$ work when the albedo is set to 1?

The problem is that to calculate these values one must know the optical thickness. Fortunately with the recent addition of constrained minimization, we can do exactly this.

The only thing that remains is to sort out the light lost effect.

$\langle$ Prototype for *MinMax_MR_MT* 81 $\rangle \equiv$
int *MinMax_MR_MT*(**struct** **measure_type** m , **struct** **invert_type** r)

This code is used in sections 41 and 82.

82. $\langle$ Definition for *MinMax_MR_MT* 82 $\rangle \equiv$

$\langle$ Prototype for *MinMax_MR_MT* 81 $\rangle$

```
{
    double distance, measured_m_r, min_possible_m_r, max_possible_m_r, temp_m_t;
    if (m.m_r < 0) return IAD_MR_TOO_SMALL;
    if (m.m_r * m.rstd_r > 1) return IAD_MR_TOO_BIG;
    if (m.m_t < 0) return IAD_MT_TOO_SMALL;
    if (m.m_t  $\equiv$  0) return IAD_NO_ERROR;
    measured_m_r = m.m_r;
    m.m_r = 0;
    r.search = FIND_B;
    if (Debug(DEBUG_ITERATIONS))
        fprintf(stderr, "Determining minimum possible M_R for given M_T\n");
    r.default_a = 0;
    U_Find_B(m, &r);
    Calculate_Distance(&min_possible_m_r, &temp_m_t, &distance);
    if (measured_m_r < min_possible_m_r) return IAD_MR_TOO_SMALL;
    if (Debug(DEBUG_ITERATIONS))
        fprintf(stderr, "Determining maximum possible M_R for given M_T\n");
    r.default_a = 1.0;
    U_Find_B(m, &r);
    Calculate_Distance(&max_possible_m_r, &temp_m_t, &distance);
    if (measured_m_r > max_possible_m_r) return IAD_MR_TOO_BIG;
    return IAD_NO_ERROR;
}
```

This code is used in section 40.

83. $\langle$ Prototype for *Spheres_Inverse_RT2* 83 $\rangle \equiv$

```
void Spheres_Inverse_RT2(double *sample, double *illumination, double *sphere_r, double
    *sphere_t, double *analysis, double *measurement, double *a, double *b, double *g)
```

This code is used in sections 41, 42, and 84.

84. $\langle$ Definition for *Spheres_Inverse_RT2* 84 $\rangle \equiv$
 $\langle$ Prototype for *Spheres_Inverse_RT2* 83 $\rangle$
{
 struct **measure_type** *m*;
 struct **invert_type** *r*;
 long *num_photons*;
 double *ur1*, *ut1*, *uru*, *utu*;
 int *i*, *mc_runs* = 1;
 Initialize_Measure(&*m*);
 $\langle$ handle2 sample 85 $\rangle$
 $\langle$ handle2 illumination 86 $\rangle$
 $\langle$ handle2 reflection sphere 87 $\rangle$
 $\langle$ handle2 transmission sphere 88 $\rangle$
 $\langle$ handle2 analysis 89 $\rangle$
 $\langle$ handle2 measurement 90 $\rangle$
 Initialize_Result(*m*, &*r*, TRUE);
 Inverse_RT(*m*, &*r*);
 for (*i* = 0; *i* < *mc_runs*; *i*++) {
 MC_Lost(*m*, *r*, *num_photons*, &*ur1*, &*ut1*, &*uru*, &*utu*, &*m.ur1_lost*, &*m.ut1_lost*, &*m.uru_lost*,
 &*m.utu_lost*);
 Inverse_RT(*m*, &*r*);
 }
 if (*r.error* $\equiv$ IAD_NO_ERROR) {
 a = *r.a*;
 b = *r.b*;
 g = *r.g*;
 }
}

This code is used in section 40.

85. Just move the values from the sample array into the right places

$\langle$ handle2 sample 85 $\rangle \equiv$
 m.slab_index = *sample*[0];
 m.slab_top_slide_index = *sample*[1];
 m.slab_bottom_slide_index = *sample*[2];
 m.slab_thickness = *sample*[3];
 m.slab_top_slide_thickness = *sample*[4];
 m.slab_bottom_slide_thickness = *sample*[5];
 m.slab_top_slide_thickness = 0;
 m.slab_bottom_slide_thickness = 0;

This code is used in section 84.

86. Just move the values from the illumination array into the right places. Need to spend time to figure out how to integrate items 2, 3, and 4

$\langle$ handle2 illumination 86 $\rangle \equiv$
 m.d.beam = *illumination*[0]; /* *m.lambda* = *illumination*[1]; */ /* *m.specular-reflection-excluded*
 = *illumination*[2]; */ /* *m.direct-transmission-excluded* = *illumination*[3]; */
 /* *m.diffuse-illumination* = *illumination*[4]; */
 m.num_spheres = *illumination*[5];

This code is used in section 84.

87.

```

⟨ handle2 reflection sphere 87 ⟩ ≡
{
    double d_sample_r, d_entrance_r, d_detector_r;
    m.d_sphere_r = sphere_r[0];
    d_sample_r = sphere_r[1];
    d_entrance_r = sphere_r[2];
    d_detector_r = sphere_r[3];
    m.rw_r = sphere_r[4];
    m.rd_r = sphere_r[5];
    m.as_r = (d_sample_r/m.d_sphere_r/2.0) * (d_sample_r/m.d_sphere_r/2.0);
    m.at_r = (d_entrance_r/m.d_sphere_r/2.0) * (d_entrance_r/m.d_sphere_r/2.0);
    m.ad_r = (d_detector_r/m.d_sphere_r/2.0) * (d_detector_r/m.d_sphere_r/2.0);
    m.aw_r = 1.0 - m.as_r - m.at_r - m.ad_r;
}

```

This code is used in section 84.

88.

```

⟨ handle2 transmission sphere 88 ⟩ ≡
{
    double d_sample_t, d_entrance_t, d_detector_t;
    m.d_sphere_t = sphere_t[0];
    d_sample_t = sphere_t[1];
    d_entrance_t = sphere_t[2];
    d_detector_t = sphere_t[3];
    m.rw_t = sphere_t[4];
    m.rd_t = sphere_t[5];
    m.as_t = (d_sample_t/m.d_sphere_t/2.0) * (d_sample_t/m.d_sphere_t/2.0);
    m.at_t = (d_entrance_t/m.d_sphere_t/2.0) * (d_entrance_t/m.d_sphere_t/2.0);
    m.ad_t = (d_detector_t/m.d_sphere_t/2.0) * (d_detector_t/m.d_sphere_t/2.0);
    m.aw_t = 1.0 - m.as_t - m.at_t - m.ad_t;
}

```

This code is used in section 84.

89.

```

⟨ handle2 analysis 89 ⟩ ≡
    r.method.quad_pts = (int) analysis[0];
    mc_runs = (int) analysis[1];
    num_photons = (long) analysis[2];

```

This code is used in section 84.

90.

```

⟨ handle2 measurement 90 ⟩ ≡
    m.rstd_r = measurement[0];
    m.m_r = measurement[1];
    m.m_t = measurement[2];
    m.m_u = measurement[3];
    m.num_measures = 3;
    if (m.m_t == 0) m.num_measures--;
    if (m.m_u == 0) m.num_measures--;

```

This code is used in section 84.

91. IAD Input Output.

The special define below is to get Visual C to suppress silly warnings.

```
<iad_io.c 91> ≡
#define _CRT_SECURE_NO_WARNINGS
#define MAX_COLUMNS 256
    char COLUMN_LABELS[MAX_COLUMNS] = "";
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <math.h>
#include "ad_globl.h"
#include "iad_type.h"
#include "iad_io.h"
#include "iad_util.h"
#include "iad_pub.h"
#include "version.h"

<Definition for get_current_line_number 116>
<Definition for skip_white 103>
<Definition for read_number 105>
<Definition for check_magic 107>
<Definition for remove_whitespace 117>
<Definition for remove_comment 118>
<Definition for remove_first_char 119>
<Definition for print_maybe 120>
<Definition for Read_Data_Legend 122>
<Definition for Read_Data_Line_Per_Labels 101>
<Definition for Read_Header 95>
<Definition for Write_Header 109>
<Definition for Read_Data_Line 100>
```

```
92. <iad_io.h 92> ≡
<Prototype for Read_Header 94>;
<Prototype for Write_Header 108>;
<Prototype for Read_Data_Line 99>;
```

93. Reading the file header.

94. $\langle$ Prototype for *Read_Header* 94 $\rangle \equiv$

```
int Read_Header(FILE *fp, struct measure_type *m, int *params)
```

This code is used in sections 92 and 95.

95. Pretty straightforward stuff. The only thing that needs to be commented on is that only one slide thickness/index is specified in the file. This must be applied to both the top and bottom slides. Finally, to specify no slide, then either setting the slide index to 1.0 or the thickness to 0.0 should do the trick.

$\langle$ Definition for *Read_Header* 95 $\rangle \equiv$

$\langle$ Prototype for *Read_Header* 94 $\rangle$

```
{
    double x;
    Initialize_Measure(m);
    if (check_magic(fp)) return 1;
    if (read_number(fp, &m->slab_index)) return 1;
    if (read_number(fp, &m->slab_top_slide_index)) return 1;
    if (read_number(fp, &m->slab_thickness)) return 1;
    if (read_number(fp, &m->slab_top_slide_thickness)) return 1;
    if (read_number(fp, &m->d_beam)) return 1;
    if (m->slab_top_slide_thickness  $\equiv$  0.0) m->slab_top_slide_index = 1.0;
    if (m->slab_top_slide_index  $\equiv$  1.0) m->slab_top_slide_thickness = 0.0;
    if (m->slab_top_slide_index  $\equiv$  0.0) {
        m->slab_top_slide_thickness = 0.0;
        m->slab_top_slide_index = 1.0;
    }
    m->slab_bottom_slide_index = m->slab_top_slide_index;
    m->slab_bottom_slide_thickness = m->slab_top_slide_thickness;
    if (read_number(fp, &m->rstd_r)) return 1;
    if (read_number(fp, &x)) return 1;
    m->num_spheres = (int) x;
    m->method = SUBSTITUTION;
     $\langle$ Read coefficients for reflection sphere 96 $\rangle$ 
     $\langle$ Read coefficients for transmission sphere 97 $\rangle$ 
     $\langle$ Read info about measurements 98 $\rangle$ 
    return 0;
}
```

This code is used in section 91.

96. So the port areas are normalized by the sphere area

$$a = \frac{A_{port}}{A_{sphere}} = \frac{\pi(d_{port}/2)^2}{4\pi(d_{sphere}/2)^2} = \left[\frac{d_{port}}{2d_{sphere}} \right]^2$$

⟨ Read coefficients for reflection sphere 96 ⟩ ≡

```
{
    double d_sample_r, d_third_r, d_detector_r;
    if (read_number(fp, &m-d_sphere_r)) return 1;
    if (read_number(fp, &d_sample_r)) return 1;
    if (read_number(fp, &d_third_r)) return 1;
    if (read_number(fp, &d_detector_r)) return 1;
    if (read_number(fp, &m-rw_r)) return 1;
    m-as_r = sqr(d_sample_r/(2 * m-d_sphere_r));
    m-at_r = sqr(d_third_r/(2 * m-d_sphere_r));
    m-ad_r = sqr(d_detector_r/(2 * m-d_sphere_r));
    m-aw_r = 1.0 - m-as_r - m-at_r - m-ad_r;
}
```

This code is used in section 95.

97. ⟨ Read coefficients for transmission sphere 97 ⟩ ≡

```
{
    double d_sample_t, d_third_t, d_detector_t;
    if (read_number(fp, &m-d_sphere_t)) return 1;
    if (read_number(fp, &d_sample_t)) return 1;
    if (read_number(fp, &d_third_t)) return 1;
    if (read_number(fp, &d_detector_t)) return 1;
    if (read_number(fp, &m-rw_t)) return 1;
    m-as_t = sqr(d_sample_t/(2 * m-d_sphere_t));
    m-at_t = sqr(d_third_t/(2 * m-d_sphere_t));
    m-ad_t = sqr(d_detector_t/(2 * m-d_sphere_t));
    m-aw_t = 1.0 - m-as_t - m-at_t - m-ad_t;
}
```

This code is used in section 95.

98. ⟨ Read info about measurements 98 ⟩ ≡

```
*params = Read_Data_Legend(fp);
if (COLUMN_LABELS[0] != '\0') {
    m-num_measures = 0;
    if (strchr(COLUMN_LABELS, 'r')) m-num_measures++;
    if (strchr(COLUMN_LABELS, 't')) m-num_measures++;
    if (strchr(COLUMN_LABELS, 'u')) m-num_measures++;
    if (m-num_measures == 0) {
        fprintf(stderr, "Column labels must have at least one 'r', 't', or 'u'\n");
        fprintf(stderr, "Column labels = '%s'\n", COLUMN_LABELS);
        exit(EXIT_FAILURE);
    }
}
else m-num_measures = (*params >= 3) ? 3 : *params;
```

This code is used in section 95.

99. Reading just one line of a data file.

This reads a line of data based on the value of *params*.

If the first number is greater than one then it is assumed to be the wavelength and is ignored. test on the first value of the line.

A non-zero value is returned upon a failure.

⟨Prototype for *Read_Data_Line* 99⟩ ≡

```
int Read_Data_Line(FILE *fp, struct measure_type *m, struct invert_type *r, int params)
```

This code is used in sections 92 and 100.

100. ⟨Definition for *Read_Data_Line* 100⟩ ≡

⟨Prototype for *Read_Data_Line* 99⟩

```
{
  if (strlen(COLUMN_LABELS) > 0) return Read_Data_Line_Per_Labels(fp, m, r, params);
  if (read_number(fp, &m→m_r)) return 1;
  if (m→m_r > 1) {
    m→lambda = m→m_r;
    if (read_number(fp, &m→m_r)) return 1;
  }
  if (params ≡ 1) return 0;
  if (read_number(fp, &m→m_t)) return 1;
  if (params ≡ 2) return 0;
  if (read_number(fp, &m→m_u)) return 1;
  if (params ≡ 3) return 0;
  if (read_number(fp, &m→rw_r)) return 1;
  m→rw_t = m→rw_r;
  if (params ≡ 4) return 0;
  if (read_number(fp, &m→rw_t)) return 1;
  if (params ≡ 5) return 0;
  if (read_number(fp, &m→rstd_r)) return 1;
  if (params ≡ 6) return 0;
  if (read_number(fp, &m→rstd_t)) return 1;
  return 0;
}
```

This code is used in section 91.

101. $\langle$ Definition for *Read_Data_Line_Per_Labels* 101 $\rangle \equiv$

```

int Read_Data_Line_Per_Labels(FILE *fp, struct measure_type *m, struct invert_type *r, int
    params)
{
    int count = 0;
    double x;
    while (count < params) {
        if (read_number(fp, &x)) return 1;
        char c = COLUMN_LABELS[count];
        if (FALSE) fprintf(stderr, "count=%2d, option=%c, value=%10.5f\n", count, c, x);
        switch (c) {
            case 'a': r→default_a = x;
                break;
            case 'A': r→default_mua = x;
                r→default_ba = x * m→slab_thickness;
                break;
            case 'b': r→default_b = x;
                break;
            case 'B': m→d_beam = x;
                break;
            case 'c': m→fraction_of_ru_in_mr = x;
                break;
            case 'C': m→fraction_of_tu_in_mt = x;
                break;
            case 'd': m→slab_thickness = x;
                break;
            case 'D': m→slab_top_slide_thickness = x;
                m→slab_bottom_slide_thickness = x;
                break;
            case 'e': r→tolerance = x;
                r→MC_tolerance = x;
                break;
            case 'E': m→slab_bottom_slide_b = x;
                m→slab_top_slide_b = x;
                break;
            case 'F': r→default_mus = x;
                r→default_bs = x * m→slab_thickness;
                break;
            case 'g': r→default_g = x;
                break;
            case 'L': m→lambda = x;
                break;
            case 'M': m→num_spheres = (int) x;
                break;
            case 'n': m→slab_index = x;
                break;
            case 'N': m→slab_top_slide_index = x;
                m→slab_bottom_slide_index = x;
                break;
            case 'q': r→method_quad_pts = (int) x;
                break;
        }
    }
}

```

```

    case 'r': m-m_r = x;
        break;
    case 'R': m-rstd_r = x;
        break;
    case 't': m-m_t = x;
        break;
    case 'S': m-num_spheres = (int)x;
        break;
    case 'T': m-rstd_t = x;
        break;
    case 'u': m-m_u = x;
        break;
    case 'w': m-rw_r = x;
        break;
    case 'W': m-rw_t = x;
        break;
    default: fprintf(stderr, "legend_variable_ '%c' unimplemented", c);
        return 1;
    }
    count++;
}
return 0;
}

```

This code is used in section 91.

102. Skip over white space and comments. It is assumed that `#` starts all comments and continues to the end of a line. This routine should work on files with nearly any line ending CR, LF, CRLF.

Failure is indicated by a non-zero return value.

⟨Prototype for *skip_white* 102⟩ ≡
int *skip_white*(**FILE** *fp)

This code is used in section 103.

103. ⟨Definition for *skip_white* 103⟩ ≡
 ⟨Prototype for *skip_white* 102⟩
 {
 int c = fgetc(fp);
 while (!feof(fp)) {
 if (isspace(c)) c = fgetc(fp);
 else if (c == '#') **do** c = fgetc(fp); **while** (!feof(fp) ∧ c ≠ '\n' ∧ c ≠ '\r');
 else break;
 }
 if (feof(fp)) **return** 1;
 ungetc(c, fp);
 return 0;
 }

This code is used in section 91.

104. Read a single number. Return 0 if there are no problems, otherwise return 1.

⟨Prototype for *read_number* 104⟩ ≡
int *read_number*(**FILE** *fp, **double** *x)

This code is used in section 105.

105. $\langle$ Definition for *read_number* 105 $\rangle \equiv$

$\langle$ Prototype for *read_number* 104 $\rangle$

```
{
    long line_no = 0;
    if (skip_white(fp)) return 1;
    if (fscanf(fp, "%lf", x)) return 0;
    line_no = get_current_line_number(fp);
    fprintf(stderr, "\nIAD_ERROR: Bad number on line %ld in input file.", line_no);
    return 1;
}
```

This code is used in section 91.

106. Ensure that the data file is actually in the right form. Return 0 if the file has the right starting characters. Return 1 if on a failure.

$\langle$ Prototype for *check_magic* 106 $\rangle \equiv$

int *check_magic*(**FILE** *fp)

This code is used in section 107.

107. $\langle$ Definition for *check_magic* 107 $\rangle \equiv$

$\langle$ Prototype for *check_magic* 106 $\rangle$

```
{
    char magic[] = "IAD1";
    int i, c;
    for (i = 0; i < 4; i++) {
        c = fgetc(fp);
        if (feof(fp)  $\vee$  c  $\neq$  magic[i]) {
            fprintf(stderr, "Sorry, but iad input files must begin with IAD1\n");
            fprintf(stderr, "as the first four characters of the file.\n");
            fprintf(stderr, "Perhaps you are using an old iad format?\n");
            return 1;
        }
    }
    return 0;
}
```

This code is used in section 91.

108. Formatting the header information.

⟨Prototype for *Write_Header* 108⟩ ≡

```
void Write_Header(struct measure_type m, struct invert_type r, int params, char *cmd)
```

This code is used in sections 92 and 109.

109. ⟨Definition for *Write_Header* 109⟩ ≡

⟨Prototype for *Write_Header* 108⟩

```
{
  ⟨Write slab info 110⟩
  ⟨Write irradiation info 111⟩
  ⟨Write general sphere info 112⟩
  ⟨Write first sphere info 113⟩
  ⟨Write second sphere info 114⟩
  ⟨Write measure and inversion info 115⟩
}
```

This code is used in section 91.

110. ⟨Write slab info 110⟩ ≡

```
double xx;

printf("#_Inverse_Adding-Doubling_%s_\n", Version);
printf("#_%s_\n", cmd);
printf("#_Beam_diameter=_");
print_maybe('B', "%7.1f_mm\n", m.d_beam);
printf("#_Sample_thickness=_");
print_maybe('d', "%7.3f_mm\n", m.slab_thickness);
printf("#_Top_slide_thickness=_");
print_maybe('D', "%7.3f_mm\n", m.slab_top_slide_thickness);
printf("#_Bottom_slide_thickness=_");
print_maybe('D', "%7.3f_mm\n", m.slab_bottom_slide_thickness);
printf("#_Sample_index_of_refraction=_");
print_maybe('n', "%7.4f_\n", m.slab_index);
printf("#_Top_slide_index_of_refraction=_");
print_maybe('N', "%7.4f_\n", m.slab_top_slide_index);
printf("#_Bottom_slide_index_of_refraction=_");
print_maybe('N', "%7.4f_\n", m.slab_bottom_slide_index);
```

This code is used in section 109.

111. ⟨Write irradiation info 111⟩ ≡

```
printf("#_\n");
```

This code is used in section 109.

112. ⟨Write general sphere info 112⟩ ≡

```
printf("#_Percentage_unscattered_refl._in_M_R=_");
print_maybe('c', "%7.1f_%_\n", m.fraction_of_ru_in_mr * 100);
printf("#_Percentage_unscattered_trans._in_M_T=_");
print_maybe('C', "%7.1f_%_\n", m.fraction_of_tu_in_mt * 100);
printf("#_\n");
```

This code is used in section 109.

113. $\langle$ Write first sphere info 113 $\rangle \equiv$

```
printf("#_Reflection_sphere");
if (m.baffle_r) printf("_has_a_baffle_between_sample_and_detector");
else printf("_has_no_baffle_between_sample_and_detector");
if (m.num_spheres > 0) printf("\n");
else printf("_ignored_since_no_spheres_used\n");
printf("#_sphere_diameter=_%7.1f_mm\n", m.d_sphere_r);
printf("#_sample_port_diameter=_%7.1f_mm\n", 2 * m.d_sphere_r * sqrt(m.as_r));
printf("#_entrance_port_diameter=_%7.1f_mm\n", 2 * m.d_sphere_r * sqrt(m.at_r));
printf("#_detector_port_diameter=_%7.1f_mm\n", 2 * m.d_sphere_r * sqrt(m.ad_r));
printf("#_detector_reflectance=_%7.1f%%\n", m.rd_r * 100);
printf("#_wall_reflectance=_");
printf("'%w', "%7.1f%%\n", m.rw_r * 100);
printf("#_calibration_standard=_");
printf("'%R', "%7.1f%%\n", m.rstd_r * 100);
printf("#\n");
```

This code is used in section 109.

114. $\langle$ Write second sphere info 114 $\rangle \equiv$

```
printf("#_Transmission_sphere");
if (m.baffle_t) printf("_has_a_baffle_between_sample_and_detector");
else printf("_has_no_baffle_between_sample_and_detector");
if (m.num_spheres > 0) printf("\n");
else printf("_ignored_since_no_spheres_used\n");
printf("#_sphere_diameter=_%7.1f_mm\n", m.d_sphere_t);
printf("#_sample_port_diameter=_%7.1f_mm\n", 2 * m.d_sphere_r * sqrt(m.as_t));
printf("#_third_port_diameter=_%7.1f_mm\n", 2 * m.d_sphere_r * sqrt(m.at_t));
printf("#_detector_port_diameter=_%7.1f_mm\n", 2 * m.d_sphere_r * sqrt(m.ad_t));
printf("#_detector_reflectance=_%7.1f%%\n", m.rd_t * 100);
if (m.at_t == 0) printf("#_wall_reflectance_and_cal_standard=_");
else printf("#_wall_reflectance=_");
printf("'%w', "%7.1f%%\n", m.rw_t * 100);
printf("#_calibration_standard=_%7.1f%%", m.rstd_t * 100);
if (m.at_t == 0) printf("_ignored");
printf("\n");
```

This code is used in section 109.

```

115.  < Write measure and inversion info 115 > ≡
printf("#\n");
if (COLUMN_LABELS[0] ≡ '\0') {
    switch (params) {
        case -1: printf("#No_M_R_or_M_T--_forward_calculation");
            break;
        case 1: printf("#_Just_M_R_was_measured");
            break;
        case 2: printf("#_M_R_and_M_T_were_measured");
            break;
        case 3: printf("#_M_R,_M_T,_and_M_U_were_measured");
            break;
        case 4: printf("#_M_R,_M_T,_M_U,_and_r_w_were_measured");
            break;
        case 5: printf("#_M_R,_M_T,_M_U,_r_w,_and_t_w_were_measured");
            break;
        case 6: printf("#_M_R,_M_T,_M_U,_r_w,_t_w,_and_r_std_were_measured");
            break;
        case 7: printf("#_M_R,_M_T,_M_U,_r_w,_t_w,_r_std_and_t_std_were_measured");
            break;
        default: printf("#Something_went_wrong..._measures_should_be_1_to_7!\n");
            break;
    }
}
else {
    int i;
    printf("#%d_input_columns_with_LABELS:", params);
    for (i = 0; i < params; i++) {
        printf(" %c", COLUMN_LABELS[i]);
    }
}
if (m.flip_sample) printf("_ (sample_flipped)");
switch (m.method) {
    case UNKNOWN: printf("_using_an_unknown_method.\n");
        break;
    case SUBSTITUTION: printf("_using_the_substitution_(single-beam)_method.\n");
        break;
    case COMPARISON: printf("_using_the_comparison_(dual-beam)_method.\n");
}
switch (m.num_spheres) {
    case 0: printf("#No_sphere_corrections_were_used");
        break;
    case 1:
        if (m.method ≡ COMPARISON) printf("#No_sphere_corrections_were_needed");
        else printf("#Single_sphere_corrections_were_used");
        break;
    case 2: printf("#Double_sphere_corrections_were_used");
        break;
}
printf("_and_light_was_incident_at_%d_degrees_from_the_normal",
    (int)(acos(m.slabs_cos_angle) * 57.2958));
printf(".\n");

```

```

switch (r.search) {
case FIND_AB: printf("#_The_inverse_routine_varied_the_albedo_and_optical_depth.\n");
               printf("#_\n");
               xx = (r.default_g ≠ UNINITIALIZED) ? r.default_g : 0;
               printf("#_Default_single_scattering_anisotropy=_%7.3f\n", xx);
               break;
case FIND_AG: printf("#_The_inverse_routine_varied_the_albedo_and_anisotropy.\n");
               printf("#_\n");
               if (r.default_b ≠ UNINITIALIZED)
                   printf("#_Default(mu_t*d)=_%7.3g\n", r.default_b);
               else printf("#_\n");
               break;
case FIND_AUTO: printf("#_The_inverse_routine_adapted_to_the_input_data.\n");
                 printf("#_\n");
                 printf("#_\n");
                 break;
case FIND_A: printf("#_The_inverse_routine_varied_only_the_albedo.\n");
              printf("#_\n");
              xx = (r.default_g ≠ UNINITIALIZED) ? r.default_g : 0;
              printf("#_Default_single_scattering_anisotropy_is_%7.3f", xx);
              xx = (r.default_b ≠ UNINITIALIZED) ? r.default_b : HUGE_VAL;
              printf("_and(mu_t*d)=_%7.3g\n", xx);
              break;
case FIND_B: printf("#_The_inverse_routine_varied_only_the_optical_depth.\n");
              printf("#_\n");
              xx = (r.default_g ≠ UNINITIALIZED) ? r.default_g : 0;
              printf("#_Default_single_scattering_anisotropy_is_%7.3f", xx);
              if (r.default_a ≠ UNINITIALIZED) printf("and_default_albedo=_%7.3g\n", r.default_a);
              else printf("\n");
              break;
case FIND_Ba: printf("#_The_inverse_routine_varied_only_the_absorption.\n");
               printf("#_\n");
               xx = (r.default_bs ≠ UNINITIALIZED) ? r.default_bs : 0;
               printf("#_Default(mu_s*d)=_%7.3g\n", xx);
               break;
case FIND_Bs: printf("#_The_inverse_routine_varied_only_the_scattering.\n");
               printf("#_\n");
               xx = (r.default_ba ≠ UNINITIALIZED) ? r.default_ba : 0;
               printf("#_Default(mu_a*d)=_%7.3g\n", xx);
               break;
default: printf("#_\n");
          printf("#_\n");
          printf("#_\n");
          break;
}
printf("#_AD_quadrature_points=_%3d\n", r.method.quad_pts);
printf("#_AD_tolerance_for_success=_%9.5f\n", r.tolerance);
printf("#_MC_tolerance_for_mu_a_and_mu_s'=_%7.3f%%\n", r.MC_tolerance);

```

This code is used in section 109.

116. Get the current line number by counting.

⟨ Definition for *get_current_line_number* 116 ⟩ ≡

```

long get_current_line_number(FILE *file)
{
    long line_number = 0;
    long current_position = ftell(file);
    fseek(file, 0, SEEK_SET);
    int c;
    while ((c = fgetc(file)) ≠ EOF ∧ ftell(file) < current_position) {
        if (c ≡ '\n') {
            line_number++;
        }
    }
    fseek(file, current_position, SEEK_SET);
    return line_number + 1;
}

```

This code is used in section 91.

117. Discard white space and dashes in the legend string

⟨ Definition for *remove_whitespace* 117 ⟩ ≡

```

void remove_whitespace(char *str)
{
    int i, j = 0;
    for (i = 0; str[i] ≠ '\0'; i++) {
        if (¬isspace(str[i]) ∧ str[i] ≠ '-') {
            str[j++] = str[i];
        }
    }
    str[j] = '\0';
}

```

This code is used in section 91.

118. ⟨ Definition for *remove_comment* 118 ⟩ ≡

```

void remove_comment(char *str)
{
    int i;
    for (i = 0; str[i] ≠ '\0'; i++) {
        if (str[i] ≡ '#') {
            str[i] = '\0';
            break;
        }
    }
}

```

This code is used in section 91.

119. $\langle$ Definition for *remove_first_char* 119 $\rangle \equiv$

```
void remove_first_char(char *str)
{
    int len = strlen(str);
    if (len > 0) {
        for (int i = 0; i < len; i++) {
            str[i] = str[i + 1];
        }
    }
}
```

This code is used in section 91.

120. $\langle$ Definition for *print_maybe* 120 $\rangle \equiv$

```
void print_maybe(char c, char *format, double x)
{
    char *result = strchr(COLUMN_LABELS, c);
    if (result  $\equiv \Lambda$ ) printf(format, x);
    else printf("_(varies_with_input_row)\n");
}
```

This code is used in section 91.

121. $\langle$ Prototype for *Read_Data_Legend* 121 $\rangle \equiv$

```
int Read_Data_Legend(FILE *fp)
```

This code is used in section 122.

122. $\langle$ Definition for *Read_Data_Legend* 122 $\rangle \equiv$

```
 $\langle$  Prototype for Read_Data_Legend 121  $\rangle$ 
{
    int n = 0;
    char c;
    skip_white(fp);
    if (fgets(COLUMN_LABELS, MAX_COLUMNS, fp)  $\equiv \Lambda$ ) {
        fprintf(stderr, "could_not_read_Data_Legend_String_in_file\n");
        exit(EXIT_FAILURE);
    }
    remove_whitespace(COLUMN_LABELS);
    remove_comment(COLUMN_LABELS);
    c = COLUMN_LABELS[0];
    if (c  $\equiv$  '1'  $\vee$  c  $\equiv$  '2'  $\vee$  c  $\equiv$  '3'  $\vee$  c  $\equiv$  '4'  $\vee$  c  $\equiv$  '5'  $\vee$  c  $\equiv$  '6'  $\vee$  c  $\equiv$  '7') {
        n = COLUMN_LABELS[0] - '0';
        COLUMN_LABELS[0] = '\\0';
    }
    else {
        n = strlen(COLUMN_LABELS);
    }
    return n;
}
```

This code is used in section 91.

123. IAD Calculation.

```

⟨ iad_calc.c 123 ⟩ ≡
#include <math.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include "nr_util.h"
#include "nr_zbrent.h"
#include "ad_globl.h"
#include "ad_frsnl.h"
#include "ad_prime.h"
#include "iad_type.h"
#include "iad_util.h"
#include "iad_calc.h"
#define ABIT 1·10-6
#define A_COLUMN 1
#define B_COLUMN 2
#define G_COLUMN 3
#define URU_COLUMN 4
#define UTU_COLUMN 5
#define UR1_COLUMN 6
#define UT1_COLUMN 7
#define REFLECTION_SPHERE 1
#define TRANSMISSION_SPHERE 0
#define T_TRUST_FACTOR 1
#define SWAP(a,b)
{
    double swap = (a);
    (a) = (b);
    (b) = swap;
}
static int CALCULATING_GRID = 0;
static struct measure_type MM;
static struct invert_type RR;
static struct measure_type MGRID;
static struct invert_type RGRID;
static double **The_Grid = Λ;
static double GG_a;
static double GG_b;
static double GG_g;
static double GG_bs;
static double GG_ba;
static boolean_type The_Grid_Initialized = FALSE;
static boolean_type The_Grid_Search = -1;
⟨ Definition for Set_Calc_State 139 ⟩
⟨ Definition for Get_Calc_State 141 ⟩
⟨ Definition for Same_Calc_State 143 ⟩
⟨ Prototype for Fill_AB_Grid 161 ⟩;
⟨ Prototype for Fill_AG_Grid 167 ⟩;
⟨ Definition for RT_Flip 159 ⟩
⟨ Definition for Allocate_Grid 145 ⟩

```

- ⟨ Definition for *Valid_Grid* 149 ⟩
- ⟨ Definition for *fill_grid_entry* 160 ⟩
- ⟨ Definition for *Fill_Grid* 177 ⟩
- ⟨ Definition for *Near_Grid_Points* 157 ⟩
- ⟨ Definition for *Fill_AB_Grid* 162 ⟩
- ⟨ Definition for *Fill_AG_Grid* 168 ⟩
- ⟨ Definition for *Fill_BG_Grid* 171 ⟩
- ⟨ Definition for *Fill_BaG_Grid* 173 ⟩
- ⟨ Definition for *Fill_BsG_Grid* 175 ⟩
- ⟨ Definition for *Grid_ABG* 147 ⟩
- ⟨ Definition for *Gain* 128 ⟩
- ⟨ Definition for *Gain_11* 130 ⟩
- ⟨ Definition for *Gain_22* 132 ⟩
- ⟨ Definition for *Two_Sphere_R* 134 ⟩
- ⟨ Definition for *Two_Sphere_T* 136 ⟩
- ⟨ Definition for *Calculate_Distance_With_Corrections* 183 ⟩
- ⟨ Definition for *Calculate_Grid_Distance* 181 ⟩
- ⟨ Definition for *Calculate_Distance* 179 ⟩
- ⟨ Definition for *abg_distance* 155 ⟩
- ⟨ Definition for *Find_AG_fn* 208 ⟩
- ⟨ Definition for *Find_AB_fn* 210 ⟩
- ⟨ Definition for *Find_Ba_fn* 212 ⟩
- ⟨ Definition for *Find_Bs_fn* 214 ⟩
- ⟨ Definition for *Find_A_fn* 216 ⟩
- ⟨ Definition for *Find_B_fn* 218 ⟩
- ⟨ Definition for *Find_G_fn* 220 ⟩
- ⟨ Definition for *Find_BG_fn* 222 ⟩
- ⟨ Definition for *Find_BaG_fn* 224 ⟩
- ⟨ Definition for *Find_BsG_fn* 226 ⟩
- ⟨ Definition for *maxloss* 228 ⟩
- ⟨ Definition for *Max_Light_Loss* 230 ⟩

124.

```

< iad_calc.h 124 > ≡
  < Prototype for Gain 127 >;
  < Prototype for Gain_11 129 >;
  < Prototype for Gain_22 131 >;
  < Prototype for Two_Sphere_R 133 >;
  < Prototype for Two_Sphere_T 135 >;
  < Prototype for Set_Calc_State 138 >;
  < Prototype for Get_Calc_State 140 >;
  < Prototype for Same_Calc_State 142 >;
  < Prototype for Valid_Grid 148 >;
  < Prototype for Allocate_Grid 144 >;
  < Prototype for Fill_Grid 176 >;
  < Prototype for Near_Grid_Points 156 >;
  < Prototype for Grid_ABG 146 >;
  < Prototype for Find_AG_fn 207 >;
  < Prototype for Find_AB_fn 209 >;
  < Prototype for Find_Ba_fn 211 >;
  < Prototype for Find_Bs_fn 213 >;
  < Prototype for Find_A_fn 215 >;
  < Prototype for Find_B_fn 217 >;
  < Prototype for Find_G_fn 219 >;
  < Prototype for Find_BG_fn 221 >;
  < Prototype for Find_BsG_fn 225 >;
  < Prototype for Find_BaG_fn 223 >;
  < Prototype for Fill_BG_Grid 170 >;
  < Prototype for Fill_BsG_Grid 174 >;
  < Prototype for Fill_BaG_Grid 172 >;
  < Prototype for Calculate_Distance_With_Corrections 182 >;
  < Prototype for Calculate_Distance 178 >;
  < Prototype for Calculate_Grid_Distance 180 >;
  < Prototype for abg_distance 154 >;
  < Prototype for maxloss 227 >;
  < Prototype for Max_Light_Loss 229 >;
  < Prototype for RT_Flip 158 >;

```

125. Initialization.

The functions in this file assume that the local variables **MM** and **RR** have been initialized appropriately. The variable **MM** contains all the information about how a particular experiment was done. The structure **RR** contains the data structure that is passed to the adding-doubling routines as well as the number of quadrature points.

126. Gain.

Assume that a sphere is illuminated with diffuse light having a power P . This light will undergo multiple reflections in the sphere walls that will increase the power falling on the detector. The gain on the detector due to integrating sphere effects varies with the presence of a baffle between the sample and the detector. If a baffle is present then

$$G_{\text{no baffle}}(r_s, r_t) = \frac{1}{1 - a_w r_w - a_d r_d - a_s r_s - a_s r_t}$$

or with a baffle as

$$G_{\text{baffle}}(r_s, r_t) = \frac{1}{1 - a_w r'_w - r'_w (1 - a_t)(a_d r_d + a_s r_s)}$$

where

$$r'_w = r_w + (a_t/a_w)r_t$$

For a black sphere the gain does not depend on the diffuse reflectivity of the sample and is unity. $G(r_s) = 1$, which is easily verified by setting $r_w = 0$.

The value *uru_sample* is the total reflectance from the sample for diffuse incident light and *uru_third* is the total reflectance from the third port for diffuse incident light. For a reflection sphere, the third port is the entrance port and *uru_third* = 0.

127. $\langle$ Prototype for *Gain* 127 $\rangle \equiv$

double *Gain*(**int** *sphere*, **struct** *measure_type* *m*, **double** *uru_sample*, **double** *uru_third*)

This code is used in sections 124 and 128.

128. $\langle$ Definition for *Gain* 128 $\rangle \equiv$

$\langle$ Prototype for *Gain* 127 $\rangle$

```
{
    double inv_gain;
    if (sphere  $\equiv$  REFLECTION_SPHERE) {
        if (m.baffle_r) {
            inv_gain = m.rw_r + (m.at_r/m.aw_r) * uru_third;
            inv_gain *= m.aw_r + (1 - m.at_r) * (m.ad_r * m.rd_r + m.as_r * uru_sample);
            inv_gain = 1.0 - inv_gain;
        }
        else {
            inv_gain = 1.0 - m.aw_r * m.rw_r - m.ad_r * m.rd_r - m.as_r * uru_sample - m.at_r * uru_third;
        }
    }
    else if (m.baffle_t) {
        inv_gain = m.rw_t + (m.at_t/m.aw_t) * uru_third;
        inv_gain *= m.aw_t + (1 - m.at_t) * (m.ad_t * m.rd_t + m.as_t * uru_sample);
        inv_gain = 1.0 - inv_gain;
    }
    else {
        inv_gain = 1.0 - m.aw_t * m.rw_t - m.ad_t * m.rd_t - m.as_t * uru_sample - m.at_t * uru_third;
    }
    return 1.0/inv_gain;
}
```

This code is used in section 123.

129. The gain for light on the detector in the first sphere for diffuse light starting in that same sphere is defined as

$$G_{1 \rightarrow 1}(r_s, t_s) \equiv \frac{P_{1 \rightarrow 1}(r_s, t_s)/A_d}{P/A}$$

then the full expression for the gain is

$$G_{1 \rightarrow 1}(r_s, t_s) = \frac{G(r_s)}{1 - a_s a'_s r_w r'_w (1 - a_t)(1 - a'_t) G(r_s) G'(r_s) t_s^2}$$

⟨Prototype for *Gain_11* 129⟩ ≡

```
double Gain_11(struct measure_type m, double URU, double tdiffuse)
```

This code is used in sections 124 and 130.

130. ⟨Definition for *Gain_11* 130⟩ ≡

⟨Prototype for *Gain_11* 129⟩

```
{
  double G, GP, G11;
  G = Gain(REFLECTION_SPHERE, m, URU, 0);
  GP = Gain(TRANSMISSION_SPHERE, m, URU, 0);
  G11 = G / (1 - m.as_r * m.as_t * m.aw_r * m.aw_t * (1 - m.at_r) * (1 - m.at_t) * G * GP * tdiffuse * tdiffuse);
  return G11;
}
```

This code is used in section 123.

131. Similarly, when the light starts in the second sphere, the gain for light on the detector in the second sphere $G_{2 \rightarrow 2}$ is found by switching all primed variables to unprimed. Thus $G_{2 \rightarrow 1}(r_s, t_s)$ is

$$G_{2 \rightarrow 2}(r_s, t_s) = \frac{G'(r_s)}{1 - a_s a'_s r_w r'_w (1 - a_t)(1 - a'_t) G(r_s) G'(r_s) t_s^2}$$

⟨Prototype for *Gain_22* 131⟩ ≡

```
double Gain_22(struct measure_type m, double URU, double tdiffuse)
```

This code is used in sections 124 and 132.

132. ⟨Definition for *Gain_22* 132⟩ ≡

⟨Prototype for *Gain_22* 131⟩

```
{
  double G, GP, G22;
  G = Gain(REFLECTION_SPHERE, m, URU, 0);
  GP = Gain(TRANSMISSION_SPHERE, m, URU, 0);
  G22 = GP / (1 - m.as_r * m.as_t * m.aw_r * m.aw_t * (1 - m.at_r) * (1 - m.at_t) * G * GP * tdiffuse * tdiffuse);
  return G22;
}
```

This code is used in section 123.

133. The reflected power for two spheres makes use of the formulas for *Gain_11* above.

The light on the detector in the reflection (first) sphere arises from three sources: the fraction of light directly reflected off the sphere wall $fr_w^2(1 - a_t)P$, the fraction of light reflected by the sample $(1 - f)r_s^{\text{direct}}r_w^2(1 - a_t)P$, and the light transmitted through the sample $(1 - f)t_s^{\text{direct}}r'_w(1 - a'_t)P$,

$$\begin{aligned} R(r_s^{\text{direct}}, r_s, t_s^{\text{direct}}, t_s) &= G_{1 \rightarrow 1}(r_s, t_s) \cdot a_d(1 - a_t)r_w^2fP \\ &\quad + G_{1 \rightarrow 1}(r_s, t_s) \cdot a_d(1 - a_t)r_w(1 - f)r_s^{\text{direct}}P \\ &\quad + G_{2 \rightarrow 1}(r_s, t_s) \cdot a_d(1 - a'_t)r'_w(1 - f)t_s^{\text{direct}}P \end{aligned}$$

which simplifies slightly to

$$\begin{aligned} R(r_s^{\text{direct}}, r_s, t_s^{\text{direct}}, t_s) &= a_d(1 - a_t)r_wP \cdot G_{1 \rightarrow 1}(r_s, t_s) \\ &\quad \times \left[(1 - f)r_s^{\text{direct}} + fr_w + (1 - f)a'_s(1 - a'_t)r'_wt_s^{\text{direct}}t_sG'(r_s) \right] \end{aligned}$$

$\langle \text{Prototype for } Two_Sphere_R \text{ 133} \rangle \equiv$

double *Two_Sphere_R*(**struct** *measure_type* *m*, **double** UR1, **double** URU, **double** UT1, **double** UTU)

This code is used in sections 124 and 134.

134. $\langle \text{Definition for } Two_Sphere_R \text{ 134} \rangle \equiv$

$\langle \text{Prototype for } Two_Sphere_R \text{ 133} \rangle$

```
{
  double x, GP;
  GP = Gain(TRANSMISSION_SPHERE, m, URU, 0);
  x = m.ad_r * (1 - m.at_r) * m.rw_r * Gain_11(m, URU, UTU);
  x *= (1 - m.f_r) * UR1 + m.rw_r * m.f_r + (1 - m.f_r) * m.as_t * (1 - m.at_t) * m.rw_t * UT1 * UTU * GP;
  return x;
}
```

This code is used in section 123.

135. For the power on the detector in the transmission (second) sphere we have the same three sources. The only difference is that the subscripts on the gain terms now indicate that the light ends up in the second sphere

$$\begin{aligned} T(r_s^{\text{direct}}, r_s, t_s^{\text{direct}}, t_s) &= G_{1 \rightarrow 2}(r_s, t_s) \cdot a'_d(1 - a_t)r_w^2fP \\ &\quad + G_{1 \rightarrow 2}(r_s, t_s) \cdot a'_d(1 - a_t)r_w(1 - f)r_s^{\text{direct}}P \\ &\quad + G_{2 \rightarrow 2}(r_s, t_s) \cdot a'_d(1 - a'_t)r'_w(1 - f)t_s^{\text{direct}}P \end{aligned}$$

or

$$\begin{aligned} T(r_s^{\text{direct}}, r_s, t_s^{\text{direct}}, t_s) &= a'_d(1 - a'_t)r'_wP \cdot G_{2 \rightarrow 2}(r_s, t_s) \\ &\quad \times \left[(1 - f)t_s^{\text{direct}} + (1 - a_t)r_wa_s t_s(fr_w + (1 - f)r_s^{\text{direct}})G(r_s) \right] \end{aligned}$$

$\langle \text{Prototype for } Two_Sphere_T \text{ 135} \rangle \equiv$

double *Two_Sphere_T*(**struct** *measure_type* *m*, **double** UR1, **double** URU, **double** UT1, **double** UTU)

This code is used in sections 124 and 136.

136. $\langle \text{Definition for } Two_Sphere_T \text{ 136} \rangle \equiv$
 $\langle \text{Prototype for } Two_Sphere_T \text{ 135} \rangle$

```
{
  double x, G;
  G = Gain(REFLECTION_SPHERE, m, URU, 0);
  x = m.ad_t * (1 - m.at_t) * m.rw_t * Gain_22(m, URU, UTU);
  x *= (1 - m.f_r) * UT1 + (1 - m.at_r) * m.rw_r * m.as_r * UTU * (m.f_r * m.rw_r + (1 - m.f_r) * UR1) * G;
  return x;
}
```

This code is used in section 123.

137. Grid Routines. There is a long story associated with these routines. I spent a lot of time trying to find an empirical function to allow a guess at a starting value for the inversion routine. Basically nothing worked very well. There were too many special cases and what not. So I decided to calculate a whole bunch of reflection and transmission values and keep their associated optical properties linked nearby.

I did the very simplest thing. I just allocate a matrix that is five columns wide. Then I fill every row with a calculated set of optical properties and observables. The distribution of values that I use could certainly use some work, but they currently work.

SO... how does this thing work anyway? There are two possible grids one for calculations requiring the program to find the albedo and the optical depth (a and b) and one to find the albedo and anisotropy (a and g). These grids must be allocated and initialized before use.

138. This is a pretty important routine that should have some explanation. The reason that it exists, is that we need some ‘out-of-band’ information during the minimization process. Since the light transport calculation depends on all sorts of stuff (e.g., the sphere parameters) and the minimization routines just vary one or two parameters this information needs to be put somewhere.

I chose the global variables MM and RR to save things in.

The bottom line is that you cannot do a light transport calculation without calling this routine first.

⟨Prototype for *Set_Calc_State* 138⟩ ≡

```
void Set_Calc_State(struct measure_type m, struct invert_type r)
```

This code is used in sections 124 and 139.

139. ⟨Definition for *Set_Calc_State* 139⟩ ≡

⟨Prototype for *Set_Calc_State* 138⟩

```
{
    memcpy(&MM, &m, sizeof(struct measure_type));
    memcpy(&RR, &r, sizeof(struct invert_type));
}
```

This code is used in section 123.

140. The inverse of the previous routine. Note that you must have space for the parameters m and r already allocated.

⟨Prototype for *Get_Calc_State* 140⟩ ≡

```
void Get_Calc_State(struct measure_type *m, struct invert_type *r)
```

This code is used in sections 124 and 141.

141. ⟨Definition for *Get_Calc_State* 141⟩ ≡

⟨Prototype for *Get_Calc_State* 140⟩

```
{
    memcpy(m, &MM, sizeof(struct measure_type));
    memcpy(r, &RR, sizeof(struct invert_type));
}
```

This code is used in section 123.

142. The inverse of the previous routine. Note that you must have space for the parameters m and r already allocated.

⟨Prototype for *Same_Calc_State* 142⟩ ≡

```
boolean_type Same_Calc_State(struct measure_type m, struct invert_type r)
```

This code is used in sections 124 and 143.

143. $\langle$ Definition for *Same_Calc_State* 143 $\rangle \equiv$
 $\langle$ Prototype for *Same_Calc_State* 142 $\rangle$
{
 if (*The_Grid* $\equiv \Lambda$) **return** FALSE;
 if ($\neg$ *The_Grid_Initialized*) **return** FALSE;
 if (*r.search* $\neq$ *RR.search*) **return** FALSE;
 if (*r.method.quad_pts* $\neq$ *RR.method.quad_pts*) **return** FALSE;
 if (*r.slab.a* $\neq$ *RR.slab.a*) **return** FALSE;
 if (*r.slab.b* $\neq$ *RR.slab.b*) **return** FALSE;
 if (*r.slab.g* $\neq$ *RR.slab.g*) **return** FALSE;
 if (*r.slab.phase_function* $\neq$ *RR.slab.phase_function*) **return** FALSE;
 if (*r.slab.n_slab* $\neq$ *RR.slab.n_slab*) **return** FALSE;
 if (*r.slab.n_top_slide* $\neq$ *RR.slab.n_top_slide*) **return** FALSE;
 if (*r.slab.n_bottom_slide* $\neq$ *RR.slab.n_bottom_slide*) **return** FALSE;
 if (*r.slab.b_top_slide* $\neq$ *RR.slab.b_top_slide*) **return** FALSE;
 if (*r.slab.b_bottom_slide* $\neq$ *RR.slab.b_bottom_slide*) **return** FALSE;
 if (*r.slab.cos_angle* $\neq$ *RR.slab.cos_angle*) **return** FALSE;
 if (*m.num_measures* $\equiv 3$) $\wedge$ (*m.m_u* $\neq$ *MGRID.m_u*) **return** (FALSE);
 return TRUE;
}

This code is used in section 123.

144. $\langle$ Prototype for *Allocate_Grid* 144 $\rangle \equiv$
void *Allocate_Grid*(**search_type** *s*)

This code is used in sections 124 and 145.

145. $\langle$ Definition for *Allocate_Grid* 145 $\rangle \equiv$
 $\langle$ Prototype for *Allocate_Grid* 144 $\rangle$
{
 (**void**) *s*;
 The_Grid = *dmatrix*(0, GRID_SIZE * GRID_SIZE, 1, 7);
 if (*The_Grid* $\equiv \Lambda$) *AD_error*("unable to allocate the grid matrix");
 The_Grid_Initialized = FALSE;
}

This code is used in section 123.

146. This routine will return the *a*, *b*, and *g* values for a particular row in the grid.

$\langle$ Prototype for *Grid_ABG* 146 $\rangle \equiv$
void *Grid_ABG*(**int** *i*, **int** *j*, **guess_type** **guess*)

This code is used in sections 124 and 147.

147. $\langle \text{Definition for } Grid_ABG \text{ 147} \rangle \equiv$
 $\langle \text{Prototype for } Grid_ABG \text{ 146} \rangle$

```

{
    if (0 ≤ i ∧ i < GRID_SIZE ∧ 0 ≤ j ∧ j < GRID_SIZE) {
        guess→a = The_Grid[GRID_SIZE * i + j][A_COLUMN];
        guess→b = The_Grid[GRID_SIZE * i + j][B_COLUMN];
        guess→g = The_Grid[GRID_SIZE * i + j][G_COLUMN];
        guess→distance = Calculate_Grid_Distance(i, j);
    }
    else {
        guess→a = 0.5;
        guess→b = 0.5;
        guess→g = 0.5;
        guess→distance = 999;
    }
}

```

This code is used in section 123.

148. This routine is used to figure out if the current grid is valid. This can fail for several reasons. First the grid may not have been allocated. Or it may not have been initialized. The boundary conditions may have changed. The number or values of the sphere parameters may have changed. It is tedious, but straightforward to check these cases out.

If this routine returns true, then it is a pretty good bet that the values in the current grid can be used to guess the next starting set of values.

$\langle \text{Prototype for } Valid_Grid \text{ 148} \rangle \equiv$
boolean_type *Valid_Grid*(**struct** *measure_type* *m*, **struct** *invert_type* *r*)

This code is used in sections 124 and 149.

149. $\langle \text{Definition for } Valid_Grid \text{ 149} \rangle \equiv$
 $\langle \text{Prototype for } Valid_Grid \text{ 148} \rangle$

```

{
    int s = r.search;
     $\langle \text{Tests for invalid grid 150} \rangle$ 
    return (TRUE);
}

```

This code is used in section 123.

150. First check are to test if the grid has ever been filled

$\langle \text{Tests for invalid grid 150} \rangle \equiv$

```

if (The_Grid ≡ Λ) {
    if (Debug(DEBUG_GRID)) fprintf(stderr, "GRID: Fill because NULL\n");
    return (FALSE);
}
if (¬The_Grid_Initialized) {
    if (Debug(DEBUG_GRID)) fprintf(stderr, "GRID: Fill because not initialized\n");
    return (FALSE);
}

```

See also sections 151, 152, and 153.

This code is used in section 149.

151. If the type of search has changed then report the grid as invalid

⟨ Tests for invalid grid 150 ⟩ +≡

```

    if (The_Grid_Search ≠ s) {
        if (Debug(DEBUG_GRID)) fprintf(stderr, "GRID: Fill because search type changed\n");
        return (FALSE);
    }

```

152. Compare the *m.m_u* value only if there are three measurements

⟨ Tests for invalid grid 150 ⟩ +≡

```

    if ((m.num_measures ≡ 3) ∧ (m.m_u ≠ MGRID.m_u)) {
        if (Debug(DEBUG_GRID)) fprintf(stderr, "GRID: Fill because unscattered light changed\n");
        return (FALSE);
    }

```

153. Make sure that the boundary conditions have not changed.

⟨ Tests for invalid grid 150 ⟩ +≡

```

    if (m.slab_index ≠ MGRID.slab_index) {
        if (Debug(DEBUG_GRID))
            fprintf(stderr, "GRID: Fill because slab refractive index changed\n");
        return (FALSE);
    }
    if (m.slab_cos_angle ≠ MGRID.slab_cos_angle) {
        if (Debug(DEBUG_GRID)) fprintf(stderr, "GRID: Fill because light angle changed\n");
        return (FALSE);
    }
    if (m.slab_top_slide_index ≠ MGRID.slab_top_slide_index) {
        if (Debug(DEBUG_GRID)) fprintf(stderr, "GRID: Fill because top slide index changed\n");
        return (FALSE);
    }
    if (m.slab_bottom_slide_index ≠ MGRID.slab_bottom_slide_index) {
        if (Debug(DEBUG_GRID)) fprintf(stderr, "GRID: Fill because bottom slide index changed\n");
        return (FALSE);
    }
    if (s ≡ FIND_AB ∧ r.slab.g ≠ RGRID.slab.g) {
        if (Debug(DEBUG_GRID)) fprintf(stderr, "GRID: Fill because anisotropy changed\n");
        return (FALSE);
    }
    if (s ≡ FIND_AG ∧ r.slab.b ≠ RGRID.slab.b) {
        if (Debug(DEBUG_GRID)) fprintf(stderr, "GRID: Fill because optical depth changed\n");
        return (FALSE);
    }
    if (s ≡ FIND_BsG ∧ r.default.ba ≠ RGRID.default.ba) {
        if (Debug(DEBUG_GRID)) fprintf(stderr, "GRID: Fill because mu_a changed\n");
        return (FALSE);
    }
    if (s ≡ FIND_BaG ∧ r.default.bs ≠ RGRID.default.bs) {
        if (Debug(DEBUG_GRID)) fprintf(stderr, "GRID: Fill because mu_s changed\n");
        return (FALSE);
    }
}

```

154. Routine to just figure out the distance to a particular a, b, g point

⟨Prototype for *abg_distance* 154⟩ ≡

```
void abg_distance(double a, double b, double g, guess_type *guess)
```

This code is used in sections 124 and 155.

155. ⟨Definition for *abg_distance* 155⟩ ≡

⟨Prototype for *abg_distance* 154⟩

```
{
    double m_r, m_t, distance;
    struct measure_type old_mm;
    struct invert_type old_rr;
    Get_Calc_State(&old_mm, &old_rr);
    RR.slab.a = a;
    RR.slab.b = b;
    RR.slab.g = g;
    Calculate_Distance(&m_r, &m_t, &distance);
    Set_Calc_State(old_mm, old_rr);
    guess-a = a;
    guess-b = b;
    guess-g = g;
    guess-distance = distance;
}
```

This code is used in section 123.

156. This just searches through the grid to find the minimum entry and returns the optical properties of that entry. The smallest, the next smallest, and the third smallest values are returned.

This has been rewritten to use *Calculate_Distance_With_Corrections* so that changes in sphere parameters won't necessitate recalculating the grid.

⟨Prototype for *Near_Grid_Points* 156⟩ ≡

```
void Near_Grid_Points(double r, double t, search_type s, int *i_min, int *j_min)
```

This code is used in sections 124 and 157.

157. $\langle$ Definition for *Near_Grid_Points* 157 $\rangle \equiv$
 $\langle$ Prototype for *Near_Grid_Points* 156 $\rangle$

```

{
    int i, j;
    double fval;
    double smallest = 10.0;
    struct measure_type old_mm;
    struct invert_type old_rr;

    (void) r;
    (void) t;
    (void) s;
    if (Debug(DEBUG_GRID)) fprintf(stderr, "GRID: Finding best grid points\n");
    Get_Calc_State(&old_mm, &old_rr);
    *i_min = 0;
    *j_min = 0;
    for (i = 0; i < GRID_SIZE; i++) {
        for (j = 0; j < GRID_SIZE; j++) {
            CALCULATING_GRID = 1;
            fval = Calculate_Grid_Distance(i, j);
            CALCULATING_GRID = 0;
            if (fval < smallest) {
                *i_min = i;
                *j_min = j;
                smallest = fval;
            }
        }
    }
    Set_Calc_State(old_mm, old_rr);
}

```

This code is used in section 123.

158. Routine to incorporate flipping of sample if needed. This is pretty simple. The assumption is that flipping is handled relative to the reflection side of the sphere. Thus even when flipping is needed, the usual call to `RT()` will result in the correct values for the reflectances. The transmission values can then be calculated by swapping the top and bottom slides.

Technically, the value of `slab` should be `const` but it is not so that we don't pay a copying overhead whenever `flip` is false (the usual case).

$\langle$ Prototype for *RT_Flip* 158 $\rangle \equiv$

```

void RT_Flip(int flip, int n, struct AD_slab_type *slab, double *UR1, double *UT1, double
             *URU, double *UTU)

```

This code is used in sections 124 and 159.

159. $\langle \text{Definition for } RT_Flip \text{ 159} \rangle \equiv$

$\langle \text{Prototype for } RT_Flip \text{ 158} \rangle$

```
{
  double correct_UR1, correct_URU;
  RT(n, slab, UR1, UT1, URU, UTU);
  if (flip) {
    correct_UR1 = *UR1;
    correct_URU = *URU;
    SWAP(slab→n_top_slide, slab→n_bottom_slide)SWAP(slab→b_top_slide, slab→b_bottom_slide)RT(n, slab, UR1,
      UT1, URU, UTU);
    SWAP(slab→n_top_slide, slab→n_bottom_slide)SWAP(slab→b_top_slide,
      slab→b_bottom_slide) * UR1 = correct_UR1;
    *URU = correct_URU;
  }
}
```

This code is used in section 123.

160. Simple routine to put values into the grid

Presumes that *RR.slab* is properly set up.

⟨Definition for *fill_grid_entry* 160⟩ ≡

```
static void fill_grid_entry(int i, int j)
{
    double ur1, ut1, uru, utu;
    if (RR.slab.b ≤ 1 · 10-6) RR.slab.b = 1 · 10-6;
    if (Debug(DEBUG_GRID_CALC) ∧ i ≡ 0 ∧ j ≡ 0) {
        fprintf(stderr, "+_i_j_");
        fprintf(stderr, "a_b_g_|");
        fprintf(stderr, "M_R_grid|");
        fprintf(stderr, "M_T_grid\n");
    }
    if (Debug(DEBUG_EVERY_CALC)) {
        if (¬CALCULATING_GRID)
            fprintf(stderr, "a=%8.5f_b=%10.5f_g=%8.5f", RR.slab.a, RR.slab.b, RR.slab.g);
        else {
            if (j ≡ 0) fprintf(stderr, ".");
            if (i + 1 ≡ GRID_SIZE ∧ j ≡ 0) fprintf(stderr, "\n");
        }
    }
    RT_Flip(MM.flip_sample, RR.method.quad_pts, &RR.slab, &ur1, &ut1, &uru, &utu);
    if (Debug(DEBUG_EVERY_CALC) ∧ ¬CALCULATING_GRID)
        fprintf(stderr, "ur1=%8.5f_ut1=%8.5f\n", ur1, ut1);
    The_Grid[GRID_SIZE * i + j][A_COLUMN] = RR.slab.a;
    The_Grid[GRID_SIZE * i + j][B_COLUMN] = RR.slab.b;
    The_Grid[GRID_SIZE * i + j][G_COLUMN] = RR.slab.g;
    The_Grid[GRID_SIZE * i + j][UR1_COLUMN] = ur1;
    The_Grid[GRID_SIZE * i + j][UT1_COLUMN] = ut1;
    The_Grid[GRID_SIZE * i + j][URU_COLUMN] = uru;
    The_Grid[GRID_SIZE * i + j][UTU_COLUMN] = utu;
    if (Debug(DEBUG_GRID_CALC)) {
        fprintf(stderr, "+_3d_3d_", i, j);
        fprintf(stderr, "%10.5f_%10.5f_%10.5f_|", RR.slab.a, RR.slab.b, RR.slab.g);
        fprintf(stderr, "%10.5f_%10.5f_|", MM.m_r, uru);
        fprintf(stderr, "%10.5f_%10.5f_\n", MM.m_t, utu);
    }
}
```

This code is used in section 123.

161. This routine fills the grid with a proper set of values. With a little work, this routine could be made much faster by (1) only generating the phase function matrix once, (2) Making only one pass through the array for each albedo value, i.e., using the matrix left over from $b = 1$ to generate the solution for $b = 2$. Unfortunately this would require a complete revision of the *Calculate_Distance* routine. Fortunately, this routine should only need to be calculated once at the beginning of each run.

⟨Prototype for *Fill_AB_Grid* 161⟩ ≡

```
void Fill_AB_Grid(struct measure_type m, struct invert_type r)
```

This code is used in sections 123 and 162.

162. $\langle$ Definition for *Fill_AB_Grid* 162 $\rangle \equiv$
 $\langle$ Prototype for *Fill_AB_Grid* 161 $\rangle$
{
 int *i, j*;
 double *min_log_b* = -8; /* exp(-10) is smallest thickness */
 double *max_log_b* = +8; /* exp(+8) is greatest thickness */
 if (*Debug*(DEBUG_GRID)) *fprintf*(*stderr*, "GRID: Filling AB grid (g=%.5f)\n", *RR.slab.g*);
 if (*The_Grid* $\equiv$ Λ) *Allocate_Grid*(*r.search*);
 $\langle$ Zero *GG* 169 $\rangle$
 Set_Calc_State(*m, r*);
 GG_g = *RR.slab.g*;
 for (*i* = 0; *i* < GRID_SIZE; *i*++) {
 double *x* = (**double**) *i* / (GRID_SIZE - 1.0);
 RR.slab.b = *exp*(*min_log_b* + (*max_log_b* - *min_log_b*) * *x*);
 for (*j* = 0; *j* < GRID_SIZE; *j*++) {
 $\langle$ Generate next albedo using *j* 164 $\rangle$
 fill_grid_entry(*i, j*);
 }
 }
 The_Grid_Initialized = TRUE;
 The_Grid_Search = FIND_AB;
}

This code is used in section 123.

163. Now it seems that we must be a bit more subtle in choosing the values of albedos to use in the grid. Spacing the points sometimes gives too coarse of spacing as $a \rightarrow 0$ or $a \rightarrow 1$. A function that seems to work is

$$a = 1 - (1 - x)^2 \cdot (1 + 2x) \quad \text{where} \quad x = \frac{j - 1}{n - 1}$$

where $j = 0, 1, \dots, n$. Thus instead of getting

$$0.0, 0.1, 0.2, \dots, 0.9, 1.0$$

we get

$$0.000, 0.028, 0.104, 0.216, 0.352, 0.500, 0.648, 0.784, 0.896, 0.972, 1.000$$

which has symmetric spacing at both ends of the range of a .

164. Here is heuristic that seems to work well

$\langle$ Generate next albedo using *j* 164 $\rangle \equiv$
{
 double *x* = (**double**) *j* / (GRID_SIZE - 1.0);
 RR.slab.a = $1.0 - (1.0 - x) * (1.0 - x) * (1.0 + 2.0 * x)$;
}

This code is used in sections 162 and 168.

165. Let's do the same thing for g grid points with the exception that points are constrained between $-\text{MAX_ABS_G}$ and $+\text{MAX_ABS_G}$

```
<Generate next anisotropy using i 165> ≡
{
  double x = (double) i / (GRID_SIZE - 1.0);
  double xx = (1.0 - x) * (1.0 - x) * (1.0 + 2.0 * x);
  RR.slabs.g = (1.0 - 2.0 * xx) * MAX_ABS_G;
}
```

This code is used in section 168.

```
166. <Generate next anisotropy using j 166> ≡
{
  double x = (double) j / (GRID_SIZE - 1.0);
  double xx = (1.0 - x) * (1.0 - x) * (1.0 + 2.0 * x);
  RR.slabs.g = (1.0 - 2.0 * xx) * MAX_ABS_G;
}
```

This code is used in sections 171, 173, and 175.

167. This is quite similar to *Fill_AB_Grid*, with the exception of the little shuffle I do at the beginning to figure out the optical thickness to use. The problem is that the optical thickness may not be known. If it is known then the only way that we could have gotten here is if the user dictated **FIND_AG** and specified b and only provided two measurements. Otherwise, the user must have made three measurements and the optical depth can be figured out from $m.m.u$.

This routine could also be improved by not recalculating the anisotropy matrix for every point. But this would only end up being a minor performance enhancement if it were fixed.

```
<Prototype for Fill_AG_Grid 167> ≡
void Fill_AG_Grid(struct measure_type m, struct invert_type r)
```

This code is used in sections 123 and 168.

168. $\langle$ Definition for *Fill_AG_Grid* 168 $\rangle \equiv$

$\langle$ Prototype for *Fill_AG_Grid* 167 $\rangle$

```
{
    int i, j;
    double max_a = -10;
    double min_a = 10;
    if (Debug(DEBUG_GRID)) fprintf(stderr, "GRID: Filling AG grid\n");
    if (The_Grid  $\equiv$   $\Lambda$ ) Allocate_Grid(r.search);
     $\langle$  Zero GG 169  $\rangle$ 
    Set_Calc_State(m, r);
    GG_b = r.slab.b;
    for (i = 0; i < GRID_SIZE; i++) {
         $\langle$  Generate next anisotropy using i 165  $\rangle$ 
        for (j = 0; j < GRID_SIZE; j++) {
             $\langle$  Generate next albedo using j 164  $\rangle$ 
            fill_grid_entry(i, j);
            if (RR.slab.a < min_a) min_a = RR.slab.a;
            if (RR.slab.a > max_a) max_a = RR.slab.a;
        }
    }
    if (Debug(DEBUG_GRID)) {
        fprintf(stderr, "GRID: a = %9.7f to %9.7f\n", min_a, max_a);
        fprintf(stderr, "GRID: b = %9.5f\n", r.slab.b);
        fprintf(stderr, "GRID: g range = %9.6f to %9.6f\n", -MAX_ABS_G, MAX_ABS_G);
    }
    The_Grid_Initialized = TRUE;
    The_Grid_Search = FIND_AG;
}
```

This code is used in section 123.

169.

$\langle$ Zero GG 169 $\rangle \equiv$

```
GG_a = 0.0;
GG_b = 0.0;
GG_g = 0.0;
GG_bs = 0.0;
GG_ba = 0.0;
```

This code is used in sections 162, 168, 171, 173, and 175.

170. This is quite similar to *Fill_AB_Grid*, with the exception of the that the albedo is held fixed while *b* and *g* are varied.

This routine could also be improved by not recalculating the anisotropy matrix for every point. But this would only end up being a minor performance enhancement if it were fixed.

$\langle$ Prototype for *Fill_BG_Grid* 170 $\rangle \equiv$

```
void Fill_BG_Grid(struct measure_type m, struct invert_type r)
```

This code is used in sections 124 and 171.

171. $\langle$ Definition for *Fill_BG_Grid* 171 $\rangle \equiv$
 $\langle$ Prototype for *Fill_BG_Grid* 170 $\rangle$
 $\{$
 int *i, j*;
 double *min_log_b* = -8; /* exp(-10) is smallest thickness */
 double *max_log_b* = +10; /* exp(+8) is greatest thickness */
 if (*The_Grid* $\equiv \Lambda$) *Allocate_Grid*(*r.search*);
 $\langle$ Zero *GG* 169 $\rangle$
 if (*Debug*(DEBUG_GRID)) *fprintf*(*stderr*, "GRID: Filling BG grid\n");
 Set_Calc_State(*m, r*);
 RR.slab.a = *RR.default_a*;
 GG_a = *RR.slab.a*;
 for (*i* = 0; *i* < GRID_SIZE; *i*++) {
 double *x* = (**double**) *i* / (GRID_SIZE - 1.0);
 RR.slab.b = *exp*(*min_log_b* + (*max_log_b* - *min_log_b*) * *x*);
 for (*j* = 0; *j* < GRID_SIZE; *j*++) {
 $\langle$ Generate next anisotropy using *j* 166 $\rangle$
 fill_grid_entry(*i, j*);
 }
 }
 if (*Debug*(DEBUG_GRID)) {
 fprintf(*stderr*, "GRID: a = %9.7f\n", *RR.default_a*);
 fprintf(*stderr*, "GRID: b range = %9.5f to %9.3f\n", *exp*(*min_log_b*), *exp*(*max_log_b*));
 fprintf(*stderr*, "GRID: g range = %9.6f to %9.6f\n", -MAX_ABS_G, MAX_ABS_G);
 }
 The_Grid_Initialized = TRUE;
 The_Grid_Search = FIND_BG;
 $\}$

This code is used in section 123.

172. This is quite similar to *Fill_BG_Grid*, with the exception of the that the $b_s = \mu_s d$ is held fixed. Here *b* and *g* are varied on the usual grid, but the albedo is forced to take whatever value is needed to ensure that the scattering remains fixed.

$\langle$ Prototype for *Fill_BaG_Grid* 172 $\rangle \equiv$
 void *Fill_BaG_Grid*(**struct** *measure_type m, struct invert_type r*)

This code is used in sections 124 and 173.

173. $\langle$ Definition for *Fill_BaG_Grid* 173 $\rangle \equiv$

$\langle$ Prototype for *Fill_BaG_Grid* 172 $\rangle$

```
{
  int i, j;
  double max_a = -10;
  double min_a = 10;
  double bs = r.default_bs;
  double min_log_ba = -8; /* exp(-10) is smallest thickness */
  double max_log_ba = +10; /* exp(+8) is greatest thickness */
  if (The_Grid  $\equiv$   $\Lambda$ ) Allocate_Grid(r.search);
   $\langle$  Zero GG 169  $\rangle$ 
  if (Debug(DEBUG_GRID)) {
    fprintf(stderr, "GRID: Filling BaG grid\n");
    fprintf(stderr, "GRID: bs = %9.5f\n", bs);
    fprintf(stderr, "GRID: ba range = %9.6f to %9.3f\n", exp(min_log_ba), exp(max_log_ba));
  }
  Set_Calc_State(m, r);
  GG_bs = bs;
  for (i = 0; i < GRID_SIZE; i++) {
    double x = (double)i / (GRID_SIZE - 1.0);
    double ba = exp(min_log_ba + (max_log_ba - min_log_ba) * x);
    RR.slab.b = ba + bs;
    if (RR.slab.b > 0) RR.slab.a = bs / RR.slab.b;
    else RR.slab.a = 0;
    if (RR.slab.a < 0) RR.slab.a = 0;
    if (RR.slab.a < min_a) min_a = RR.slab.a;
    if (RR.slab.a > max_a) max_a = RR.slab.a;
    for (j = 0; j < GRID_SIZE; j++) {
       $\langle$  Generate next anisotropy using j 166  $\rangle$ 
      fill_grid_entry(i, j);
    }
  }
  if (Debug(DEBUG_GRID)) {
    fprintf(stderr, "GRID: a range = %9.7f to %9.7f\n", min_a, max_a);
    fprintf(stderr, "GRID: b range = %9.5f to %9.3f\n", exp(min_log_ba) + bs, exp(max_log_ba) + bs);
    fprintf(stderr, "GRID: g range = %9.6f to %9.6f\n", -MAX_ABS_G, MAX_ABS_G);
  }
  The_Grid_Initialized = TRUE;
  The_Grid_Search = FIND_BaG;
}
```

This code is used in section 123.

174. Very similar to the above routine, but holding $b_a = \mu_a d$ fixed. Here b and g are varied on the usual grid, but the albedo is forced to take whatever value is needed to ensure that the absorption remains fixed.

$\langle$ Prototype for *Fill_BsG_Grid* 174 $\rangle \equiv$

void *Fill_BsG_Grid*(**struct** *measure_type* m , **struct** *invert_type* r)

This code is used in sections 124 and 175.

175. $\langle$ Definition for *Fill_BsG_Grid* 175 $\rangle \equiv$

$\langle$ Prototype for *Fill_BsG_Grid* 174 $\rangle$

```
{
    int i, j;
    double max_a = -10;
    double min_a = 10;
    double ba = r.default_ba;
    double min_log_bs = -8; /* exp(-10) is smallest thickness */
    double max_log_bs = +10; /* exp(+8) is greatest thickness */
    if (The_Grid  $\equiv$   $\Lambda$ ) Allocate_Grid(r.search);
     $\langle$  Zero GG 169  $\rangle$ 
    if (Debug(DEBUG_GRID)) {
        fprintf(stderr, "GRID: Filling BsG_grid\n");
        fprintf(stderr, "GRID: ba=%9.5f\n", ba);
        fprintf(stderr, "GRID: bs_range=%9.6f to %9.3f\n", exp(min_log_bs), exp(max_log_bs));
    }
    Set_Calc_State(m, r);
    GG_ba = RR.default_ba;
    for (i = 0; i < GRID_SIZE; i++) {
        double x = (double)i/(GRID_SIZE - 1.0);
        double bs = exp(min_log_bs + (max_log_bs - min_log_bs) * x);
        RR.slab.b = ba + bs;
        if (RR.slab.b > 0) RR.slab.a = 1 - RR.default_ba/RR.slab.b;
        else RR.slab.a = 0;
        if (RR.slab.a < 0) RR.slab.a = 0;
        if (RR.slab.a < min_a) min_a = RR.slab.a;
        if (RR.slab.a > max_a) max_a = RR.slab.a;
        for (j = 0; j < GRID_SIZE; j++) {
             $\langle$  Generate next anisotropy using j 166  $\rangle$ 
            fill_grid_entry(i, j);
        }
    }
    if (Debug(DEBUG_GRID)) {
        fprintf(stderr, "GRID: a_range=%9.7f to %9.7f\n", min_a, max_a);
        fprintf(stderr, "GRID: b_range=%9.5f to %9.3f\n", exp(min_log_bs)+ba, exp(max_log_bs)+ba);
        fprintf(stderr, "GRID: g_range=%9.6f to %9.6f\n", -MAX_ABS_G, MAX_ABS_G);
    }
    The_Grid_Initialized = TRUE;
    The_Grid_Search = FIND_BsG;
}
```

This code is used in section 123.

176. $\langle$ Prototype for *Fill_Grid* 176 $\rangle \equiv$

void *Fill_Grid*(**struct** *measure_type* m, **struct** *invert_type* r, **int** *force_new*)

This code is used in sections 124 and 177.

177. $\langle$ Definition for *Fill_Grid* 177 $\rangle \equiv$
 $\langle$ Prototype for *Fill_Grid* 176 $\rangle$
 $\{$
 if (*force_new* $\vee \neg$ *Same_Calc_State*(*m*, *r*)) $\{$
 switch (*r.search*) $\{$
 case FIND_AB: *Fill_AB_Grid*(*m*, *r*);
 break;
 case FIND_AG: *Fill_AG_Grid*(*m*, *r*);
 break;
 case FIND_BG: *Fill_BG_Grid*(*m*, *r*);
 break;
 case FIND_BaG: *Fill_BaG_Grid*(*m*, *r*);
 break;
 case FIND_BsG: *Fill_BsG_Grid*(*m*, *r*);
 break;
 default: *AD_error*("Attempt to fill grid for unknown search case.");
 $\}$
 $\}$
 Get_Calc_State(&MGRID, &RGRID);
 $\}$

This code is used in section 123.

178. Calculating R and T.

Calculate_Distance returns the distance between the measured values in MM and the calculated values for the current guess at the optical properties. It assumes that the everything in the local variables MM and RR have been set appropriately.

⟨Prototype for *Calculate_Distance* 178⟩ ≡

```
void Calculate_Distance(double *M_R, double *M_T, double *deviation)
```

This code is used in sections 124 and 179.

179. ⟨Definition for *Calculate_Distance* 179⟩ ≡

⟨Prototype for *Calculate_Distance* 178⟩

```
{
    double Ru, Tu, ur1, ut1, uru, utu;
    if (RR.slab.b ≤ 1 · 10-6) RR.slab.b = 1 · 10-6;
    RT_Flip(MM.flip_sample, RR.method.quad_pts, &RR.slab, &ur1, &ut1, &uru, &utu);
    Sp_mu_RT_Flip(MM.flip_sample, RR.slab.n_top_slide, RR.slab.n_slab, RR.slab.n_bottom_slide,
        RR.slab.b_top_slide, RR.slab.b, RR.slab.b_bottom_slide, RR.slab.cos_angle, &Ru, &Tu);
    if ((¬CALCULATING_GRID ∧ Debug(DEBUG_ITERATIONS)) ∨ (CALCULATING_GRID ∧
        Debug(DEBUG_GRID_CALC))) fprintf(stderr, "UUUUUUUU");
    Calculate_Distance_With_Corrections(ur1, ut1, Ru, Tu, uru, utu, M_R, M_T, deviation);
}
```

This code is used in section 123.

180. ⟨Prototype for *Calculate_Grid_Distance* 180⟩ ≡

```
double Calculate_Grid_Distance(int i, int j)
```

This code is used in sections 124 and 181.

181. $\langle$ Definition for *Calculate_Grid_Distance* 181 $\rangle \equiv$
 $\langle$ Prototype for *Calculate_Grid_Distance* 180 $\rangle$
{
 double *ur1, ut1, uru, utu, Ru, Tu, b, dev, LR, LT*;
 if (*Debug*(DEBUG_GRID_CALC) $\wedge$ $i \equiv 0 \wedge j \equiv 0$) {
 fprintf(stderr, "+i+j\n");
 fprintf(stderr, "a b g\n");
 fprintf(stderr, "M_R grid\n");
 fprintf(stderr, "M_T grid distance\n");
 }
 if (*Debug*(DEBUG_GRID_CALC)) *fprintf(stderr, "g%3d%3d\n", i, j)*;
 b = The_Grid[GRID_SIZE * $i + j$][B_COLUMN];
 ur1 = The_Grid[GRID_SIZE * $i + j$][UR1_COLUMN];
 ut1 = The_Grid[GRID_SIZE * $i + j$][UT1_COLUMN];
 uru = The_Grid[GRID_SIZE * $i + j$][URU_COLUMN];
 utu = The_Grid[GRID_SIZE * $i + j$][UTU_COLUMN];
 RR.slab.a = The_Grid[GRID_SIZE * $i + j$][A_COLUMN];
 RR.slab.b = The_Grid[GRID_SIZE * $i + j$][B_COLUMN];
 RR.slab.g = The_Grid[GRID_SIZE * $i + j$][G_COLUMN];
 Sp_mu_RT_Flip(*MM.flip_sample*, *RR.slab.n_top_slide*, *RR.slab.n_slab*, *RR.slab.n_bottom_slide*,
 RR.slab.b_top_slide, *b*, *RR.slab.b_bottom_slide*, *RR.slab.cos_angle*, &*Ru*, &*Tu*);
 CALCULATING_GRID = 1;
 Calculate_Distance_With_Corrections(*ur1, ut1, Ru, Tu, uru, utu, &LR, <, &dev*);
 CALCULATING_GRID = 0;
 return dev;
}

This code is used in section 123.

182. This is the routine that actually finds the distance. I have factored this part out so that it can be used in the *Near_Grid_Points* routine.

Ru and *Tu* refer to the unscattered (collimated) reflection and transmission.

The only tricky part is to remember that the we are trying to match the measured values. The measured values are affected by sphere parameters and light loss. Since the values UR1 and UT1 are for an infinite slab sample with no light loss, the light loss out the edges must be subtracted. It is these values that are used with the sphere formulas to convert the modified UR1 and UT1 to values for *M_R and *M_T.

$\langle$ Prototype for *Calculate_Distance_With_Corrections* 182 $\rangle \equiv$
void *Calculate_Distance_With_Corrections*(**double** *UR1, double UT1, double Ru, double Tu, double*
 *URU, double UTU, double *M_R, double *M_T, double *dev*)

This code is used in sections 124 and 183.

183. $\langle$ Definition for *Calculate_Distance_With_Corrections* 183 $\rangle \equiv$
 $\langle$ Prototype for *Calculate_Distance_With_Corrections* 182 $\rangle$
{
 $\langle$ Determine calculated light to be used 184 $\rangle$
switch (MM.num_spheres) {
case 0: $\langle$ Calc M_R and M_T for no spheres 186 $\rangle$
break;
case 1:
if (MM.method $\equiv$ COMPARISON) {
 $\langle$ Calc M_R and M_T for dual beam sphere 200 $\rangle$
}
else {
 $\langle$ Calc M_R and M_T for single beam sphere 193 $\rangle$
}
break;
case 2: $\langle$ Calc M_R and M_T for two spheres 202 $\rangle$
break;
default: *fprintf*(stderr, "Bad_number_of_spheres=%d\n", MM.num_spheres);
exit(EXIT_FAILURE);
}
 $\langle$ Calculate the deviation 203 $\rangle$
 $\langle$ Print diagnostics 206 $\rangle$
}

This code is used in section 123.

184. The calculated values for M_R and M_T must be adapted to match the measurements. The diffuse light URU and UTU are used to determine the gain from the sphere. They're only modified by the lost light calculation. All values can become slightly negative because the Monte Carlo is noisy. Negative values are set to zero.

$\langle$ Determine calculated light to be used 184 $\rangle \equiv$
double UR1_calc, UT1_calc, URU_calc, UTU_calc;
URU_calc = URU - MM.uru_lost;
if (URU_calc < 0) URU_calc = 0;
UTU_calc = UTU - MM.utu_lost;
if (UTU_calc < 0) UTU_calc = 0;

See also section 185.

This code is used in section 183.

185. The measurements for UR1 and UT1 need to be modified to accommodate light that misses the detector either because it is intentionally not collected (unscattered light) or it leaks out (lost light). Since none of the light that leaks out could be unscattered light, these two are independent of one another.

$\langle$ Determine calculated light to be used 184 $\rangle + \equiv$
UR1_calc = UR1 - (1.0 - MM.fraction_of_ru_in_mr) * Ru - MM.ur1_lost;
if (UR1_calc < 0) UR1_calc = 0;
UT1_calc = UT1 - (1.0 - MM.fraction_of_tu_in_mt) * Tu - MM.ut1_lost;
if (UT1_calc < 0) UT1_calc = 0;

186. When no spheres are used, then no corrections can or need to be made. The lost light estimates in `MM.ur1_lost` and `MM.ut1_lost` should be zero and so the values for `UR1_calc` and `UT1_calc` properly account for the presence or absence of unscattered light.

```
< Calc M_R and M_T for no spheres 186 > =
{
  *M_R = UR1_calc;
  *M_T = UT1_calc;
}
```

This code is used in section 183.

187. Reflectance measurement for one sphere.

The total reflection from a slab may be broken down into light that has and has not been scattered. The total reflectance for normal incidence on an infinite slab is denoted by van de Hulst by `UR1`. To track integrating sphere corrections, this is separated into scattered and unscattered parts.

$$\text{UR1} = R_{\text{unscat}} + R_{\text{scat}}$$

`UR1` is calculated from the current guess of optical properties, but we can also calculate R_{unscat} by setting $\mu_s = 0$ which means that the albedo is zero. Assuming the incident power is P_i then the scattered light in the sphere P_{ss} and the unscattered light in the sphere P_{su} start as

$$P_{ss} = (\text{UR1} - R_{\text{unscat}})P_i \quad \text{and} \quad P_{su} = R_{\text{unscat}}P_i$$

188. Light that is lost.

In an experiment, the scattered light in the sphere will be reduced by any light that leaks out. This is determined by doing a Monte Carlo simulation to determine `UR1_lost` or how much light is scattered within the sample and escapes outside the sphere. Since the only way that light may be lost is through scattering, this does not affect the unscattered fraction.

The fraction of unscattered light collected by the sphere f_{unscat} is determined by the experimentalist. The unscattered reflected light can be aligned so that it bounces off the sample and exits completely through the entrance port so that $f_{\text{unscat}} = 0$. Alternatively, the beam may be incident on the sample at an angle so that the unscattered light will bounce and remain completely within the sphere so that $f_{\text{unscat}} = 1$.

$$P_{ss} = (\text{UR1} - \text{UR1}_{\text{unscat}} - R_{\text{lost}})P_i \quad \text{and} \quad P_{su} = f_{\text{unscat}}R_{\text{unscat}}P_i$$

189. Incident light that misses the sample.

In an experiment, a fraction f_{miss} of the incident beam might miss the sample and hit the sphere wall instead. In this case, both R_{ss} and R_{su} are affected. Typically the beam is much smaller than the sample and $f_{\text{miss}} = 0$, however sometimes the beam diverges too much and some of the beam hits the wall. After hitting the sphere wall then $f_{\text{miss}}r_w$ will be added to the scattered light in the sphere. However, now only $(1 - f_{\text{miss}})$ hits the sample directly, both the scattered and unscattered light in the sphere must be adjusted accordingly. So in the unusual case of non-zero f_{miss} , we have

$$R_{ss} = (1 - f_{\text{miss}})(\text{UR1} - R_{\text{unscat}} - \text{UR1}_{\text{lost}})P_i + f_{\text{miss}}r_wP_i \quad \text{and} \quad R_{su} = (1 - f_{\text{miss}})f_{\text{unscat}}R_{\text{unscat}}P_i$$

190. Reflectance with baffle.

When a baffle blocks light from passing directly between the sample to the detector then the light reflected by the sample must bounce once. Some of the light will be reflected by the sphere walls, but some may be reflected by the third port. The weighted reflectance of the first bounce is

$$r_{\text{first}} = (1 - a_t)r_w + a_t r_t$$

We can safely assume that the fraction of light generated by f_{miss} will originate close enough to the sample that a baffle, if present, will prevent light from directly reaching the detector. The scattered light in the sphere after the first bounce will be

$$P_{ss} = r_{\text{first}} [(1 - f_{\text{miss}})(\text{UR1} - R_{\text{unscat}} - R_{\text{lost}}) + f_{\text{miss}}r_w] P_i$$

All unscattered reflectance that is collected must hit the sphere wall (otherwise it would exit through the entrance port and not be collected!). This means that the correction in r_{first} for the entrance port is not needed and the unreflected light can just be multiplied by r_w

$$P_{su} = r_w(1 - f_{\text{miss}})f_{\text{unscat}}R_{\text{unscat}}P_i$$

The last step is to account for the sphere gain. The sample is held in the sample port and the entrance port is empty. The total reflection for diffuse illumination of the sample is URU . This quantity must also be corrected for light that is not collected by the sphere UR1_{lost} . The gain for such a sphere is $G(\text{URU} - \text{URU}_{\text{lost}}, 0)$. Finally,

$$P_s = [a_d(1 - r_d)] \cdot G(\text{URU} - \text{URU}_{\text{lost}}, 0) [P_{ss} + P_{su}]$$

191. The reflection standard.

We let $\text{UR1} = \text{URU} = r_{\text{std}}$, $R_{\text{unscat}} = 0$, $R_{\text{lost}} = 0$ to get

$$P_{\text{std}} = [a_d(1 - r_d)] \cdot r_{\text{first}} [(1 - f_{\text{miss}})r_{\text{std}} + f_{\text{miss}}r_w] G(r_{\text{std}}, 0) P_i$$

192. The open port.

We let $\text{UR1} = \text{URU} = 0$, $R_{\text{unscat}} = 0$, $R_{\text{lost}} = 0$ to get

$$P_0 = [a_d(1 - r_d)] \cdot r_{\text{first}} f_{\text{miss}} r_w G(0, 0) P_i$$

193. The un baffled reflectance sphere.

In this case, light can reach the detector from the sample. The first bounce is not needed which can be accommodated by letting $r_{\text{first}} = 1$. We, of course, assume that the gain is calculated assuming that no baffle is present. The incident power P_i and the quantities in square brackets are identical for P_s , P_{std} , and P_0 and cancel in the normalized reflection fraction

$$M_R = r_{\text{std}} \cdot \frac{P_s - P_0}{P_{\text{std}} - P_0}$$

In addition, the entrance port is empty and therefore $r_t = 0$ and can be omitted from the r_{first} calculation. This leads to the following code for M_R

```

< Calc M_R and M_T for single beam sphere 193 > ≡
  double P_std, P, P_0, G, G_0, G_std, r_first, P_ss, P_su;
  r_first = 1;
  if (MM.baffle_r) r_first = MM.rw_r * (1 - MM.at_r);
  UR1_calc = UR1 - Ru - MM.ur1_lost;
  if (UR1_calc < 0) UR1_calc = 0;
  G_0 = Gain(REFLECTION_SPHERE, MM, 0.0, 0.0);
  G = Gain(REFLECTION_SPHERE, MM, URU_calc, 0.0);
  G_std = Gain(REFLECTION_SPHERE, MM, MM.rstd_r, 0.0);
  P_std = G_std * (MM.rstd_r * (1 - MM.f_r) + MM.f_r * MM.rw_r);
  P_0 = G_0 * (MM.f_r * MM.rw_r);
  P_ss = r_first * (UR1_calc * (1 - MM.f_r) + MM.f_r * MM.rw_r);
  P_su = MM.rw_r * (1 - MM.f_r) * MM.fraction_of_ru_in_mr * Ru;
  P = G * (P_ss + P_su);
  *M_R = MM.rstd_r * (P - P_0) / (P_std - P_0);
  if (Debug(DEBUG_SPHERE_GAIN) ∧ ¬CALCULATING_GRID) {
    fprintf(stderr, "SPHERE: REFLECTION\n");
    fprintf(stderr, "SPHERE: G0=%7.3f G=%7.3f G_std=%7.3f\n", G_0, G, G_std);
    fprintf(stderr, "SPHERE: P0=%7.3f P=%7.3f P_std=%7.3f\n", P_0, P, P_std);
    fprintf(stderr, "SPHERE: UR1=%7.3f UR1_calc=%7.3f M_R=%7.3f\n", UR1, UR1_calc,
      *M_R);
  }

```

See also section 198.

This code is used in section 183.

194. Transmittance measurement for one sphere.

Like in the reflection case, the total transmission can be split into unscattered transmission and scattered transmission,

$$UT1 = T_u + T_s$$

We define P_{ss} as the scattered light in the sphere and P_{su} as the unscattered light in the sphere for an incident power P_i

$$P_{ss} = (UT1 - T_u)P_i \quad \text{and} \quad P_{su} = f_{\text{unscat}}T_uP_i$$

195. Transmitted light not collected.

The scattered light will be reduced by $\text{UT}1_{\text{lost}}$. The unscattered light will be affected by the fraction of unscattered light collected by the sphere. When transmission measurements are made, the third port (opposite the sample port in the sphere) is typically filled with a known standard. However, the third port might also be left open so that all the scattered light might exit and only scattered light is collected. In the former case $f_{\text{unscat}} = 1$ and in the latter case $f_{\text{unscat}} = 0$. So including these effects gives

$$P_{ss} = (\text{UT}1 - T_u - T_{\text{lost}})P_i \quad \text{and} \quad P_{su} = f_{\text{unscat}}T_u$$

196. The baffling case of transmission.

With a baffle, then scattered light from the sample cannot reach the detector without a bounce. This weighted reflection is given by

$$r_{\text{first}} = (1 - a_t)r_w + a_tr_t$$

The unscattered light will be reflected by the standard r_t in the third port and so

$$P_{ss} = r_{\text{first}}(\text{UT}1 - T_u - T_{\text{lost}})P_i \quad \text{and} \quad P_{su} = r_tf_{\text{unscat}}T_u$$

The last step is to account for the sphere gain. The sample is held in the sample port and the third port reflects r_t . The total reflection for diffuse illumination of the sample is URU . This quantity must also be corrected for light that is not collected by the sphere URU_{lost} . The gain for such a sphere is $G(\text{URU} - \text{URU}_{\text{lost}}, r_t)$. Finally,

$$P_s = G(\text{URU} - \text{URU}_{\text{lost}}, r_t)(P_{ss} + P_{su})$$

197. No baffle.

Here $r_{\text{first}} = 1$ and the gain should be calculated assuming no baffle.

198. The standard measurement.

When transmission measurements are made, typically the third port (the one that let the light into the sphere for the reflection measurement) is filled with a known standard. In this case, the natural way to make the 100% transmission measurement is to shine the beam through the empty sample port onto the known standard.

We let $UT1 = T_u = 1$, $T_{lost} = 0$, $URU = 0$, $r_t = r_{std}$, and $f_{unscat} = 1$ so

$$P_{std} = G(0, r_{std}) r_{std} P_i$$

The estimate for the measured transmittance is

$$M_T = r_{std} \frac{P_s}{P_{std}} = \frac{P_{ss} + P_{su}}{P_i} \cdot \frac{G(URU - URU_{lost}, r_{third})}{G(0, r_{std})}$$

⟨ Calc M_R and M_T for single beam sphere 193 ⟩ +≡

```
{
  double r_first = 1;
  double r_third = MM.rstd_t;
  if (MM.fraction_of_tu_in_mt ≡ 0) r_third = 0;
  if (MM.baffle_t) r_first = MM.rw_t * (1 - MM.at_t) + MM.rstd_t * MM.at_t;
  UT1_calc = UT1 - Tu - MM.ut1_lost;
  if (UT1_calc < 0) UT1_calc = 0;
  G = Gain(TRANSMISSION_SPHERE, MM, URU_calc, r_third);
  G_std = Gain(TRANSMISSION_SPHERE, MM, 0, MM.rstd_t);
  *M_T = (r_third * Tu * MM.fraction_of_tu_in_mt + r_first * UT1_calc) * G / G_std;
  if (Debug(DEBUG_SPHERE_GAIN) ∧ ¬CALCULATING_GRID) {
    fprintf(stderr, "SPHERE: TRANSMISSION\n");
    fprintf(stderr, "SPHERE: G=%7.3f G_std=%7.3f\n", G, G_std);
    fprintf(stderr, "SPHERE: UT1=%7.3f UT1calc=%7.3f T_c=%7.3f\n", UT1, UT1_calc, Tu);
    fprintf(stderr, "SPHERE: M_T=%7.3f\n", *M_T);
    fprintf(stderr, "\n");
  }
}
```

199. Dual beam case for one sphere.

200. The dual beam case is different because the sphere efficiency is equivalent for measurement of light hitting the sample first or hitting the reference standard first. The dual beam measurement should report the ratio of these two reflectance measurements, thereby eliminating the need to calculate the sphere gain.

The only correction that needs to be made have already been made, namely subtracting the UR1 or UT1 lost light and also accounting for whether or not unscattered light is collected.

Originally, I had a bunch of calculations trying to account for light that hits the sphere wall first. Since the exact details of how a dual beam spectrometer reports its measurements is unknown, it makes no sense to try and account for it.

⟨ Calc M_R and M_T for dual beam sphere 200 ⟩ ≡

```
{
  *M_R = UR1_calc;
  *M_T = UT1_calc;
}
```

This code is used in section 183.

201. Double integrating spheres.

202. When two integrating spheres are present then the double integrating sphere formulas are slightly more complicated.

The normalized sphere measurements for two spheres are

$$M_R = \frac{R(r_s^{\text{direct}}, r_s, t_s^{\text{direct}}, t_s) - R(0, 0, 0, 0)}{R(r_{\text{std}}, r_{\text{std}}, 0, 0) - R(0, 0, 0, 0)}$$

and

$$M_T = \frac{T(r_s^{\text{direct}}, r_s, t_s^{\text{direct}}, t_s) - T(0, 0, 0, 0)}{T(0, 0, 1, 1) - T(0, 0, 0, 0)}$$

Note that R_0 and T_0 will be zero unless one has explicitly set the fraction $m.f_r$ or $m.f_t$ to be non-zero.

⟨ Calc M_R and M_T for two spheres 202 ⟩ ≡

```
{
  double R_0, T_0;
  R_0 = TwoSphere_R(MM, 0, 0, 0, 0);
  T_0 = TwoSphere_T(MM, 0, 0, 0, 0);
  *M_R = MM.rstd_r * (TwoSphere_R(MM, UR1_calc, URU_calc, UT1_calc,
    UTU_calc) - R_0) / (TwoSphere_R(MM, MM.rstd_r, MM.rstd_r, 0, 0) - R_0);
  *M_T = (TwoSphere_T(MM, UR1_calc, URU_calc, UT1_calc, UTU_calc) - T_0) / (TwoSphere_T(MM, 0, 0,
    1, 1) - T_0);
}
```

This code is used in section 183.

203. There are at least three things that need to be considered here. First, the number of measurements. Second, is the metric is relative or absolute. And third, is the albedo fixed at zero which means that the transmission measurement should be used instead of the reflection measurement.

⟨ Calculate the deviation 203 ⟩ ≡

```
if (RR.search ≡ FIND_A ∨ RR.search ≡ FIND_G ∨ RR.search ≡ FIND_B ∨ RR.search ≡ FIND_Bs ∨ RR.search ≡
  FIND_Ba) {
  ⟨ One parameter deviation 204 ⟩
}
else {
  ⟨ Two parameter deviation 205 ⟩
}
```

This code is used in section 183.

204. This part was slightly tricky. The crux of the problem was to decide if the transmission or the reflection was trustworthy. After looking a bunches of measurements, I decided that the transmission measurement was almost always more reliable. So when there is just a single measurement known, then use the total transmission if it exists.

⟨ One parameter deviation 204 ⟩ ≡

```
if (MM.m_t > 0) {
  if (RR.metric ≡ RELATIVE) *dev = fabs(MM.m_t - *M_T) / (MM.m_t + ABIT);
  else *dev = fabs(MM.m_t - *M_T);
}
else {
  if (RR.metric ≡ RELATIVE) *dev = fabs(MM.m_r - *M_R) / (MM.m_r + ABIT);
  else *dev = fabs(MM.m_r - *M_R);
}
```

This code is used in section 203.

205. This stuff happens when we are doing two parameter searches. In these cases there should be information in both R and T. The distance should be calculated using the deviation from both. The albedo stuff might be able to be take out. We'll see.

```

⟨Two parameter deviation 205⟩ ≡
  if (RR.metric ≡ RELATIVE) {
    if (MM.m_t > ABIT) *dev = T_TRUST_FACTOR * fabs(MM.m_t - *M_T)/(UTU_calc + ABIT);
    if (RR.default_a ≠ 0) {
      *dev += fabs(MM.m_r - *M_R)/(URU_calc + ABIT);
    }
  }
  else {
    *dev = T_TRUST_FACTOR * fabs(MM.m_t - *M_T);
    if (RR.default_a ≠ 0) *dev += fabs(MM.m_r - *M_R);
  }

```

This code is used in section 203.

206. This is here so that I can figure out why the program is not converging. This is a little convoluted so that the global constants at the top of this file interact properly.

```

⟨Print diagnostics 206⟩ ≡
  if ((Debug(DEBUG_ITERATIONS) ∧ ¬CALCULATING_GRID) ∨
      (Debug(DEBUG_GRID_CALC) ∧ CALCULATING_GRID)) {
    fprintf(stderr, "%10.5f□%10.4f□%10.5f□| ", RR.slab.a, RR.slab.b, RR.slab.g);
    fprintf(stderr, "□%10.5f□%10.5f□| ", MM.m_r, *M_R);
    fprintf(stderr, "□%10.5f□%10.5f□| ", MM.m_t, *M_T);
    fprintf(stderr, "%10.3f\n", *dev);
  }

```

This code is used in section 183.

207. ⟨Prototype for *Find_AG_fn* 207⟩ ≡
double *Find_AG_fn*(**double** x[])

This code is used in sections 124 and 208.

208. ⟨Definition for *Find_AG_fn* 208⟩ ≡
 ⟨Prototype for *Find_AG_fn* 207⟩
 {
double m_r, m_t, deviation;
 RR.slab.a = *acalc2a*(x[1]);
 RR.slab.g = *gcalc2g*(x[2]);
Calculate_Distance(&m_r, &m_t, &deviation);
return deviation;
 }

This code is used in section 123.

209. ⟨Prototype for *Find_AB_fn* 209⟩ ≡
double *Find_AB_fn*(**double** x[])

This code is used in sections 124 and 210.

210. $\langle$ Definition for *Find_AB_fn* 210 $\rangle \equiv$
 $\langle$ Prototype for *Find_AB_fn* 209 $\rangle$
 {
 double *m_r*, *m_t*, *deviation*;
 RR.slab.a = *acalc2a*(*x*[1]);
 RR.slab.b = *bcalc2b*(*x*[2]);
 Calculate_Distance(&*m_r*, &*m_t*, &*deviation*);
 return *deviation*;
 }

This code is used in section 123.

211. $\langle$ Prototype for *Find_Ba_fn* 211 $\rangle \equiv$
double *Find_Ba_fn*(**double** *x*)

This code is used in sections 124 and 212.

212. This is tricky only because the value in RR.slab.b is used to hold the value of *bs* or $d \cdot \mu_s$. It must be switched to the correct value for the optical thickness and then switched back at the end of the routine.

$\langle$ Definition for *Find_Ba_fn* 212 $\rangle \equiv$
 $\langle$ Prototype for *Find_Ba_fn* 211 $\rangle$
 {
 double *m_r*, *m_t*, *deviation*, *ba*, *bs*;
 bs = RR.slab.b;
 ba = *bcalc2b*(*x*);
 RR.slab.b = *ba* + *bs*; /* unswindle */
 RR.slab.a = *bs* / (*ba* + *bs*);
 Calculate_Distance(&*m_r*, &*m_t*, &*deviation*);
 RR.slab.b = *bs*; /* swindle */
 return *deviation*;
 }

This code is used in section 123.

213. See the comments for the *Find_Ba_fn* routine above. Play the same trick but use *ba*.

$\langle$ Prototype for *Find_Bs_fn* 213 $\rangle \equiv$
double *Find_Bs_fn*(**double** *x*)

This code is used in sections 124 and 214.

214. $\langle$ Definition for *Find_Bs_fn* 214 $\rangle \equiv$
 $\langle$ Prototype for *Find_Bs_fn* 213 $\rangle$
 {
 double *m_r*, *m_t*, *deviation*, *ba*, *bs*;
 ba = RR.slab.b; /* unswindle */
 bs = *bcalc2b*(*x*);
 RR.slab.b = *ba* + *bs*;
 RR.slab.a = *bs* / (*ba* + *bs*);
 Calculate_Distance(&*m_r*, &*m_t*, &*deviation*);
 RR.slab.b = *ba*; /* swindle */
 return *deviation*;
 }

This code is used in section 123.

215. $\langle$ Prototype for *Find_A_fn* 215 $\rangle \equiv$
double *Find_A_fn*(**double** *x*)

This code is used in sections 124 and 216.

216. $\langle$ Definition for *Find_A_fn* 216 $\rangle \equiv$
 $\langle$ Prototype for *Find_A_fn* 215 $\rangle$
 {
 double *m_r*, *m_t*, *deviation*;
 RR.slab.a = *acalc2a*(*x*);
 Calculate_Distance(&*m_r*, &*m_t*, &*deviation*);
 return *deviation*;
 }

This code is used in section 123.

217. $\langle$ Prototype for *Find_B_fn* 217 $\rangle \equiv$
double *Find_B_fn*(**double** *x*)

This code is used in sections 124 and 218.

218. $\langle$ Definition for *Find_B_fn* 218 $\rangle \equiv$
 $\langle$ Prototype for *Find_B_fn* 217 $\rangle$
 {
 double *m_r*, *m_t*, *deviation*;
 RR.slab.b = *bcalc2b*(*x*);
 Calculate_Distance(&*m_r*, &*m_t*, &*deviation*);
 return *deviation*;
 }

This code is used in section 123.

219. $\langle$ Prototype for *Find_G_fn* 219 $\rangle \equiv$
double *Find_G_fn*(**double** *x*)

This code is used in sections 124 and 220.

220. $\langle$ Definition for *Find_G_fn* 220 $\rangle \equiv$
 $\langle$ Prototype for *Find_G_fn* 219 $\rangle$
 {
 double *m_r*, *m_t*, *deviation*;
 RR.slab.g = *gcalc2g*(*x*);
 Calculate_Distance(&*m_r*, &*m_t*, &*deviation*);
 return *deviation*;
 }

This code is used in section 123.

221. $\langle$ Prototype for *Find_BG_fn* 221 $\rangle \equiv$
double *Find_BG_fn*(**double** *x*[])

This code is used in sections 124 and 222.

222. $\langle \text{Definition for } Find_BG_fn \text{ 222} \rangle \equiv$
 $\langle \text{Prototype for } Find_BG_fn \text{ 221} \rangle$

```

{
    double m_r, m_t, deviation;
    RR.slab.b = bcalc2b(x[1]);
    RR.slab.g = gcalc2g(x[2]);
    RR.slab.a = RR.default_a;
    Calculate_Distance(&m_r, &m_t, &deviation);
    return deviation;
}

```

This code is used in section 123.

223. For this function the first term $x[1]$ will contain the value of $\mu_s d$, the second term will contain the anisotropy. Of course the first term is in the bizarre calculation space and needs to be translated back into normal terms before use. We just at the scattering back on and voilà we have a useable value for the optical depth.

$\langle \text{Prototype for } Find_BaG_fn \text{ 223} \rangle \equiv$

```

double Find_BaG_fn(double x[])

```

This code is used in sections 124 and 224.

224. $\langle \text{Definition for } Find_BaG_fn \text{ 224} \rangle \equiv$
 $\langle \text{Prototype for } Find_BaG_fn \text{ 223} \rangle$

```

{
    double m_r, m_t, deviation;
    RR.slab.b = bcalc2b(x[1]) + RR.default_bs;
    if (RR.slab.b ≤ 0) RR.slab.a = 0;
    else RR.slab.a = RR.default_bs/RR.slab.b;
    RR.slab.g = gcalc2g(x[2]);
    Calculate_Distance(&m_r, &m_t, &deviation);
    return deviation;
}

```

This code is used in section 123.

225. $\langle \text{Prototype for } Find_BsG_fn \text{ 225} \rangle \equiv$

```

double Find_BsG_fn(double x[])

```

This code is used in sections 124 and 226.

226. $\langle \text{Definition for } Find_BsG_fn \text{ 226} \rangle \equiv$
 $\langle \text{Prototype for } Find_BsG_fn \text{ 225} \rangle$

```

{
    double m_r, m_t, deviation;
    RR.slab.b = bcalc2b(x[1]) + RR.default_ba;
    if (RR.slab.b ≤ 0) RR.slab.a = 0;
    else RR.slab.a = 1.0 - RR.default_ba/RR.slab.b;
    RR.slab.g = gcalc2g(x[2]);
    Calculate_Distance(&m_r, &m_t, &deviation);
    return deviation;
}

```

This code is used in section 123.

227. Routine to figure out if the light loss exceeds what is physically possible. Returns the discrepancy between the current values and the maximum possible values for the the measurements m_r and m_t .

⟨Prototype for *maxloss* 227⟩ ≡
double *maxloss*(**double** *f*)

This code is used in sections 124 and 228.

228. ⟨Definition for *maxloss* 228⟩ ≡
 ⟨Prototype for *maxloss* 227⟩
 {
 struct **measure_type** *m_old*;
 struct **invert_type** *r_old*;
 double *m_r*, *m_t*, *deviation*;
 Get_Calc_State(&*m_old*, &*r_old*);
 RR.slabs.a = 1.0;
 MM.ur1_lost *= *f*;
 MM.ut1_lost *= *f*;
 Calculate_Distance(&*m_r*, &*m_t*, &*deviation*);
 Set_Calc_State(*m_old*, *r_old*);
 deviation = ((*MM.m_r* + *MM.m_t*) - (*m_r* + *m_t*));
 return *deviation*;
 }

This code is used in section 123.

229. This checks the two light loss values *ur1_loss* and *ut1_loss* to see if they exceed what is physically possible. If they do, then these values are replaced by a couple that are the maximum possible for the current values in m and r .

⟨Prototype for *Max_Light_Loss* 229⟩ ≡
void *Max_Light_Loss*(**struct** **measure_type** *m*, **struct** **invert_type** *r*, **double** **ur1_loss*, **double** **ut1_loss*)

This code is used in sections 124 and 230.

230. $\langle$ Definition for *Max_Light_Loss* 230 $\rangle \equiv$

$\langle$ Prototype for *Max_Light_Loss* 229 $\rangle$

```
{
    struct measure_type m_old;
    struct invert_type r_old;
    *ur1_loss = m.ur1_lost;
    *ut1_loss = m.ut1_lost;
    if (Debug(DEBUG_LOST_LIGHT))
        fprintf(stderr, "\nlost before ur1=%7.5f, ut1=%7.5f\n", *ur1_loss, *ut1_loss);
    Get_Calc_State(&m_old, &r_old);
    Set_Calc_State(m, r);
    if (maxloss(1.0) * maxloss(0.0) < 0) {
        double frac;
        frac = zbrent(maxloss, 0.00, 1.0, 0.001);
        *ur1_loss = m.ur1_lost * frac;
        *ut1_loss = m.ut1_lost * frac;
    }
    Set_Calc_State(m_old, r_old);
    if (Debug(DEBUG_LOST_LIGHT))
        fprintf(stderr, "lost after ur1=%7.5f, ut1=%7.5f\n", *ur1_loss, *ut1_loss);
}
```

This code is used in section 123.

231. this is currently unused

⟨ Unused diffusion fragment 231 ⟩ ≡

```

typedef struct {
    double f;
    double aprime;
    double bprime;
    double gprime;
    double boundary_method;
    double n_top;
    double n_bottom;
    double slide_top;
    double slide_bottom;
    double F0;
    double depth;
    double Exact_coll_flag;
} slabtype;

static void DE_RT(int nfluxes, AD_slab_type slab, double *UR1, double *UT1, double *URU, double
    *UTU)
{
    slabtype s;
    double rp, tp, rs, ts;
    s.f = slab.g * slab.g;
    s.gprime = slab.g / (1 + slab.g);
    s.aprime = (1 - s.f) * slab.a / (1 - slab.a * s.f);
    s.bprime = (1 - slab.a * s.f) * slab.b;
    s.boundary_method = Egan;
    s.n_top = slab.n_slab;
    s.n_bottom = slab.n_slab;
    s.slide_top = slab.n_top_slide;
    s.slide_bottom = slab.n_bottom_slide;
    s.F0 = 1/M_PI;
    s.depth = 0.0;
    s.Exact_coll_flag = FALSE;
    if (MM.illumination ≡ collimated) {
        compute_R_and_T(&s, 1.0, &rp, &rs, &tp, &ts);
        *UR1 = rp + rs;
        *UT1 = tp + ts;
        *URU = 0.0;
        *UTU = 0.0;
        return;
    }
    quad_Dif_Calc_R_and_T(&s, &rp, &rs, &tp, &ts);
    *URU = rp + rs;
    *UTU = tp + ts;
    *UR1 = 0.0;
    *UT1 = 0.0;
}

```

232. IAD Find. March 1995. Incorporated the *quick_guess* algorithm for low albedos.

```

<iad_find.c 232> ≡
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "ad_globl.h"
#include "nr_util.h"
#include "nr_mnbrk.h"
#include "nr_brent.h"
#include "nr_amoeb.h"
#include "iad_type.h"
#include "iad_util.h"
#include "iad_calc.h"
#include "iad_find.h"
#define NUMBER_OF_GUESSES 10
guess_type guess[NUMBER_OF_GUESSES];
int compare_guesses(const void *p1, const void *p2)
{
    guess_type *g1 = (guess_type *) p1;
    guess_type *g2 = (guess_type *) p2;
    if (g1->distance < g2->distance) return -1;
    else if (g1->distance == g2->distance) return 0;
    else return 1;
}
<Definition for U_Find_Ba 246>
<Definition for U_Find_Bs 244>
<Definition for U_Find_A 248>
<Definition for U_Find_B 252>
<Definition for U_Find_G 250>
<Definition for U_Find_AG 255>
<Definition for U_Find_AB 235>
<Definition for U_Find_BG 260>
<Definition for U_Find_BaG 266>
<Definition for U_Find_BsG 271>

```

233. All the information that needs to be written to the header file `iad_find.h`. This eliminates the need to maintain a set of header files as well.

```

<iad_find.h 233> ≡
<Prototype for U_Find_Ba 245>;
<Prototype for U_Find_Bs 243>;
<Prototype for U_Find_A 247>;
<Prototype for U_Find_B 251>;
<Prototype for U_Find_G 249>;
<Prototype for U_Find_AG 254>;
<Prototype for U_Find_AB 234>;
<Prototype for U_Find_BG 259>;
<Prototype for U_Find_BaG 265>;
<Prototype for U_Find_BsG 270>;

```

234. Fixed Anisotropy.

This is the most common case.

⟨Prototype for *U_Find_AB* 234⟩ ≡

```
void U_Find_AB(struct measure_type m, struct invert_type *r)
```

This code is used in sections 233 and 235.

235. ⟨Definition for *U_Find_AB* 235⟩ ≡

⟨Prototype for *U_Find_AB* 234⟩

```
{
  ⟨Allocate local simplex variables 236⟩
  if (Debug(DEBUG_SEARCH)) {
    fprintf(stderr, "SEARCH: Using U_Find_AB()");
    fprintf(stderr, " mu=%4.2f, g=", r→slab.cos_angle);
    if (r→default_g ≠ UNINITIALIZED) fprintf(stderr, " %7.3f (constrained g)", r→default_g);
    else fprintf(stderr, " %7.3f (default)", 0.0);
    fprintf(stderr, "\n");
  }
  r→slab.g = (r→default_g ≡ UNINITIALIZED) ? 0 : r→default_g;
  Set_Calc.State(m, *r);
  ⟨Get the initial a, b, and g 237⟩
  ⟨Initialize the nodes of the a and b simplex 238⟩
  ⟨Evaluate the a and b simplex at the nodes 239⟩
  amoeba(p, y, 2, r→tolerance, Find_AB_fn, &r→AD_iterations);
  ⟨Choose the best node of the a and b simplex 240⟩
  ⟨Free simplex data structures 242⟩
  ⟨Put final values in result 241⟩
}
```

This code is used in section 232.

236. To use the simplex algorithm, we need to vectors and a matrix.

⟨Allocate local simplex variables 236⟩ ≡

```
int i, i_best, j_best;
double *x, *y, **p;

x = dvector(1, 2);
y = dvector(1, 3);
p = dmatrix(1, 3, 1, 2);
```

This code is used in sections 235, 255, 260, 266, and 271.

237. Just get the optimal optical properties to start the search process.

I had to add the line that tests to make sure the albedo is greater than 0.2 because the grid just does not work so well in this case. The problem is that for low albedos there is really very little information about the anisotropy available. This change was also made in the analagous code for a and b .

⟨ Get the initial a , b , and g 237 ⟩ ≡

```
{
    /* double a3,b3,g3; */
    size_t count = NUMBER_OF_GUESSES; /* distance to last result */
    abg_distance(r→slab.a, r→slab.b, r→slab.g, &(guess[0]));
    if (¬Valid_Grid(m, *r)) Fill_Grid(m, *r, 1); /* distance to nearest grid point */
    Near_Grid_Points(m.m_r, m.m_t, r→search, &i_best, &j_best);
    Grid_ABG(i_best, j_best, &(guess[1]));
    Grid_ABG(i_best + 1, j_best, &(guess[2]));
    Grid_ABG(i_best - 1, j_best, &(guess[3]));
    Grid_ABG(i_best, j_best + 1, &(guess[4]));
    Grid_ABG(i_best, j_best - 1, &(guess[5]));
    Grid_ABG(i_best + 1, j_best + 1, &(guess[6]));
    Grid_ABG(i_best - 1, j_best - 1, &(guess[7]));
    Grid_ABG(i_best + 1, j_best - 1, &(guess[8]));
    Grid_ABG(i_best - 1, j_best + 1, &(guess[9]));
    qsort((void *) guess, count, sizeof(guess_type), compare_guesses);
    if (Debug(DEBUG_BEST_GUESS)) {
        int k;
        fprintf(stderr, "BEST: _GRID_GUESSES\n");
        fprintf(stderr, "BEST: _k_ albedo _b_ g _distance\n");
        for (k = 0; k ≤ 6; k++) {
            fprintf(stderr, "BEST: %3d _", k);
            fprintf(stderr, "%10.5f _", guess[k].a);
            fprintf(stderr, "%10.5f _", guess[k].b);
            fprintf(stderr, "%10.5f _", guess[k].g);
            fprintf(stderr, "%10.5f\n", guess[k].distance);
        }
    }
}
```

This code is used in sections 235, 255, 260, 266, and 271.

238. $\langle$ Initialize the nodes of the a and b simplex 238 $\rangle \equiv$

```
{
    int k, kk;
    p[1][1] = a2acalc(guess[0].a);
    p[1][2] = b2bcalc(guess[0].b);
    for (k = 1; k < 7; k++) {
        if (guess[0].a  $\neq$  guess[k].a) break;
    }
    p[2][1] = a2acalc(guess[k].a);
    p[2][2] = b2bcalc(guess[k].b);
    for (kk = 1; kk < 7; kk++) {
        if (k  $\equiv$  kk) continue;
        if (guess[0].b  $\neq$  guess[kk].b  $\vee$  guess[k].b  $\neq$  guess[kk].b) break;
    }
    p[3][1] = a2acalc(guess[kk].a);
    p[3][2] = b2bcalc(guess[kk].b);
    if (Debug(DEBUG_BEST_GUESS)) {
        fprintf(stderr, "-----\n");
        fprintf(stderr, "BEST: <1>");
        fprintf(stderr, "%10.5f", guess[0].a);
        fprintf(stderr, "%10.5f", guess[0].b);
        fprintf(stderr, "%10.5f", guess[0].g);
        fprintf(stderr, "%10.5f\n", guess[0].distance);
        fprintf(stderr, "BEST: <2>");
        fprintf(stderr, "%10.5f", guess[k].a);
        fprintf(stderr, "%10.5f", guess[k].b);
        fprintf(stderr, "%10.5f", guess[k].g);
        fprintf(stderr, "%10.5f\n", guess[k].distance);
        fprintf(stderr, "BEST: <3>");
        fprintf(stderr, "%10.5f", guess[kk].a);
        fprintf(stderr, "%10.5f", guess[kk].b);
        fprintf(stderr, "%10.5f", guess[kk].g);
        fprintf(stderr, "%10.5f\n", guess[kk].distance);
        fprintf(stderr, "\n");
    }
}
```

This code is used in section 235.

239. $\langle$ Evaluate the a and b simplex at the nodes 239 $\rangle \equiv$

```
for (i = 1; i  $\leq$  3; i++) {
    x[1] = p[i][1];
    x[2] = p[i][2];
    y[i] = Find_AB_fn(x);
}
```

This code is used in section 235.

240. $\langle$ Choose the best node of the a and b simplex 240 $\rangle \equiv$
 $r\text{-final_distance} = 10;$
for ($i = 1; i \leq 3; i++$) {
 if ($y[i] < r\text{-final_distance}$) {
 $r\text{-slab}.a = \text{acalc2a}(p[i][1]);$
 $r\text{-slab}.b = \text{bcalc2b}(p[i][2]);$
 $r\text{-final_distance} = y[i];$
 }
}

This code is used in section 235.

241. $\langle$ Put final values in result 241 $\rangle \equiv$
 $r\text{-a} = r\text{-slab}.a;$
 $r\text{-b} = r\text{-slab}.b;$
 $r\text{-g} = r\text{-slab}.g;$
 $r\text{-found} = (r\text{-tolerance} > r\text{-final_distance});$
 $\text{Set_Calc_State}(m, *r);$

This code is used in sections 235, 244, 246, 248, 250, 252, 255, 260, 266, and 271.

242. Since we allocated these puppies, we got to get rid of them.

$\langle$ Free simplex data structures 242 $\rangle \equiv$
 $\text{free_dvector}(x, 1, 2);$
 $\text{free_dvector}(y, 1, 3);$
 $\text{free_dmatrix}(p, 1, 3, 1, 2);$

This code is used in sections 235, 255, 260, 266, and 271.

243. Fixed Absorption and Anisotropy. Typically, this routine is called when the absorption coefficient is known, the anisotropy is known, and the physical thickness of the sample is known. This routine calculates the varies the scattering coefficient until the measurements are matched.

This was written for Ted Moffitt to analyze some intralipid data. We wanted to know what the scattering coefficient of the Intralipid was and made total transmission measurements through a sample with a fixed physical thickness. We did not make reflection measurements because the light source diverged too much, and we could not make reflection measurements easily.

In retrospect, we could have made URU measurements by illuminating the wall of the integrating sphere. However, these diffuse type of measurements are very difficult to make accurately.

This is tricky only because the value in *slab.b* is used to hold the value of *ba* or $d \cdot \mu_a$ when the *Find_Bs_fn* is used.

⟨Prototype for *U_Find_Bs* 243⟩ ≡

```
void U_Find_Bs(struct measure_type m, struct invert_type *r)
```

This code is used in sections 233 and 244.

244. ⟨Definition for *U_Find_Bs* 244⟩ ≡

⟨Prototype for *U_Find_Bs* 243⟩

```
{
    double ax, bx, cx, fa, fb, fc, bs;
    if (Debug(DEBUG_SEARCH)) {
        fprintf(stderr, "SEARCH: Using U_Find_Bs()");
        fprintf(stderr, " (mu=%6.4f)", r->slab.cos_angle);
        if (r->default_ba != UNINITIALIZED) fprintf(stderr, " default_ba=%8.5f", r->default_ba);
        if (r->default_g != UNINITIALIZED) fprintf(stderr, " default_g=%8.5f", r->default_g);
        fprintf(stderr, "\n");
    }
    if (m.m_t == 0) {
        r->slab.b = HUGE_VAL;
        U_Find_A(m, r);
        return;
    }
    r->slab.a = 0;
    r->slab.g = (r->default_g == UNINITIALIZED) ? 0 : r->default_g;
    r->slab.b = (r->default_ba == UNINITIALIZED) ? HUGE_VAL : r->default_ba;
    Set_Calc_State(m, *r); /* store ba in RR.slab.b */
    ax = b2bcalc(0.1); /* first try for bs */
    bx = b2bcalc(1.0);
    mnbrak(&ax, &bx, &cx, &fa, &fb, &fc, Find_Bs_fn, &r->AD_iterations);
    r->final_distance = brent(ax, bx, cx, Find_Bs_fn, r->tolerance, &bs, &r->AD_iterations);
    /* recover true values */
    r->slab.a = bcalc2b(bs) / (bcalc2b(bs) + r->slab.b);
    r->slab.b = bcalc2b(bs) + r->slab.b;
    ⟨Put final values in result 241⟩
}
```

This code is used in section 232.

245. Fixed Absorption and Scattering. Typically, this routine is called when the scattering coefficient is known, the anisotropy is known, and the physical thickness of the sample is known. This routine calculates the varies the absorption coefficient until the measurements are matched.

This is tricky only because the value in *slab.b* is used to hold the value of *bs* or $d \cdot \mu_s$ when the *Find_Ba_fn* is used.

⟨Prototype for *U_Find_Ba* 245⟩ ≡

```
void U_Find_Ba(struct measure_type m, struct invert_type *r)
```

This code is used in sections 233 and 246.

246. ⟨Definition for *U_Find_Ba* 246⟩ ≡

⟨Prototype for *U_Find_Ba* 245⟩

```
{
    double ax, bx, cx, fa, fb, fc, ba;
    if (Debug(DEBUG_SEARCH)) {
        fprintf(stderr, "SEARCH: Using U_Find_Bs()");
        fprintf(stderr, " (mu=%6.4f)", r→slab.cos_angle);
        if (r→default_bs ≠ UNINITIALIZED) fprintf(stderr, " default_bs = %8.5f", r→default_bs);
        if (r→default_g ≠ UNINITIALIZED) fprintf(stderr, " default_g = %8.5f", r→default_g);
        fprintf(stderr, "\n");
    }
    r→slab.a = 0;
    r→slab.g = (r→default_g ≡ UNINITIALIZED) ? 0 : r→default_g;
    r→slab.b = (r→default_bs ≡ UNINITIALIZED) ? HUGE_VAL : r→default_bs;
    if (m.m_t ≡ 0) {
        r→slab.b = HUGE_VAL;
        U_Find_A(m, r);
        return;
    }
    Set_Calc_State(m, *r); /* store bs in RR.slabs.b */
    ax = b2bcalc(0.1); /* first try for ba */
    bx = b2bcalc(1.0);
    mnbrak(&ax, &bx, &cx, &fa, &fb, &fc, Find_Ba_fn, &r→AD_iterations);
    r→final_distance = brent(ax, bx, cx, Find_Ba_fn, r→tolerance, &ba, &r→AD_iterations);
    /* recover true values */
    r→slab.a = (r→slab.b) / (bcalc2b(ba) + r→slab.b);
    r→slab.b = bcalc2b(ba) + r→slab.b; /* actual value of b */
    ⟨Put final values in result 241⟩
}
```

This code is used in section 232.

247. Fixed Optical Depth and Anisotropy. Typically, this routine is called when the optical thickness is assumed infinite. However, it may also be called when the optical thickness is assumed to be fixed at a particular value. Typically the only reasonable situation for this to occur is when the diffuse transmission is non-zero but the collimated transmission is zero. If this is the case then there is no information in the collimated transmission measurement and there is no sense even using it because the slab is not infinitely thick.

⟨Prototype for *U_Find_A* 247⟩ ≡

```
void U_Find_A(struct measure_type m, struct invert_type *r)
```

This code is used in sections 233 and 248.

248. ⟨Definition for *U_Find_A* 248⟩ ≡

⟨Prototype for *U_Find_A* 247⟩

```
{
    double Rt, Tt, Rd, Ru, Td, Tu;
    if (Debug(DEBUG_SEARCH)) {
        fprintf(stderr, "SEARCH: Using U_Find_A()");
        fprintf(stderr, " (mu=%6.4f)", r→slab.cos_angle);
        if (r→default_b ≠ UNINITIALIZED) fprintf(stderr, " default_b=%8.5f", r→default_b);
        if (r→default_g ≠ UNINITIALIZED) fprintf(stderr, " default_g=%8.5f", r→default_g);
        fprintf(stderr, "\n");
    }
    Estimate_RT(m, *r, &Rt, &Tt, &Rd, &Ru, &Td, &Tu);
    r→slab.g = (r→default_g ≡ UNINITIALIZED) ? 0 : r→default_g;
    r→slab.b = (r→default_b ≡ UNINITIALIZED) ? HUGE_VAL : r→default_b;
    r→slab.a = 0.0;
    r→final_distance = 0.0;
    Set_Calc_State(m, *r);
    if (Rt > 0.99999) {
        r→final_distance = Find_A_fn(a2acalc(1.0));
        r→slab.a = 1.0;
    }
    else {
        double x, ax, bx, cx, fa, fb, fc;
        ax = a2acalc(0.3);
        bx = a2acalc(0.5);
        mnbrak(&ax, &bx, &cx, &fa, &fb, &fc, Find_A_fn, &r→AD_iterations);
        r→final_distance = brent(ax, bx, cx, Find_A_fn, r→tolerance, &x, &r→AD_iterations);
        r→slab.a = acalc2a(x);
    }
    ⟨Put final values in result 241⟩
}
```

This code is used in section 232.

249. Fixed Optical Depth and Albedo.

⟨Prototype for *U_Find_G* 249⟩ ≡

```
void U_Find_G(struct measure_type m, struct invert_type *r)
```

This code is used in sections 233 and 250.

250. ⟨Definition for *U_Find_G* 250⟩ ≡

⟨Prototype for *U_Find_G* 249⟩

```
{
  double Rt, Tt, Rd, Ru, Td, Tu;
  double x, ax, bx, cx, fa, fb, fc;
  if (Debug(DEBUG_SEARCH)) {
    fprintf(stderr, "SEARCH: Using U_Find_G()");
    fprintf(stderr, " (mu=%6.4f)", r→slab.cos_angle);
    if (r→default_a ≠ UNINITIALIZED) fprintf(stderr, " default_a = %8.5f", r→default_a);
    if (r→default_b ≠ UNINITIALIZED) fprintf(stderr, " default_b = %8.5f", r→default_b);
    fprintf(stderr, "\n");
  }
  Estimate_RT(m, *r, &Rt, &Tt, &Rd, &Ru, &Td, &Tu);
  r→slab.a = (r→default_a ≡ UNINITIALIZED) ? 0.5 : r→default_a;
  if (r→default_b ≠ UNINITIALIZED) r→slab.b = r→default_b;
  else if (m.m_u > 0) r→slab.b = What_Is_B(r→slab, m.m_u);
  else r→slab.b = HUGE_VAL;
  r→slab.g = 0.0;
  r→final_distance = 0.0;
  Set_Calc_State(m, *r);
  ax = g2gcalc(−0.99);
  bx = g2gcalc(0.99);
  mnbrak(&ax, &bx, &cx, &fa, &fb, &fc, Find_G_fn, &r→AD_iterations);
  r→final_distance = brent(ax, bx, cx, Find_G_fn, r→tolerance, &x, &r→AD_iterations);
  r→slab.g = gcalc2g(x);
  Set_Calc_State(m, *r);
  ⟨Put final values in result 241⟩
}
```

This code is used in section 232.

251. Fixed Anisotropy and Albedo. This routine can be called in three different situations: (1) the albedo is zero, (2) the albedo is one, or (3) the albedo is fixed at a default value. I calculate the individual reflections and transmissions to establish which of these cases we happen to have.

⟨Prototype for *U_Find_B* 251⟩ ≡

```
void U_Find_B(struct measure_type m, struct invert_type *r)
```

This code is used in sections 233 and 252.

252. ⟨Definition for *U_Find_B* 252⟩ ≡

⟨Prototype for *U_Find_B* 251⟩

```
{
    double Rt, Tt, Rd, Ru, Td, Tu;
    if (Debug(DEBUG_SEARCH)) {
        fprintf(stderr, "SEARCH: Using U_Find_B()");
        fprintf(stderr, "(mu=%6.4f)", r->slab.cos_angle);
        if (r->default_a ≠ UNINITIALIZED) fprintf(stderr, " default_a=%8.5f", r->default_a);
        if (r->default_g ≠ UNINITIALIZED) fprintf(stderr, " default_g=%8.5f", r->default_g);
        fprintf(stderr, "\n");
    }
    Estimate_RT(m, *r, &Rt, &Tt, &Rd, &Ru, &Td, &Tu);
    r->slab.g = (r->default_g ≠ UNINITIALIZED) ? 0 : r->default_g;
    r->slab.a = (r->default_a ≠ UNINITIALIZED) ? 0 : r->default_a;
    r->slab.b = 0.5;
    r->final_distance = 0.0;
    Set_Calc_State(m, *r);
    ⟨Iteratively solve for b 253⟩
    ⟨Put final values in result 241⟩
}
```

This code is used in section 232.

253. This could be improved tremendously. I just don't want to mess with it at the moment.

⟨Iteratively solve for *b* 253⟩ ≡

```
{
    double x, ax, bx, cx, fa, fb, fc;
    ax = b2bcalc(0.1);
    bx = b2bcalc(10);
    mnbrak(&ax, &bx, &cx, &fa, &fb, &fc, Find_B_fn, &r->AD_iterations);
    r->final_distance = brent(ax, bx, cx, Find_B_fn, r->tolerance, &x, &r->AD_iterations);
    r->slab.b = bcalc2b(x);
    Set_Calc_State(m, *r);
}
```

This code is used in section 252.

254. Fixed Optical Depth.

We can get here a couple of different ways.

First there can be three real measurements, i.e., t_c is not zero, in this case we want to fix b based on the t_c measurement.

Second, we can get here if a default value for b has been set.

Otherwise, we really should not be here. Just set $b = 1$ and calculate away.

⟨Prototype for *U_Find_AG* 254⟩ ≡

```
void U_Find_AG(struct measure_type m, struct invert_type *r)
```

This code is used in sections 233 and 255.

255. ⟨Definition for *U_Find_AG* 255⟩ ≡

⟨Prototype for *U_Find_AG* 254⟩

```
{
  ⟨Allocate local simplex variables 236⟩
  if (Debug(DEBUG_SEARCH)) {
    fprintf(stderr, "SEARCH: Using U_Find_AG()");
    fprintf(stderr, "μ=%4.2f, ", r→slab.cos_angle);
    if (m.num_measures ≡ 3) fprintf(stderr, "b=%6.3f (M-U)", What_Is_B(r→slab, m.m_u));
    else if (r→default_b ≠ UNINITIALIZED) fprintf(stderr, "b=%6.3f (constrained)", r→default_b);
    else fprintf(stderr, "b=%6.3f (default)", 1.0);
    fprintf(stderr, "\n");
  }
  if (m.num_measures ≡ 3) r→slab.b = What_Is_B(r→slab, m.m_u);
  else if (r→default_b ≡ UNINITIALIZED) r→slab.b = 1;
  else r→slab.b = r→default_b;
  Set_Calc_State(m, *r);
  ⟨Get the initial a, b, and g 237⟩
  ⟨Initialize the nodes of the a and g simplex 256⟩
  ⟨Evaluate the a and g simplex at the nodes 257⟩
  amoeba(p, y, 2, r→tolerance, Find_AG_fn, &r→AD_iterations);
  ⟨Choose the best node of the a and g simplex 258⟩
  ⟨Free simplex data structures 242⟩
  ⟨Put final values in result 241⟩
}
```

This code is used in section 232.

256. $\langle$ Initialize the nodes of the a and g simplex 256 $\rangle \equiv$

```

{
  int k, kk;
  p[1][1] = a2acalc(guess[0].a);
  p[1][2] = g2gcalc(guess[0].g);
  for (k = 1; k < 7; k++) {
    if (guess[0].a  $\neq$  guess[k].a) break;
  }
  p[2][1] = a2acalc(guess[k].a);
  p[2][2] = g2gcalc(guess[k].g);
  for (kk = 1; kk < 7; kk++) {
    if (kk  $\equiv$  k) continue;
    if (guess[0].g  $\neq$  guess[kk].g  $\vee$  guess[k].g  $\neq$  guess[kk].g) break;
  }
  p[3][1] = a2acalc(guess[kk].a);
  p[3][2] = g2gcalc(guess[kk].g);
  if (Debug(DEBUG_BEST_GUESS)) {
    fprintf(stderr, "-----\n");
    fprintf(stderr, "BEST: <1>");
    fprintf(stderr, "%10.5f", guess[0].a);
    fprintf(stderr, "%10.5f", guess[0].b);
    fprintf(stderr, "%10.5f", guess[0].g);
    fprintf(stderr, "%10.5f\n", guess[0].distance);
    fprintf(stderr, "BEST: <2>");
    fprintf(stderr, "%10.5f", guess[k].a);
    fprintf(stderr, "%10.5f", guess[k].b);
    fprintf(stderr, "%10.5f", guess[k].g);
    fprintf(stderr, "%10.5f\n", guess[k].distance);
    fprintf(stderr, "BEST: <3>");
    fprintf(stderr, "%10.5f", guess[kk].a);
    fprintf(stderr, "%10.5f", guess[kk].b);
    fprintf(stderr, "%10.5f", guess[kk].g);
    fprintf(stderr, "%10.5f\n", guess[kk].distance);
    fprintf(stderr, "\n");
  }
}

```

This code is used in section 255.

257. $\langle$ Evaluate the a and g simplex at the nodes 257 $\rangle \equiv$

```

for (i = 1; i  $\leq$  3; i++) {
  x[1] = p[i][1];
  x[2] = p[i][2];
  y[i] = Find_AG_fn(x);
}

```

This code is used in section 255.

258. Here we find the node of the simplex that gave the best result and save that one. At the same time we save the whole simplex for later use if needed.

⟨ Choose the best node of the a and g simplex 258 ⟩ $\equiv$

```

     $r\text{-final\_distance} = 10;$ 
    for ( $i = 1; i \leq 3; i++$ ) {
        if ( $y[i] < r\text{-final\_distance}$ ) {
             $r\text{-slab}.a = a\text{calc}2a(p[i][1]);$ 
             $r\text{-slab}.g = g\text{calc}2g(p[i][2]);$ 
             $r\text{-final\_distance} = y[i];$ 
        }
    }

```

This code is used in section 255.

259. Fixed Albedo. Here the optical depth and the anisotropy are varied (for a fixed albedo).

⟨Prototype for *U_Find_BG* 259⟩ ≡

```
void U_Find_BG(struct measure_type m, struct invert_type *r)
```

This code is used in sections 233 and 260.

260. ⟨Definition for *U_Find_BG* 260⟩ ≡

⟨Prototype for *U_Find_BG* 259⟩

```
{
  ⟨Allocate local simplex variables 236⟩
  if (Debug(DEBUG_SEARCH)) {
    fprintf(stderr, "SEARCH: Using U_Find_BG()");
    fprintf(stderr, " (mu=%6.4f)", r_slab.cos_angle);
    if (r_default_a ≠ UNINITIALIZED) fprintf(stderr, " default_a=%8.5f", r_default_a);
    fprintf(stderr, "\n");
  }
  r_slab.a = (r_default_a ≡ UNINITIALIZED) ? 0 : r_default_a;
  Set_Calc_State(m, *r);
  ⟨Get the initial a, b, and g 237⟩
  ⟨Initialize the nodes of the b and g simplex 262⟩
  ⟨Evaluate the bg simplex at the nodes 263⟩
  amoeba(p, y, 2, r_tolerance, Find_BG_fn, &r_AD_iterations);
  ⟨Choose the best node of the b and g simplex 264⟩
  ⟨Free simplex data structures 242⟩
  ⟨Put final values in result 241⟩
}
```

This code is used in section 232.

261. A very simple start for variation of *b* and *g*. This should work fine for the cases in which the absorption or scattering are fixed.

262. $\langle$ Initialize the nodes of the b and g simplex 262 $\rangle \equiv$

```

{
  int k, kk;
  p[1][1] = b2bcalc(guess[0].b);
  p[1][2] = g2gcalc(guess[0].g);
  for (k = 1; k < 7; k++) {
    if (guess[0].b  $\neq$  guess[k].b) break;
  }
  p[2][1] = b2bcalc(guess[k].b);
  p[2][2] = g2gcalc(guess[k].g);
  for (kk = 1; kk < 7; kk++) {
    if (kk  $\equiv$  k) continue;
    if (guess[0].g  $\neq$  guess[kk].g  $\vee$  guess[k].g  $\neq$  guess[kk].g) break;
  }
  p[3][1] = b2bcalc(guess[kk].b);
  p[3][2] = g2gcalc(guess[kk].g);
  if (Debug(DEBUG_BEST_GUESS)) {
    fprintf(stderr, "-----\n");
    fprintf(stderr, "BEST: <1>");
    fprintf(stderr, "%10.5f", guess[0].a);
    fprintf(stderr, "%10.5f", guess[0].b);
    fprintf(stderr, "%10.5f", guess[0].g);
    fprintf(stderr, "%10.5f\n", guess[0].distance);
    fprintf(stderr, "BEST: <2>");
    fprintf(stderr, "%10.5f", guess[k].a);
    fprintf(stderr, "%10.5f", guess[k].b);
    fprintf(stderr, "%10.5f", guess[k].g);
    fprintf(stderr, "%10.5f\n", guess[k].distance);
    fprintf(stderr, "BEST: <3>");
    fprintf(stderr, "%10.5f", guess[kk].a);
    fprintf(stderr, "%10.5f", guess[kk].b);
    fprintf(stderr, "%10.5f", guess[kk].g);
    fprintf(stderr, "%10.5f\n", guess[kk].distance);
    fprintf(stderr, "\n");
  }
}

```

This code is used in section 260.

263. $\langle$ Evaluate the bg simplex at the nodes 263 $\rangle \equiv$

```

for (i = 1; i  $\leq$  3; i++) {
  x[1] = p[i][1];
  x[2] = p[i][2];
  y[i] = Find_BG_fn(x);
}

```

This code is used in section 260.

264. Here we find the node of the simplex that gave the best result and save that one. At the same time we save the whole simplex for later use if needed.

⟨ Choose the best node of the b and g simplex 264 ⟩ ≡

```

r-final_distance = 10;
for ( $i = 1; i \leq 3; i++$ ) {
  if ( $y[i] < r\text{-}final\_distance$ ) {
     $r\text{-}slab.b = bcalc2b(p[i][1]);$ 
     $r\text{-}slab.g = gcalc2g(p[i][2]);$ 
     $r\text{-}final\_distance = y[i];$ 
  }
}
```

This code is used in section 260.

265. Fixed Scattering. Here I assume that a constant b_s ,

$$b_s = \mu_s d$$

where d is the physical thickness of the sample and μ_s is of course the absorption coefficient. This is just like *U_Find_BaG* except that $b_a = \mu_a d$ is varied instead of b .

⟨Prototype for *U_Find_BaG* 265⟩ ≡

```
void U_Find_BaG(struct measure_type m, struct invert_type *r)
```

This code is used in sections 233 and 266.

266. ⟨Definition for *U_Find_BaG* 266⟩ ≡

⟨Prototype for *U_Find_BaG* 265⟩

```
{
  ⟨Allocate local simplex variables 236⟩
  Set_Calc_State(m, *r);
  ⟨Get the initial a, b, and g 237⟩
  ⟨Initialize the nodes of the ba and g simplex 267⟩
  ⟨Evaluate the BaG simplex at the nodes 268⟩
  amoeba(p, y, 2, r-tolerance, Find_BaG_fn, &r-AD-iterations);
  ⟨Choose the best node of the ba and g simplex 269⟩
  ⟨Free simplex data structures 242⟩
  ⟨Put final values in result 241⟩
}
```

This code is used in section 232.

267. ⟨Initialize the nodes of the *ba* and *g* simplex 267⟩ ≡

```
if (guess[0].b > r-default-bs) {
  p[1][1] = b2bcalc(guess[0].b - r-default-bs);
  p[2][1] = b2bcalc(2 * (guess[0].b - r-default-bs));
  p[3][1] = p[1][1];
}
else {
  p[1][1] = b2bcalc(0.0001);
  p[2][1] = b2bcalc(0.001);
  p[3][1] = p[1][1];
}
p[1][2] = g2gcalc(guess[0].g);
p[2][2] = p[1][2];
p[3][2] = g2gcalc(0.9 * guess[0].g + 0.05);
```

This code is used in section 266.

268. ⟨Evaluate the *BaG* simplex at the nodes 268⟩ ≡

```
for (i = 1; i ≤ 3; i++) {
  x[1] = p[i][1];
  x[2] = p[i][2];
  y[i] = Find_BaG_fn(x);
}
```

This code is used in section 266.

269. Here we find the node of the simplex that gave the best result and save that one. At the same time we save the whole simplex for later use if needed.

⟨ Choose the best node of the *ba* and *g* simplex 269 ⟩ ≡

```

    r→final_distance = 10;
    for (i = 1; i ≤ 3; i++) {
        if (y[i] < r→final_distance) {
            r→slab.b = bcalc2b(p[i][1]) + r→default_bs;
            r→slab.a = r→default_bs / r→slab.b;
            r→slab.g = gcalc2g(p[i][2]);
            r→final_distance = y[i];
        }
    }

```

This code is used in section 266.

$$b_a = \mu_a d$$
$$\langle \text{Prototype for } U_Find_BsG \text{ 270} \rangle \equiv$$

This code is used in sections 233 and 271.

⟨ Prototype for *U_Find_BsG* 270 ⟩

This code is used in section [232](#).

```

p[1][1] = b2bcalc(guess[0].b - r-default_ba);
p[1][2] = g2gcalc(guess[0].g);
p[2][1] = b2bcalc(2 * guess[0].b - 2 * r-default_ba);
p[2][2] = p[1][2];
p[3][1] = p[1][1];
p[3][2] = q2qcalc(0.9 * guess[0].g + 0.05);

```

This code is used in section [271](#).

```

for ( $i = 1$ ;  $i \leq 3$ ;  $i++$ ) {
   $x[1] = p[i][1]$ ;
   $x[2] = p[i][2]$ ;
   $y[i] = Find\_BsG\_fn(x)$ ;
}

```

This code is used in section [271](#).

274. $\langle$ Choose the best node of the *bs* and *g* simplex 274 $\rangle \equiv$
 $r\text{-final_distance} = 10;$
for ($i = 1; i \leq 3; i++$) {
 if ($y[i] < r\text{-final_distance}$) {
 $r\text{-slab}.b = bcalc2b(p[i][1]) + r\text{-default_ba};$
 $r\text{-slab}.a = 1 - r\text{-default_ba}/r\text{-slab}.b;$
 $r\text{-slab}.g = gcalc2g(p[i][2]);$
 $r\text{-final_distance} = y[i];$
 }
}

This code is used in section 271.

275. IAD Utilities.

March 1995. Reincluded *quick_guess* code.

```

< iad_util.c 275 > ≡
#include <math.h>
#include <float.h>
#include <stdio.h>
#include "nr_util.h"
#include "ad_globl.h"
#include "ad_frsnl.h"
#include "ad_bound.h"
#include "iad_type.h"
#include "iad_calc.h"
#include "iad_pub.h"
#include "iad_util.h"
    unsigned long g_util_debugging = 0;
    < Preprocessor definitions >
    < Definition for What_Is_B 278 >
    < Definition for Estimate_RT 284 >
    < Definition for a2acalc 291 >
    < Definition for acalc2a 293 >
    < Definition for g2gcalc 295 >
    < Definition for gcalc2g 297 >
    < Definition for b2bcalc 299 >
    < Definition for bcalc2b 301 >
    < Definition for twoprime 303 >
    < Definition for twounprime 305 >
    < Definition for abgg2ab 307 >
    < Definition for abgb2ag 309 >
    < Definition for quick_guess 316 >
    < Definition for Set_Debugging 329 >
    < Definition for Debug 331 >
    < Definition for sqr 333 >
    < Definition for Calculate_Mua_Musp 335 >
    < Definition for Print_Invert_Type 337 >
    < Definition for Print_Measure_Type 339 >

```

276. `<iad_util.h 276>` $\equiv$
 `<Prototype for What_Is_B 277>;`
 `<Prototype for Estimate_RT 283>;`
 `<Prototype for a2acalc 290>;`
 `<Prototype for acalc2a 292>;`
 `<Prototype for g2gcalc 294>;`
 `<Prototype for gcalc2g 296>;`
 `<Prototype for b2bcalc 298>;`
 `<Prototype for bcalc2b 300>;`
 `<Prototype for twoprime 302>;`
 `<Prototype for twounprime 304>;`
 `<Prototype for abgg2ab 306>;`
 `<Prototype for abgb2ag 308>;`
 `<Prototype for quick_guess 315>;`
 `<Prototype for Set_Debugging 328>;`
 `<Prototype for Calculate_Mua_Musp 334>;`
 `<Prototype for Debug 330>;`
 `<Prototype for sqr 332>;`
 `<Prototype for Print_Invert_Type 336>;`
 `<Prototype for Print_Measure_Type 338>;`

277. Finding optical thickness.

This routine figures out what the optical thickness of a slab based on the index of refraction of the slab and the amount of collimated light that gets through it.

It should be pointed out right here in the front that this routine does not work for diffuse irradiance, but then the whole concept of estimating the optical depth for diffuse irradiance is bogus anyway.

⟨Prototype for *What_Is_B* 277⟩ ≡

double *What_Is_B*(**struct** **AD_slab_type** *slab*, **double** *Tu*)

This code is used in sections 276 and 278.

278. ⟨Definition for *What_Is_B* 278⟩ ≡

⟨Prototype for *What_Is_B* 277⟩

{

double *r1*, *r2*, *t1*, *t2*, *mu_in_slab*;

⟨Calculate specular reflection and transmission 279⟩

⟨Check for bad values of *Tu* 280⟩

⟨Solve if multiple internal reflections are not present 281⟩

⟨Find thickness when multiple internal reflections are present 282⟩

}

This code is used in section 275.

279. The first thing to do is to find the specular reflection for light interacting with the top and bottom air-glass-sample interfaces. I make a simple check to ensure that the the indices are different before calculating the bottom reflection. Most of the time the $r1 \equiv r2$, but there are always those annoying special cases.

⟨Calculate specular reflection and transmission 279⟩ ≡

Absorbing_Glass_RT(1.0, *slab.n_top_slide*, *slab.n_slab*, *slab.cos_angle*, *slab.b_top_slide*, &*r1*, &*t1*);

mu_in_slab = *Cos_Snell*(1.0, *slab.cos_angle*, *slab.n_slab*);

Absorbing_Glass_RT(*slab.n_slab*, *slab.n_bottom_slide*, 1.0, *mu_in_slab*, *slab.b_bottom_slide*, &*r2*, &*t2*);

This code is used in section 278.

280. Bad values for the unscattered transmission are those that are non-positive, those greater than one, and those greater than are possible in a non-absorbing medium, i.e.,

$$T_c > \frac{t_1 t_2}{1 - r_1 r_2}$$

Since this routine has no way to report errors, I just set the optical thickness to the natural values in these cases.

⟨Check for bad values of *Tu* 280⟩ ≡

if (*Tu* ≤ 0) **return** (HUGE_VAL);

if (*Tu* ≥ *t1* * *t2* / (1 - *r1* * *r2*)) **return** (0.001);

This code is used in section 278.

281. If either $r1$ or $r2 \equiv 0$ then things are very simple because the sample does not sustain multiple internal reflections and the unscattered transmission is

$$T_c = t_1 t_2 \exp(-b/\nu)$$

where b is the optical thickness and ν is `slab.cos_angle`. Clearly,

$$b = -\nu \ln \left(\frac{T_c}{t_1 t_2} \right)$$

⟨Solve if multiple internal reflections are not present 281⟩ ≡

if ($r1 \equiv 0 \vee r2 \equiv 0$) **return** ($-slab.cos_angle * \log(Tu/t1/t2)$);

This code is used in section 278.

282. Well I kept putting it off, but now comes the time to solve the following equation for b

$$T_c = \frac{t_1 t_2 \exp(-b)}{1 - r_1 r_2 \exp(-2b)}$$

We note immediately that this is a quadratic equation in $x = \exp(-b)$.

$$r_1 r_2 T_c x^2 + t_1 t_2 x - T_c = 0$$

Sufficient tests have been made above to ensure that none of the coefficients are exactly zero. However, it is clear that the leading quadratic term has a much smaller coefficient than the other two. Since r_1 and r_2 are typically about four percent the product is roughly 10^{-3} . The collimated transmission can be very small and this makes things even worse. A further complication is that we need to choose the only positive root.

Now the roots of $ax^2 + bx + c = 0$ can be found using the standard quadratic formula,

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

This is very bad for small values of a . Instead I use

$$q = -\frac{1}{2} \left[b + \text{sgn}(b) \sqrt{b^2 - 4ac} \right]$$

with the two roots

$$x = \frac{q}{a} \quad \text{and} \quad x = \frac{c}{q}$$

Substituting our coefficients

$$q = -\frac{1}{2} \left[t_1 t_2 + \sqrt{t_1^2 t_2^2 + 4 r_1 r_2 T_c^2} \right]$$

With some algebra, this can be shown to be

$$q = -t_1 t_2 \left[1 + \frac{r_1 r_2 T_c^2}{t_1^2 t_2^2} + \dots \right]$$

The only positive root is $x = -T_c/q$. Therefore

$$x = \frac{2T_c}{t_1 t_2 + \sqrt{t_1^2 t_2^2 + 4 r_1 r_2 T_c^2}}$$

(Not very pretty, but straightforward enough.)

⟨Find thickness when multiple internal reflections are present 282⟩ ≡

```
{
  double B;
  B = t1 * t2;
  return (-slab.cos_angle * log(2 * Tu / (B + sqrt(B * B + 4 * Tu * Tu * r1 * r2))));
}
```

This code is used in section 278.

283. Estimating R and T.

In several places, it is useful to know an *estimate* for the values of the reflection and transmission of the sample based on the measurements. This routine provides such an estimate, but it currently ignores anything corrections that might be made for the integrating spheres.

Good values are only really obtainable when *num_measures* $\equiv$ 3, otherwise we need to make pretty strong assumptions about the reflection and transmission values. If *num_measures* < 3, then we will assume that no collimated light makes it all the way through the sample. The specular reflection is then just that for a semi-infinite sample and *Tu* = 0. If *num_measures* $\equiv$ 1, then *Td* is also set to zero.

rt total reflection
rc primary or specular reflection
rd diffuse or scattered reflection
tt total transmission
tp primary or unscattered transmission
td diffuse or scattered transmission

⟨Prototype for *Estimate_RT* 283⟩ $\equiv$

```
void Estimate_RT(struct measure_type m, struct invert_type r, double *rt, double *tt, double
    *rd, double *rc, double *td, double *tc)
```

This code is used in sections 276 and 284.

284. ⟨Definition for *Estimate_RT* 284⟩ $\equiv$

```
⟨Prototype for Estimate_RT 283⟩
{
    ⟨Calculate the unscattered transmission and reflection 285⟩
    ⟨Estimate the backscattered reflection 286⟩
    ⟨Estimate the scattered transmission 287⟩
    ⟨Debug info for estimate RT 288⟩
}
```

This code is used in section 275.

285. If there are three measurements then the specular reflection can be calculated pretty well. If there are fewer then the unscattered transmission is assumed to be zero. This is not necessarily the case, but after all, this routine only makes estimates of the various reflection and transmission quantities.

If there are three measurements, the optical thickness of the sample is required. Of course if there are three measurements then the illumination must be collimated and we can call *What_Is_B* to find out the optical thickness. We pass this value to a routine in the **fresnel.h** unit and sit back and wait.

All the above is true if sphere corrections are not needed. Now, we just fob this off on another function.

⟨Calculate the unscattered transmission and reflection 285⟩ $\equiv$

```
Calculate_Minimum_MR(m, r, rc, tc);
```

This code is used in section 284.

286. Finding the diffuse reflection is now just a matter of checking whether V1% contains the specular reflection from the sample or not and then just adding or subtracting the specular reflection as appropriate.

```

⟨ Estimate the backscattered reflection 286 ⟩ ≡
  if (m.fraction_of_ru_in_mr) {
    *rt = m.m_r;
    *rd = *rt - m.fraction_of_ru_in_mr * (*rc);
    if (*rd < 0) {
      *rd = 0;
      *rc = *rt;
    }
  }
  else {
    *rd = m.m_r;
    *rt = *rd + *rc;
  }

```

This code is used in section 284.

287. The transmission values follow in much the same way as the diffuse reflection values — just subtract the specular transmission from the total transmission.

```

⟨ Estimate the scattered transmission 287 ⟩ ≡
  if (m.m_u > 0) *tc = m.m_u;
  *td = m.m_t - m.fraction_of_tu_in_mt * (*tc);
  *tt = *td + *tc;

```

This code is used in section 284.

288. Collect debugging info here

```

⟨ Debug info for estimate RT 288 ⟩ ≡
  if (0 ^ Debug(DEBUG_SEARCH)) {
    fprintf(stderr, "SEARCH: r_t=%8.5f", *rt);
    fprintf(stderr, "r_d=%8.5f", *rd);
    fprintf(stderr, "r_u=%8.5f\n", *rc);
    fprintf(stderr, "SEARCH: t_t=%8.5f", *tt);
    fprintf(stderr, "t_d=%8.5f", *td);
    fprintf(stderr, "t_u=%8.5f\n", *tc);
  }

```

This code is used in section 284.

289. Transforming properties. Routines to convert optical properties to calculation space and back.

290. *a2acalc* is used for the albedo transformations according to

$$a_{calc} = \frac{2a - 1}{a(1 - a)}$$

Care is taken to avoid division by zero. Why was this function chosen? Well mostly because it maps the region between $[0, 1] \rightarrow (-\infty, +\infty)$.

⟨Prototype for *a2acalc* 290⟩ $\equiv$
double a2acalc(double a)

This code is used in sections 276 and 291.

291. ⟨Definition for *a2acalc* 291⟩ $\equiv$
 ⟨Prototype for *a2acalc* 290⟩
 {
 if ($a \leq 0$) **return** `-BIG_A_CALC_VALUE`;
 if ($a \geq 1$) **return** `BIG_A_CALC_VALUE`;
 return $((2 * a - 1) / a / (1 - a))$;
 }

This code is used in section 275.

292. *acalc2a* is used for the albedo transformations Now when we solve

$$a_{calc} = \frac{2a - 1}{a(1 - a)}$$

we obtain the quadratic equation

$$a_{calc}a^2 + (2 - a_{calc})a - 1 = 0$$

The only root of this equation between zero and one is

$$a = \frac{-2 + a_{calc} + \sqrt{a_{calc}^2 + 4}}{2a_{calc}}$$

I suppose that I should spend the time to recast this using the more appropriate numerical solutions of the quadratic equation, but this worked and I will leave it as it is for now.

⟨Prototype for *acalc2a* 292⟩ $\equiv$
double acalc2a(double acalc)

This code is used in sections 276 and 293.

293. ⟨Definition for *acalc2a* 293⟩ $\equiv$
 ⟨Prototype for *acalc2a* 292⟩
 {
 if ($acalc \geq \text{BIG_A_CALC_VALUE}$) **return** 1.0;
 if ($acalc \leq -\text{BIG_A_CALC_VALUE}$) **return** 0.0;
 if ($\text{fabs}(acalc) < \text{SMALL_A_CALC_VALUE}$) **return** 0.5;
 return $((-2 + acalc + \text{sqrt}(acalc * acalc + 4)) / (2 * acalc))$;
 }

This code is used in section 275.

294. *g2gcalc* is used for the anisotropy transformations according to

$$g_{calc} = \frac{g}{1 + |g|}$$

which maps $(-1, 1) \rightarrow (-\infty, +\infty)$.

⟨Prototype for *g2gcalc* 294⟩ ≡
double *g2gcalc*(**double** *g*)

This code is used in sections 276 and 295.

295. ⟨Definition for *g2gcalc* 295⟩ ≡
 ⟨Prototype for *g2gcalc* 294⟩
 {
 double *gg* = *g*;
 if (*g* < -MAX_ABS_G) *gg* = -MAX_ABS_G;
 if (*g* > MAX_ABS_G) *gg* = MAX_ABS_G;
 return *gg*/(1 - *fabs*(*gg*));
 }

This code is used in section 275.

296. *gcalc2g* is used for the anisotropy transformations it is the inverse of *g2gcalc*. The relation is

$$g = \frac{g_{calc}}{1 + |g_{calc}|}$$

⟨Prototype for *gcalc2g* 296⟩ ≡
double *gcalc2g*(**double** *gcalc*)

This code is used in sections 276 and 297.

297. ⟨Definition for *gcalc2g* 297⟩ ≡
 ⟨Prototype for *gcalc2g* 296⟩
 {
 return (*gcalc*/(1 + *fabs*(*gcalc*)));
 }

This code is used in section 275.

298. *b2bcalc* is used for the optical depth transformations it is the inverse of *bcalc2b*. The relation is

$$b_{calc} = \ln(b)$$

The only caveats are to ensure that I don't take the logarithm of something big or non-positive.

⟨Prototype for *b2bcalc* 298⟩ ≡
double *b2bcalc*(**double** *b*)

This code is used in sections 276 and 299.

299. $\langle \text{Definition for } b2bcalc \text{ 299} \rangle \equiv$
 $\langle \text{Prototype for } b2bcalc \text{ 298} \rangle$

```
{
  if (b  $\equiv$  HUGE_VAL) return HUGE_VAL;
  if (b  $\leq$  0) return 0.0;
  return (log(b));
}
```

This code is used in section 275.

300. *bcalc2b* is used for the anisotropy transformations it is the inverse of *b2bcalc*. The relation is

$$b = \exp(b_{calc})$$

The only tricky part is to ensure that I don't exponentiate something big and get an overflow error. In ANSI C the maximum value for x such that 10^x is in the range of representable finite floating point numbers (for doubles) is given by `DBL_MAX_10_EXP`. Thus if we want to know if

$$e^{b_{calc}} > 10^x$$

or

$$b_{calc} > x \ln(10) \approx 2.3x$$

and this is the criterion that I use.

$\langle \text{Prototype for } bcalc2b \text{ 300} \rangle \equiv$

```
double bcalc2b(double bcalc)
```

This code is used in sections 276 and 301.

301. $\langle \text{Definition for } bcalc2b \text{ 301} \rangle \equiv$
 $\langle \text{Prototype for } bcalc2b \text{ 300} \rangle$

```
{
  if (bcalc  $\equiv$  HUGE_VAL) return HUGE_VAL;
  if (bcalc > 2.3 * DBL_MAX_10_EXP) return HUGE_VAL;
  return (exp(bcalc));
}
```

This code is used in section 275.

302. *twoprime* converts the true albedo a , optical depth b to the reduced albedo ap and reduced optical depth bp that correspond to $g = 0$.

$\langle \text{Prototype for } twoprime \text{ 302} \rangle \equiv$

```
void twoprime(double a, double b, double g, double *ap, double *bp)
```

This code is used in sections 276 and 303.

303. $\langle \text{Definition for } twoprime \text{ 303} \rangle \equiv$
 $\langle \text{Prototype for } twoprime \text{ 302} \rangle$

```
{
  if (a  $\equiv$  1  $\wedge$  g  $\equiv$  1) *ap = 0.0;
  else *ap = (1 - g) * a / (1 - a * g);
  if (b  $\equiv$  HUGE_VAL) *bp = HUGE_VAL;
  else *bp = (1 - a * g) * b;
}
```

This code is used in section 275.

304. *twounprime* converts the reduced albedo *ap* and reduced optical depth *bp* (for $g = 0$) to the true albedo *a* and optical depth *b* for an anisotropy *g*.

⟨Prototype for *twounprime* 304⟩ ≡

```
void twounprime(double ap, double bp, double g, double *a, double *b)
```

This code is used in sections 276 and 305.

305. ⟨Definition for *twounprime* 305⟩ ≡

⟨Prototype for *twounprime* 304⟩

```
{
    *a = ap / (1 - g + ap * g);
    if (bp == HUGE_VAL) *b = HUGE_VAL;
    else *b = (1 + ap * g / (1 - g)) * bp;
}
```

This code is used in section 275.

306. *abgg2ab* assume *a*, *b*, *g*, and *g1* are given this does the similarity translation that you would expect it should by converting it to the reduced optical properties and then transforming back using the new value of *g*

⟨Prototype for *abgg2ab* 306⟩ ≡

```
void abgg2ab(double a1, double b1, double g1, double g2, double *a2, double *b2)
```

This code is used in sections 276 and 307.

307. ⟨Definition for *abgg2ab* 307⟩ ≡

⟨Prototype for *abgg2ab* 306⟩

```
{
    double a, b;
    twoprime(a1, b1, g1, &a, &b);
    twounprime(a, b, g2, a2, b2);
}
```

This code is used in section 275.

308. *abgb2ag* translates reduced optical properties to unreduced values assuming that the new optical thickness is given i.e., *a1* and *b1* are *a'* and *b'* for $g = 0$. This routine then finds the appropriate anisotropy and albedo which correspond to an optical thickness *b2*.

If both *b1* and *b2* are zero then just assume $g = 0$ for the unreduced values.

⟨Prototype for *abgb2ag* 308⟩ ≡

```
void abgb2ag(double a1, double b1, double b2, double *a2, double *g2)
```

This code is used in sections 276 and 309.

309. $\langle \text{Definition for } abgb2ag \text{ } 309 \rangle \equiv$
 $\langle \text{Prototype for } abgb2ag \text{ } 308 \rangle$
 $\{$
 if $(b1 \equiv 0 \vee b2 \equiv 0) \{$
 $*a2 = a1;$
 $*g2 = 0;$
 $\}$
 if $(b2 < b1) \ b2 = b1;$
 if $(a1 \equiv 0) \ *a2 = 0.0;$
 else if $(a1 \equiv 1) \ *a2 = 1.0;$
 else if $(b1 \equiv 0 \vee b2 \equiv \text{HUGE_VAL}) \ *a2 = a1;$
 else $*a2 = 1 + b1/b2 * (a1 - 1);$
 if $(*a2 \equiv 0 \vee b2 \equiv 0 \vee b2 \equiv \text{HUGE_VAL}) \ *g2 = 0.5;$
 else $*g2 = (1 - b1/b2)/(*a2);$
 $\}$

This code is used in section 275.

310. Guessing an inverse.

This routine is not used anymore.

⟨Prototype for *slow_guess* 310⟩ ≡

```
void slow_guess(struct measure_type m, struct invert_type *r, double *a, double *b, double *g)
```

This code is used in section 311.

311. ⟨Definition for *slow_guess* 311⟩ ≡

⟨Prototype for *slow_guess* 310⟩

```
{
  double fmin = 10.0;
  double fval;
  double *x;
  x = dvector(1, 2);
  switch (r→search) {
  case FIND_A: ⟨Slow guess for a alone 312⟩
    break;
  case FIND_B: ⟨Slow guess for b alone 313⟩
    break;
  case FIND_AB: case FIND_AG: ⟨Slow guess for a and b or a and g 314⟩
    break;
  }
  *a = r→slab.a;
  *b = r→slab.b;
  *g = r→slab.g;
  free_dvector(x, 1, 2);
}
```

312. ⟨Slow guess for *a* alone 312⟩ ≡

```
r→slab.b = HUGE_VAL;
r→slab.g = r→default_g;
Set_Calc_State(m, *r);
for (r→slab.a = 0.0; r→slab.a ≤ 1.0; r→slab.a += 0.1) {
  fval = Find_A_fn(a2acalc(r→slab.a));
  if (fval < fmin) {
    r→a = r→slab.a;
    fmin = fval;
  }
}
r→slab.a = r→a;
```

This code is used in section 311.

313. Presumably the only time that this will need to be called is when the albedo is fixed or is one. For now, I'll just assume that it is one.

```

⟨ Slow guess for b alone 313 ⟩ ≡
  r→slab.a = 1;
  r→slab.g = r→default_g;
  Set_Calc_State(m, *r);
  for (r→slab.b = 1/32.0; r→slab.b ≤ 32; r→slab.b *= 2) {
    fval = Find_B_fn(b2bcalc(r→slab.b));
    if (fval < fmin) {
      r→b = r→slab.b;
      fmin = fval;
    }
  }
  r→slab.b = r→b;

```

This code is used in section 311.

```

314. ⟨ Slow guess for a and b or a and g 314 ⟩ ≡
{
  double min_a, min_b, min_g;
  if (¬ Valid_Grid(m, r→search)) Fill_Grid(m, *r);
  Near_Grid_Points(m.m_r, m.m_t, r→search, &min_a, &min_b, &min_g);
  r→slab.a = min_a;
  r→slab.b = min_b;
  r→slab.g = min_g;
}

```

This code is used in section 311.

```

315. ⟨ Prototype for quick_guess 315 ⟩ ≡
void quick_guess(struct measure_type m, struct invert_type r, double *a, double *b, double *g)

```

This code is used in sections 276 and 316.

```

316. ⟨ Definition for quick_guess 316 ⟩ ≡
⟨ Prototype for quick_guess 315 ⟩
{
  double UR1, UT1, rd, td, tc, rc, bprime, aprime, alpha, beta, logr;
  Estimate_RT(m, r, &UR1, &UT1, &rd, &rc, &td, &tc);
  ⟨ Estimate aprime 317 ⟩
  switch (m.num_measures) {
  case 1: ⟨ Guess when only reflection is known 319 ⟩
    break;
  case 2: ⟨ Guess when reflection and transmission are known 320 ⟩
    break;
  case 3: ⟨ Guess when all three measurements are known 321 ⟩
    break;
  }
  ⟨ Clean up guesses 326 ⟩
}

```

This code is used in section 275.

317. $\langle \text{Estimate } a_{\text{prime}} \text{ 317} \rangle \equiv$
 if (UT1 $\equiv$ 1) $a_{\text{prime}} = 1.0$;
 else if ($rd/(1 - UT1) \geq 0.1$) {
 double $tmp = (1 - rd - UT1)/(1 - UT1)$;
 $a_{\text{prime}} = 1 - 4.0/9.0 * tmp * tmp$;
 }
 else if ($rd < 0.05 \wedge UT1 < 0.4$) $a_{\text{prime}} = 1 - (1 - 10 * rd) * (1 - 10 * rd)$;
 else if ($rd < 0.1 \wedge UT1 < 0.4$) $a_{\text{prime}} = 0.5 + (rd - 0.05) * 4$;
 else {
 double $tmp = (1 - 4 * rd - UT1)/(1 - UT1)$;
 $a_{\text{prime}} = 1 - tmp * tmp$;
 }

This code is used in section 316.

318. $\langle \text{Estimate } b_{\text{prime}} \text{ 318} \rangle \equiv$
 if ($rd < 0.01$) {
 $b_{\text{prime}} = \text{What_Is_B}(r.\text{slab}, UT1)$;
 $fprintf(stderr, "low_rd<0.01!_ut1=\%f_a_{\text{prime}}=\%f_b_{\text{prime}}=\%f\n", UT1, a_{\text{prime}}, b_{\text{prime}})$;
 }
 else if ($UT1 \leq 0$) $b_{\text{prime}} = \text{HUGE_VAL}$;
 else if ($UT1 > 0.1$) $b_{\text{prime}} = 2 * \exp(5 * (rd - UT1) * \log(2.0))$;
 else {
 $\alpha = 1/\log(0.05/1.0)$;
 $\beta = \log(1.0)/\log(0.05/1.0)$;
 $\logr = \log(UR1)$;
 $b_{\text{prime}} = \log(UT1) - \beta * \log(0.05) + \beta * \logr$;
 $b_{\text{prime}} /= \alpha * \log(0.05) - \alpha * \logr - 1$;
 }

This code is used in sections 320, 324, and 325.

319.

$\langle \text{Guess when only reflection is known 319} \rangle \equiv$
 $*g = r.\text{default_g}$;
 $*a = a_{\text{prime}}/(1 - *g + a_{\text{prime}} * (*g))$;
 $*b = \text{HUGE_VAL}$;

This code is used in section 316.

320. $\langle \text{Guess when reflection and transmission are known 320} \rangle \equiv$
 $\langle \text{Estimate } b_{\text{prime}} \text{ 318} \rangle$
 $*g = r.\text{default_g}$;
 $*a = a_{\text{prime}}/(1 - *g + a_{\text{prime}} * *g)$;
 $*b = b_{\text{prime}}/(1 - *a * *g)$;

This code is used in section 316.

321. $\langle \text{Guess when all three measurements are known } 321 \rangle \equiv$
`switch (r.search) {`
`case FIND_A:  $\langle \text{Guess when finding albedo } 322 \rangle$`
`break;`
`case FIND_B:  $\langle \text{Guess when finding optical depth } 323 \rangle$`
`break;`
`case FIND_AB:  $\langle \text{Guess when finding the albedo and optical depth } 324 \rangle$`
`break;`
`case FIND_AG:  $\langle \text{Guess when finding anisotropy and albedo } 325 \rangle$`
`break;`
`}`

This code is used in section 316.

322.

$\langle \text{Guess when finding albedo } 322 \rangle \equiv$
`*g = r.default_g;`
`*a = aprime / (1 - *g + aprime * *g);`
`*b = What_Is_B(r.slabs, m.m_u);`

This code is used in section 321.

323.

$\langle \text{Guess when finding optical depth } 323 \rangle \equiv$
`*g = r.default_g;`
`*a = 0.0;`
`*b = What_Is_B(r.slabs, m.m_u);`

This code is used in section 321.

324.

$\langle \text{Guess when finding the albedo and optical depth } 324 \rangle \equiv$
`*g = r.default_g;`
`if (*g  $\equiv$  1) *a = 0.0;`
`else *a = aprime / (1 - *g + aprime * *g);`
 $\langle \text{Estimate } b_{\text{prime}} 318 \rangle$
`if (bprime  $\equiv$  HUGE_VAL  $\vee$  *a * *g  $\equiv$  1) *b = HUGE_VAL;`
`else *b = bprime / (1 - *a * *g);`

This code is used in section 321.

325.

$\langle \text{Guess when finding anisotropy and albedo } 325 \rangle \equiv$
`*b = What_Is_B(r.slabs, m.m_u);`
`if (*b  $\equiv$  HUGE_VAL  $\vee$  *b  $\equiv$  0) {`
`*a = aprime;`
`*g = r.default_g;`
`}`
`else {`
 $\langle \text{Estimate } b_{\text{prime}} 318 \rangle$
`*a = 1 + bprime * (aprime - 1) / (*b);`
`if (*a < 0.1) *g = 0.0;`
`else *g = (1 - bprime / (*b)) / (*a);`
`}`

This code is used in section 321.

326.

```
⟨ Clean up guesses 326 ⟩ ≡  
  if (*a < 0) *a = 0.0;  
  if (*g < 0) *g = 0.0;  
  else if (*g ≥ 1) *g = 0.5;
```

This code is used in section 316.

327. Some debugging stuff.

328. $\langle$ Prototype for *Set_Debugging* 328 $\rangle \equiv$
void *Set_Debugging*(**unsigned long** *debug_level*)

This code is used in sections 276 and 329.

329.

$\langle$ Definition for *Set_Debugging* 329 $\rangle \equiv$
 $\langle$ Prototype for *Set_Debugging* 328 $\rangle$
 {
 g_util_debugging = *debug_level*;
 }

This code is used in section 275.

330.

$\langle$ Prototype for *Debug* 330 $\rangle \equiv$
int *Debug*(**unsigned long** *mask*)

This code is used in sections 276 and 331.

331.

$\langle$ Definition for *Debug* 331 $\rangle \equiv$
 $\langle$ Prototype for *Debug* 330 $\rangle$
 {
 if (*g_util_debugging* & *mask*) **return** 1;
 else return 0;
 }

This code is used in section 275.

332.

$\langle$ Prototype for *sqr* 332 $\rangle \equiv$
double *sqr*(**double** *x*)

This code is used in sections 276 and 333.

333.

$\langle$ Definition for *sqr* 333 $\rangle \equiv$
 $\langle$ Prototype for *sqr* 332 $\rangle$
 {
 return *x* * *x*;
 }

This code is used in section 275.

334. Just figure out the damn scattering and absorption. This is a nuisance because *b* may be infinite.

$\langle$ Prototype for *Calculate_Mua_Musp* 334 $\rangle \equiv$
void *Calculate_Mua_Musp*(**struct** *measure_type* *m*, **struct** *invert_type* *r*, **double** **mus*, **double** **musp*, **double** **mua*)

This code is used in sections 276 and 335.

335.

<Definition for *Calculate_Mua_Musp* 335> $\equiv$
 <Prototype for *Calculate_Mua_Musp* 334>
 {
 if ($r.b \equiv \text{HUGE_VAL} \vee \text{isinf}(r.b)$) {
 if ($r.a \leq 1 \cdot 10^{-5}$) {
 $*mus = 0.0;$
 $*musp = 0.0;$
 $*mua = 1.0;$
 return;
 }
 if ($r.default_mus \neq \text{UNINITIALIZED}$) {
 $*mus = r.default_mus;$
 $*musp = r.default_mus * (1 - r.g);$
 $*mua = r.default_mus / r.a - r.default_mus;$
 return;
 }
 if ($r.default_mua \neq \text{UNINITIALIZED}$) {
 $*mus = r.default_mua / (1 - r.a) - r.default_mua;$
 $*musp = (*mus) * (1 - r.g);$
 $*mua = r.default_mua;$
 return;
 }
 $*mus = 1.0;$
 $*musp = (*mus) * (1 - r.g);$
 $*mua = (1.0 - r.a) / r.a;$
 return;
 }
 $*mus = r.a * r.b / m.slabs_thickness;$
 $*musp = (*mus) * (1 - r.g);$
 $*mua = (1 - r.a) * r.b / m.slabs_thickness;$
 }

This code is used in section 275.

336.

<Prototype for *Print_Invert_Type* 336> $\equiv$
 void *Print_Invert_Type*(**struct** *invert_type* *r*)

This code is used in sections 276 and 337.

337.

⟨ Definition for *Print_Invert_Type* 337 ⟩ ≡

⟨ Prototype for *Print_Invert_Type* 336 ⟩

```
{
    fprintf(stderr, "\n");
    fprintf(stderr, "default_ua=%10.5f ub=%10.5f uug=%10.5f\n", r.default_a, r.default_b, r.default_g);
    fprintf(stderr, "slab_ua=%10.5f ub=%10.5f uug=%10.5f\n", r.slab.a, r.slab.b, r.slab.g);
    fprintf(stderr, "n_ua_top=%10.5f umid=%10.5f ubot=%10.5f\n", r.slab.n_top_slide, r.slab.n_slab,
        r.slab.n_bottom_slide);
    fprintf(stderr, "thick_ua_top=%10.5f ucos=%10.5f ubot=%10.5f\n", r.slab.b_top_slide, r.slab.cos_angle,
        r.slab.b_bottom_slide);
    fprintf(stderr, "search_ua=%d quadrature_points_ua=%d\n", r.search, r.method.quad_pts);
    fprintf(stderr, "default_a_ua=%10.5f\n", r.default_a);
    fprintf(stderr, "default_b_ua=%10.5f\n", r.default_b);
    fprintf(stderr, "default_g_ua=%10.5f\n", r.default_g);
    fprintf(stderr, "default_mua_ua=%10.5f\n", r.default_mua);
    fprintf(stderr, "default_mus_ua=%10.5f\n", r.default_mus);
}
```

This code is used in section 275.

338.

⟨ Prototype for *Print_Measure_Type* 338 ⟩ ≡

void *Print_Measure_Type*(**struct** *measure_type* *m*)

This code is used in sections 276 and 339.

339.

⟨ Definition for *Print_Measure_Type* 339 ⟩ ≡

⟨ Prototype for *Print_Measure_Type* 338 ⟩

```
{
    fprintf(stderr, "\n");
    fprintf(stderr, "#Beam_diameter=%7.1f\mm\n", m.d_beam);
    fprintf(stderr, "#Sample_thickness=%7.1f\mm\n", m.slab_thickness);
    fprintf(stderr, "#Top_slide_thickness=%7.1f\mm\n",
        m.slab_top_slide_thickness);
    fprintf(stderr, "#Bottom_slide_thickness=%7.1f\mm\n",
        m.slab_bottom_slide_thickness);
    fprintf(stderr, "#Sample_index_of_refraction=%7.3f\n", m.slab_index);
    fprintf(stderr, "#Top_slide_index_of_refraction=%7.3f\n", m.slab_top_slide_index);
    fprintf(stderr, "#Bottom_slide_index_of_refraction=%7.3f\n", m.slab_bottom_slide_index);
    fprintf(stderr, "#Fraction_unscattered_light_in_M_R=%7.1f%%\n",
        m.fraction_of_ru_in_mr * 100);
    fprintf(stderr, "#Fraction_unscattered_light_in_M_T=%7.1f%%\n",
        m.fraction_of_tu_in_mt * 100);
    fprintf(stderr, "#\n");
    fprintf(stderr, "#Reflection_sphere\n");
    fprintf(stderr, "#sphere_diameter=%7.1f\mm\n", m.d_sphere_r);
    fprintf(stderr, "#sample_port_diameter=%7.1f\mm\n",
        2 * m.d_sphere_r * sqrt(m.as_r));
    fprintf(stderr, "#entrance_port_diameter=%7.1f\mm\n",
        2 * m.d_sphere_r * sqrt(m.at_r));
    fprintf(stderr, "#detector_port_diameter=%7.1f\mm\n",
        2 * m.d_sphere_r * sqrt(m.ad_r));
    fprintf(stderr, "#wall_reflectance=%7.1f%%\n", m.rw_r * 100);
    fprintf(stderr, "#standard_reflectance=%7.1f%%\n", m.rstd_r * 100);
    fprintf(stderr, "#detector_reflectance=%7.1f%%\n", m.rd_r * 100);
    fprintf(stderr, "#spheres=%7d\n", m.num_spheres);
    fprintf(stderr, "#measures=%7d\n", m.num_measures);
    fprintf(stderr, "#method=%7d\n", m.method);
    fprintf(stderr, "area_r_as=%10.5f_ad=%10.5f_ae=%10.5f_aw=%10.5f\n", m.as_r, m.ad_r,
        m.at_r, m.aw_r);
    fprintf(stderr, "refls_rd=%10.5f_rw=%10.5f_rstd=%10.5f_rf=%10.5f\n", m.rd_r, m.rw_r,
        m.rstd_r, m.f_r);
    fprintf(stderr, "area_t_as=%10.5f_ad=%10.5f_ae=%10.5f_aw=%10.5f\n", m.as_t, m.ad_t,
        m.at_t, m.aw_t);
    fprintf(stderr, "refls_td=%10.5f_rw=%10.5f_rstd=%10.5f\n", m.rd_t, m.rw_t, m.rstd_t);
    fprintf(stderr, "lost_ur1=%10.5f_ut1=%10.5f_uuru=%10.5f_uutu=%10.5f\n", m.ur1_lost,
        m.ut1_lost, m.utu_lost, m.utu_lost);
}
```

This code is used in section 275.

340. Index. Here is a cross-reference table for the inverse adding-doubling program. All sections in which an identifier is used are listed with that identifier, except that reserved words are indexed only when they appear in format definitions, and the appearances of identifiers in section names are not indexed. Underlined entries correspond to where the identifier was declared. Error messages and a few other things like “ASCII code dependencies” are indexed here too.

`_CRT_NONSTDC_NO_WARNINGS`: [3](#).
`_CRT_SECURE_NO_WARNINGS`: [3](#), [91](#).
`a`: [30](#), [38](#), [39](#), [66](#), [83](#), [154](#), [290](#), [302](#), [304](#), [307](#), [310](#), [315](#).
`a_calc`: [65](#).
`A_COLUMN`: [123](#), [147](#), [160](#), [181](#).
`aa`: [11](#).
`abg_distance`: [154](#), [237](#).
`abgb2ag`: [308](#).
`abgg2ab`: [306](#).
`ABIT`: [123](#), [204](#), [205](#).
`ABSOLUTE`: [35](#), [40](#), [63](#).
`Absorbing-Glass-RT`: [279](#).
`acalc`: [292](#), [293](#).
`acalc2a`: [208](#), [210](#), [216](#), [240](#), [248](#), [258](#), [292](#).
`acos`: [115](#).
`AD_error`: [145](#), [177](#).
`AD_iterations`: [25](#), [38](#), [46](#), [47](#), [63](#), [235](#), [244](#), [246](#), [248](#), [250](#), [253](#), [255](#), [260](#), [266](#), [271](#).
`AD_method_type`: [38](#).
`ad_r`: [18](#), [37](#), [54](#), [69](#), [72](#), [75](#), [87](#), [96](#), [113](#), [128](#), [134](#), [339](#).
`AD_slab_type`: [38](#), [158](#), [231](#), [277](#).
`ad_t`: [18](#), [37](#), [55](#), [69](#), [72](#), [76](#), [88](#), [97](#), [114](#), [128](#), [136](#), [339](#).
`Allocate_Grid`: [144](#), [162](#), [168](#), [171](#), [173](#), [175](#).
`alpha`: [316](#), [318](#).
`amoeba`: [235](#), [255](#), [260](#), [266](#), [271](#).
`analysis`: [70](#), [73](#), [83](#), [89](#).
`any_error`: [2](#), [4](#), [13](#).
`ap`: [302](#), [303](#), [304](#), [305](#).
`aprime`: [11](#), [231](#), [316](#), [317](#), [318](#), [319](#), [320](#), [322](#), [324](#), [325](#).
`argc`: [2](#), [5](#), [6](#), [12](#).
`argv`: [2](#), [5](#), [6](#), [12](#).
`as_r`: [18](#), [19](#), [37](#), [54](#), [69](#), [72](#), [75](#), [87](#), [96](#), [113](#), [128](#), [130](#), [132](#), [136](#), [339](#).
`as_t`: [18](#), [37](#), [55](#), [69](#), [72](#), [76](#), [88](#), [97](#), [114](#), [128](#), [130](#), [132](#), [134](#), [339](#).
`at_r`: [18](#), [37](#), [54](#), [69](#), [72](#), [75](#), [87](#), [96](#), [113](#), [128](#), [130](#), [132](#), [134](#), [136](#), [193](#), [339](#).
`at_t`: [18](#), [37](#), [55](#), [69](#), [72](#), [76](#), [88](#), [97](#), [114](#), [128](#), [130](#), [132](#), [134](#), [136](#), [198](#), [339](#).
`aw_r`: [18](#), [37](#), [69](#), [72](#), [87](#), [96](#), [128](#), [130](#), [132](#), [339](#).
`aw_t`: [18](#), [37](#), [69](#), [72](#), [88](#), [97](#), [128](#), [130](#), [132](#), [339](#).
`ax`: [244](#), [246](#), [248](#), [250](#), [253](#).
`a1`: [306](#), [307](#), [308](#), [309](#).
`a2`: [306](#), [307](#), [308](#), [309](#).
`a2acalc`: [238](#), [248](#), [256](#), [290](#), [312](#).
`B`: [282](#).
`b`: [38](#), [39](#), [66](#), [83](#), [154](#), [181](#), [298](#), [302](#), [304](#), [307](#), [310](#), [315](#).
`b_bottom_slide`: [52](#), [65](#), [143](#), [159](#), [179](#), [181](#), [279](#), [337](#).
`b_calc`: [65](#).
`B_COLUMN`: [123](#), [147](#), [160](#), [181](#).
`b_thinnest`: [65](#).
`b_top_slide`: [52](#), [65](#), [143](#), [159](#), [179](#), [181](#), [279](#), [337](#).
`ba`: [173](#), [175](#), [212](#), [213](#), [214](#), [243](#), [246](#).
`baffle_r`: [18](#), [37](#), [69](#), [113](#), [128](#), [193](#).
`baffle_t`: [18](#), [37](#), [69](#), [114](#), [128](#), [198](#).
`base_name`: [12](#).
`bb`: [11](#).
`bcalc`: [300](#), [301](#).
`bcalc2b`: [210](#), [212](#), [214](#), [218](#), [222](#), [224](#), [226](#), [240](#), [244](#), [246](#), [253](#), [264](#), [269](#), [274](#), [298](#), [300](#).
`beta`: [316](#), [318](#).
`BIG_A_CALC_VALUE`: [35](#), [291](#), [293](#).
`boolean_type`: [39](#), [123](#), [142](#), [148](#).
`both`: [27](#).
`boundary_method`: [231](#).
`bp`: [302](#), [303](#), [304](#), [305](#).
`bprime`: [11](#), [231](#), [316](#), [318](#), [320](#), [324](#), [325](#).
`brent`: [244](#), [246](#), [248](#), [250](#), [253](#).
`bs`: [173](#), [175](#), [212](#), [214](#), [244](#), [245](#).
`bx`: [244](#), [246](#), [248](#), [250](#), [253](#).
`b1`: [306](#), [307](#), [308](#), [309](#).
`b2`: [306](#), [307](#), [308](#), [309](#).
`b2bcalc`: [238](#), [244](#), [246](#), [253](#), [262](#), [267](#), [272](#), [298](#), [300](#), [313](#).
`c`: [4](#), [101](#), [103](#), [107](#), [116](#), [120](#), [122](#).
`calculate_coefficients`: [13](#), [17](#), [23](#).
`Calculate_Distance`: [23](#), [78](#), [82](#), [155](#), [161](#), [178](#), [208](#), [210](#), [212](#), [214](#), [216](#), [218](#), [220](#), [222](#), [224](#), [226](#), [228](#).
`Calculate_Distance_With_Corrections`: [156](#), [179](#), [181](#), [182](#).
`Calculate_Grid_Distance`: [147](#), [157](#), [180](#).
`Calculate_Minimum_MR`: [51](#), [79](#), [285](#).
`Calculate_MR_MT`: [10](#), [11](#), [47](#), [77](#), [80](#).
`Calculate_Mua_Musp`: [10](#), [23](#), [47](#), [334](#).
`CALCULATING_GRID`: [123](#), [157](#), [160](#), [179](#), [181](#), [193](#), [198](#), [206](#).
`cc`: [6](#).
`check_magic`: [95](#), [106](#).
`cl_baffle_r`: [4](#), [6](#), [18](#).

- cl_baffle_t*: [4](#), [6](#), [18](#).
- cl_beam_d*: [4](#), [6](#), [18](#).
- cl_cos_angle*: [4](#), [6](#), [18](#).
- cl_default_a*: [4](#), [6](#), [8](#), [15](#).
- cl_default_b*: [4](#), [6](#), [8](#), [9](#), [15](#).
- cl_default_fr*: [4](#), [6](#), [18](#).
- cl_default_g*: [4](#), [6](#), [7](#), [15](#).
- cl_default_mua*: [4](#), [6](#), [7](#), [15](#).
- cl_default_mus*: [4](#), [6](#), [7](#), [15](#).
- cl_default_musp*: [4](#), [6](#), [7](#), [15](#).
- cl_forward_calc*: [2](#), [4](#), [6](#).
- cl_grid_calc*: [4](#), [6](#), [11](#).
- cl_lambda*: [4](#), [6](#), [18](#).
- cl_method*: [4](#), [6](#), [18](#), [19](#).
- cl_musp0*: [4](#), [6](#), [15](#).
- cl_mus0*: [4](#), [6](#), [15](#).
- cl_mus0_lambda*: [4](#), [6](#), [15](#).
- cl_mus0_pwr*: [4](#), [6](#), [15](#).
- cl_num_spheres*: [4](#), [6](#), [18](#).
- cl_quadrature_points*: [4](#), [6](#), [15](#), [18](#).
- cl_rstd_r*: [4](#), [6](#), [18](#).
- cl_rstd_t*: [4](#), [6](#), [18](#).
- cl_ru_fraction*: [4](#), [6](#), [18](#).
- cl_rwall_r*: [4](#), [6](#), [18](#).
- cl_rwall_t*: [4](#), [6](#), [18](#).
- cl_sample_d*: [4](#), [6](#), [8](#), [9](#), [15](#), [18](#).
- cl_sample_n*: [4](#), [6](#), [18](#).
- cl_search*: [4](#), [6](#), [15](#).
- cl_slide_d*: [4](#), [6](#), [18](#).
- cl_slide_n*: [4](#), [6](#), [18](#).
- cl_slide_OD*: [4](#), [6](#), [18](#).
- cl_slides*: [4](#), [6](#), [18](#).
- cl_sphere_one*: [4](#), [6](#), [18](#).
- cl_sphere_two*: [4](#), [6](#), [18](#).
- cl_Tc*: [4](#), [6](#), [18](#).
- cl_tolerance*: [4](#), [6](#), [15](#).
- cl_tu_fraction*: [4](#), [6](#), [18](#).
- cl_UR1*: [4](#), [6](#), [18](#).
- cl_UT1*: [4](#), [6](#), [18](#).
- cl_verbosity*: [2](#), [4](#), [6](#), [10](#), [13](#), [16](#), [17](#).
- cl_wave_limit*: [4](#), [6](#), [13](#).
- clock*: [2](#), [4](#), [29](#).
- CLOCKS_PER_SEC: [29](#).
- cmd*: [108](#), [110](#).
- COLLIMATED: [35](#).
- collimated*: [231](#).
- COLUMN_LABELS: [91](#), [98](#), [100](#), [101](#), [115](#), [120](#), [122](#).
- command_line*: [4](#), [5](#), [10](#), [11](#), [16](#).
- command_line_length*: [5](#).
- command_line_options*: [4](#), [6](#).
- compare_guesses*: [232](#), [237](#).
- COMPARISON: [6](#), [19](#), [36](#), [115](#), [183](#).
- compute_R_and_T*: [231](#).
- constraints*: [57](#).
- correct_URU*: [159](#).
- correct_UR1*: [159](#).
- cos*: [6](#).
- cos_angle*: [52](#), [65](#), [143](#), [179](#), [181](#), [235](#), [244](#), [246](#), [248](#), [250](#), [252](#), [255](#), [260](#), [271](#), [279](#), [281](#), [282](#), [337](#).
- Cos_Snell*: [279](#).
- count*: [11](#), [101](#), [237](#).
- counter*: [33](#).
- current_position*: [116](#).
- cx*: [244](#), [246](#), [248](#), [250](#), [253](#).
- d_beam*: [18](#), [37](#), [69](#), [72](#), [86](#), [95](#), [101](#), [110](#), [339](#).
- d_detector_r*: [18](#), [72](#), [87](#), [96](#).
- d_detector_t*: [18](#), [72](#), [88](#), [97](#).
- d_entrance_r*: [72](#), [87](#).
- d_entrance_t*: [72](#), [88](#).
- d_sample_r*: [18](#), [72](#), [87](#), [96](#).
- d_sample_t*: [18](#), [72](#), [88](#), [97](#).
- d_sphere_r*: [18](#), [19](#), [37](#), [69](#), [72](#), [87](#), [96](#), [113](#), [114](#), [339](#).
- d_sphere_t*: [18](#), [37](#), [69](#), [72](#), [88](#), [97](#), [114](#).
- d_third_r*: [18](#), [96](#).
- d_third_t*: [18](#), [97](#).
- DBL_MAX_10_EXP: [300](#), [301](#).
- DE_RT: [231](#).
- Debug*: [13](#), [17](#), [24](#), [25](#), [26](#), [33](#), [46](#), [47](#), [57](#), [82](#), [150](#), [151](#), [152](#), [153](#), [157](#), [160](#), [162](#), [168](#), [171](#), [173](#), [175](#), [179](#), [181](#), [193](#), [198](#), [206](#), [230](#), [235](#), [237](#), [238](#), [244](#), [246](#), [248](#), [250](#), [252](#), [255](#), [256](#), [260](#), [262](#), [271](#), [288](#), [330](#).
- DEBUG_A_LITTLE: [17](#), [36](#), [47](#).
- DEBUG_ANY: [13](#), [26](#), [33](#), [36](#).
- DEBUG_BEST_GUESS: [36](#), [237](#), [238](#), [256](#), [262](#).
- DEBUG_EVERY_CALC: [36](#), [160](#).
- DEBUG_GRID: [36](#), [150](#), [151](#), [152](#), [153](#), [157](#), [162](#), [168](#), [171](#), [173](#), [175](#).
- DEBUG_GRID_CALC: [36](#), [160](#), [179](#), [181](#), [206](#).
- DEBUG_ITERATIONS: [17](#), [36](#), [46](#), [82](#), [179](#), [206](#).
- debug_level*: [328](#), [329](#).
- DEBUG_LOST_LIGHT: [17](#), [24](#), [25](#), [36](#), [230](#).
- DEBUG_SEARCH: [36](#), [57](#), [235](#), [244](#), [246](#), [248](#), [250](#), [252](#), [255](#), [260](#), [271](#), [288](#).
- DEBUG_SPHERE_GAIN: [36](#), [193](#), [198](#).
- default_a*: [15](#), [38](#), [44](#), [51](#), [57](#), [58](#), [59](#), [64](#), [82](#), [101](#), [115](#), [171](#), [205](#), [222](#), [250](#), [252](#), [260](#), [337](#).
- default_b*: [15](#), [38](#), [57](#), [58](#), [59](#), [64](#), [79](#), [80](#), [101](#), [115](#), [248](#), [250](#), [255](#), [337](#).
- default_ba*: [15](#), [38](#), [58](#), [59](#), [64](#), [101](#), [115](#), [153](#), [175](#), [226](#), [244](#), [271](#), [272](#), [274](#).
- default_bs*: [15](#), [38](#), [58](#), [59](#), [64](#), [101](#), [115](#), [153](#), [173](#), [224](#), [246](#), [267](#), [269](#).
- default_detector_d*: [69](#).

- default_entrance_d*: [69](#).
default_g: [11](#), [15](#), [38](#), [57](#), [59](#), [60](#), [64](#), [72](#), [80](#), [101](#),
[115](#), [235](#), [244](#), [246](#), [248](#), [252](#), [312](#), [313](#), [319](#),
[320](#), [322](#), [323](#), [324](#), [325](#), [337](#).
default_mua: [15](#), [38](#), [57](#), [64](#), [101](#), [335](#), [337](#).
default_mus: [15](#), [38](#), [57](#), [64](#), [101](#), [335](#), [337](#).
default_sample_d: [69](#).
default_sphere_d: [69](#).
delta: [23](#).
depth: [231](#).
determine_search: [44](#), [56](#).
dev: [181](#), [182](#), [204](#), [205](#), [206](#).
deviation: [178](#), [179](#), [208](#), [210](#), [212](#), [214](#), [216](#), [218](#),
[220](#), [222](#), [224](#), [226](#), [228](#).
DIFFUSE: [35](#).
distance: [39](#), [78](#), [82](#), [147](#), [155](#), [232](#), [237](#), [238](#),
[256](#), [262](#).
dmatrix: [145](#), [236](#).
dvector: [236](#), [311](#).
Egan: [231](#).
endptr: [28](#).
EOF: [6](#), [116](#).
ERANGE: [28](#).
err: [31](#), [32](#), [33](#).
errno: [28](#).
error: [38](#), [50](#), [63](#), [66](#), [67](#), [71](#), [84](#).
Estimate_RT: [57](#), [248](#), [250](#), [252](#), [283](#), [316](#).
Exact_coll_flag: [231](#).
exit: [2](#), [6](#), [11](#), [12](#), [18](#), [19](#), [28](#), [30](#), [98](#), [122](#), [183](#).
EXIT_FAILURE: [2](#), [6](#), [11](#), [12](#), [28](#), [30](#), [98](#), [122](#), [183](#).
EXIT_SUCCESS: [2](#), [6](#), [18](#), [19](#).
exp: [162](#), [171](#), [173](#), [175](#), [301](#), [318](#).
ez_Inverse_RT: [66](#).
f: [227](#), [231](#).
f_r: [18](#), [19](#), [37](#), [54](#), [69](#), [75](#), [134](#), [136](#), [193](#), [202](#), [339](#).
f_t: [202](#).
fa: [244](#), [246](#), [248](#), [250](#), [253](#).
fabs: [17](#), [204](#), [205](#), [293](#), [295](#), [297](#).
FALSE: [13](#), [14](#), [17](#), [35](#), [46](#), [47](#), [63](#), [80](#), [101](#), [123](#),
[143](#), [145](#), [150](#), [151](#), [152](#), [153](#), [231](#).
fb: [244](#), [246](#), [248](#), [250](#), [253](#).
fc: [244](#), [246](#), [248](#), [250](#), [253](#).
fclose: [11](#).
feof: [103](#), [107](#).
fflush: [25](#), [33](#).
fgetc: [103](#), [107](#), [116](#).
fgets: [122](#).
file: [116](#).
Fill_AB_Grid: [161](#), [167](#), [170](#), [177](#).
Fill_AG_Grid: [167](#), [177](#).
Fill_BaG_Grid: [172](#), [177](#).
Fill_BG_Grid: [170](#), [172](#), [177](#).
Fill_BsG_Grid: [174](#), [177](#).
Fill_Grid: [176](#), [237](#), [314](#).
fill_grid_entry: [160](#), [162](#), [168](#), [171](#), [173](#), [175](#).
final: [33](#).
final_distance: [17](#), [38](#), [47](#), [63](#), [240](#), [241](#), [244](#), [246](#),
[248](#), [250](#), [252](#), [253](#), [258](#), [264](#), [269](#), [274](#).
FIND_A: [35](#), [46](#), [51](#), [57](#), [58](#), [59](#), [115](#), [203](#), [311](#), [321](#).
Find_A_fn: [215](#), [248](#), [312](#).
FIND_AB: [35](#), [46](#), [57](#), [59](#), [115](#), [153](#), [162](#), [177](#),
[311](#), [321](#).
Find_AB_fn: [209](#), [235](#), [239](#).
FIND_AG: [35](#), [46](#), [57](#), [59](#), [115](#), [153](#), [167](#), [168](#),
[177](#), [311](#), [321](#).
Find_AG_fn: [207](#), [255](#), [257](#).
FIND_AUTO: [35](#), [44](#), [57](#), [63](#), [115](#).
FIND_B: [35](#), [44](#), [46](#), [51](#), [57](#), [58](#), [59](#), [82](#), [115](#),
[203](#), [311](#), [321](#).
Find_B_fn: [217](#), [253](#), [313](#).
FIND_B_WITH_NO_ABSORPTION: [35](#), [44](#), [57](#), [58](#), [59](#).
FIND_B_WITH_NO_SCATTERING: [35](#), [44](#), [57](#), [58](#).
FIND_Ba: [35](#), [46](#), [51](#), [57](#), [58](#), [59](#), [115](#), [203](#).
Find_Ba_fn: [211](#), [213](#), [245](#), [246](#).
FIND_BaG: [35](#), [46](#), [57](#), [59](#), [153](#), [173](#), [177](#).
Find_BaG_fn: [223](#), [266](#), [268](#).
FIND_BG: [35](#), [46](#), [57](#), [59](#), [171](#), [177](#).
Find_BG_fn: [221](#), [260](#), [263](#).
FIND_Bs: [35](#), [46](#), [51](#), [57](#), [58](#), [59](#), [115](#), [203](#).
Find_Bs_fn: [213](#), [243](#), [244](#).
FIND_BsG: [35](#), [46](#), [57](#), [59](#), [153](#), [175](#), [177](#).
Find_BsG_fn: [225](#), [271](#), [273](#).
FIND_G: [35](#), [46](#), [51](#), [57](#), [58](#), [59](#), [203](#).
Find_G_fn: [219](#), [250](#).
finish_time: [29](#).
flip: [158](#), [159](#).
flip_sample: [18](#), [37](#), [52](#), [69](#), [115](#), [160](#), [179](#), [181](#).
fmin: [311](#), [312](#), [313](#).
fopen: [11](#).
force_new: [176](#), [177](#).
format: [120](#).
found: [11](#), [17](#), [23](#), [38](#), [46](#), [63](#), [241](#).
fp: [24](#), [25](#), [94](#), [95](#), [96](#), [97](#), [98](#), [99](#), [100](#), [101](#), [102](#),
[103](#), [104](#), [105](#), [106](#), [107](#), [121](#), [122](#).
fprintf: [2](#), [5](#), [6](#), [11](#), [12](#), [13](#), [16](#), [17](#), [18](#), [19](#), [21](#),
[22](#), [24](#), [25](#), [26](#), [27](#), [28](#), [30](#), [32](#), [33](#), [46](#), [47](#), [52](#),
[57](#), [72](#), [82](#), [98](#), [101](#), [105](#), [107](#), [122](#), [150](#), [151](#),
[152](#), [153](#), [157](#), [160](#), [162](#), [168](#), [171](#), [173](#), [175](#),
[179](#), [181](#), [183](#), [193](#), [198](#), [206](#), [230](#), [235](#), [237](#),
[238](#), [244](#), [246](#), [248](#), [250](#), [252](#), [255](#), [256](#), [260](#),
[262](#), [271](#), [288](#), [318](#), [337](#), [339](#).
frac: [230](#).
FRACTION: [39](#).

- fraction_of_ru_in_mr*: 18, [37](#), 69, 101, 112, 185, 193, 286, 339.
fraction_of_tu_in_mt: 18, [37](#), 69, 101, 112, 185, 198, 287, 339.
free: 12.
free_dmatrix: 242.
free_dvector: 242, 311.
freopen: 12.
fscanf: 105.
fseek: 116.
ftell: 116.
fval: [157](#), [311](#), 312, 313.
FO: [231](#).
G: [130](#), [132](#), [136](#), [193](#).
g: [11](#), [38](#), [39](#), [66](#), [83](#), [154](#), [294](#), [302](#), [304](#), [310](#), [315](#).
g_calc: 65.
G_COLUMN: [123](#), 147, 160, 181.
g_grid_name: [4](#), 11, 12.
g_out_name: [4](#), 6, 11, 12.
G_std: [193](#), 198.
g_util_debugging: [275](#), 329, 331.
G_0: [193](#).
Gain: [127](#), 130, 132, 134, 136, 193, 198.
Gain_11: [129](#), 133, 134.
Gain_22: [131](#), 136.
gcalc: [296](#), 297.
gcalc2g: 208, 220, 222, 224, 226, 250, 258, 264, 269, 274, [296](#).
Get_Calc_State: 78, [140](#), 155, 157, 177, 228, 230.
get_current_line_number: 105, [116](#).
getopt: 5, 6.
gg: [295](#).
GG_a: [123](#), 169, 171.
GG_b: [123](#), 168, 169.
GG_ba: [123](#), 169, 175.
GG_bs: [123](#), 169, 173.
GG_g: [123](#), 162, 169.
GP: [130](#), [132](#), [134](#).
gprime: [231](#).
grid: [11](#).
Grid_ABG: [146](#), 237.
GRID_SIZE: [35](#), 145, 147, 157, 160, 162, 164, 165, 166, 168, 171, 173, 175, 181.
guess: [146](#), 147, [154](#), 155, [232](#), 237, 238, 256, 262, 267, 272.
guess_t: 39.
guess_type: 39, 146, 154, 232, 237.
g1: [232](#), [306](#), 307.
G11: [130](#).
g2: [232](#), [306](#), 307, [308](#), 309.
g2gcalc: 250, 256, 262, 267, 272, [294](#), 296.
G22: [132](#).
HENY_ GREENSTEIN: 65.
HUGE_VAL: 9, 67, 80, 115, 244, 246, 248, 250, 280, 299, 301, 303, 305, 309, 312, 318, 319, 324, 325, 335.
i: [5](#), [11](#), [30](#), [71](#), [84](#), [107](#), [115](#), [117](#), [118](#), [119](#), [146](#), [157](#), [160](#), [162](#), [168](#), [171](#), [173](#), [175](#), [180](#), [236](#).
i_best: [236](#), 237.
i_min: [156](#), 157.
IAD_AD_NOT_VALID: [36](#), 54, 55.
IAD_AE_NOT_VALID: [36](#), 54, 55.
IAD_AS_NOT_VALID: [36](#), 54, 55.
IAD_BAD_G_VALUE: [36](#).
IAD_BAD_PHASE_FUNCTION: [36](#).
IAD_EXCESSIVE_LIGHT_LOSS: [36](#).
IAD_F_NOT_VALID: [36](#), 54.
IAD_FILE_ERROR: [36](#).
IAD_GAMMA_NOT_VALID: [36](#).
IAD_invert_type: [38](#).
IAD_MAX_ITERATIONS: [35](#), 46.
IAD_measure_type: [37](#).
IAD_MEMORY_ERROR: [36](#).
IAD_MR_TOO_BIG: 31, 32, [36](#), 51, 52, 82.
IAD_MR_TOO_SMALL: 31, 32, [36](#), 51, 82.
IAD_MT_TOO_BIG: 31, 32, [36](#), 52.
IAD_MT_TOO_SMALL: 31, 32, [36](#), 52, 82.
IAD_MU_TOO_BIG: 31, 32, [36](#), 53.
IAD_MU_TOO_SMALL: 31, 32, [36](#), 53.
IAD_NO_ERROR: 13, 31, 32, [36](#), 45, 50, 63, 67, 71, 82, 84.
IAD_QUAD_PTS_NOT_VALID: [36](#), 45.
IAD_RD_NOT_VALID: [36](#), 54, 55.
IAD_RSTD_NOT_VALID: [36](#), 54.
IAD_RT_LT_MINIMUM: [36](#).
IAD_RW_NOT_VALID: [36](#), 54, 55.
IAD_TOO_MANY_ITERATIONS: 23, 31, 32, [36](#), 46.
IAD_TOO_MANY_LAYERS: [36](#).
IAD_TOO_MUCH_LIGHT: 31, 32, [36](#).
IAD_TSTD_NOT_VALID: [36](#), 54, 55.
illumination: 83, 86, 231.
illumination_type: [39](#).
include_MC: [77](#), 78.
include_spheres: [77](#), 78.
independent: 57.
Initialize_Measure: 2, 67, [68](#), 71, 84, 95.
Initialize_Result: 2, 13, [60](#), 67, 71, 84.
inv_gain: [128](#).
Inverse_RT: 13, 17, 23, 40, [43](#), 66, 67, 71, 84.
invert_type: 4, 23, 25, [38](#), 43, 49, 56, 60, 67, 71, 77, 78, 79, 81, 84, 99, 101, 108, 123, 138, 139, 140, 141, 142, 148, 155, 157, 161, 167, 170, 172, 174, 176, 228, 229, 230, 234, 243, 245, 247, 249, 251, 254, 259, 265, 270, 283, 310, 315, 334, 336.

- isdigit*: [6](#).
- isinf*: [335](#).
- isspace*: [103](#), [117](#).
- j*: [11](#), [117](#), [146](#), [157](#), [160](#), [162](#), [168](#), [171](#), [173](#), [175](#), [180](#).
- j_best*: [236](#), [237](#).
- j_min*: [156](#), [157](#).
- k*: [237](#), [238](#), [256](#), [262](#).
- kk*: [238](#), [256](#), [262](#).
- lambda*: [13](#), [15](#), [18](#), [25](#), [37](#), [69](#), [100](#), [101](#).
- last*: [30](#).
- last_final_distance*: [17](#).
- last_mu_a*: [17](#).
- last_mu_sp*: [17](#).
- len*: [119](#).
- line*: [25](#).
- line_no*: [105](#).
- line_number*: [116](#).
- log*: [281](#), [282](#), [299](#), [318](#).
- logr*: [316](#), [318](#).
- LR*: [13](#), [14](#), [17](#), [23](#), [25](#), [181](#).
- LT*: [13](#), [14](#), [17](#), [23](#), [25](#), [181](#).
- m*: [4](#), [23](#), [25](#), [43](#), [49](#), [56](#), [60](#), [67](#), [68](#), [71](#), [77](#), [79](#), [81](#), [84](#), [94](#), [99](#), [101](#), [108](#), [127](#), [129](#), [131](#), [133](#), [135](#), [138](#), [140](#), [142](#), [148](#), [161](#), [167](#), [170](#), [172](#), [174](#), [176](#), [229](#), [234](#), [243](#), [245](#), [247](#), [249](#), [251](#), [254](#), [259](#), [265](#), [270](#), [283](#), [310](#), [315](#), [334](#), [338](#).
- m_old*: [228](#), [230](#).
- M_PI*: [6](#), [69](#), [231](#).
- M_R*: [47](#), [77](#), [78](#), [79](#), [178](#), [179](#), [182](#), [184](#), [186](#), [193](#), [200](#), [202](#), [204](#), [205](#), [206](#).
- m_r*: [10](#), [11](#), [13](#), [18](#), [20](#), [25](#), [37](#), [47](#), [51](#), [57](#), [67](#), [69](#), [74](#), [82](#), [90](#), [100](#), [101](#), [155](#), [160](#), [204](#), [205](#), [206](#), [208](#), [210](#), [212](#), [214](#), [216](#), [218](#), [220](#), [222](#), [224](#), [226](#), [227](#), [228](#), [237](#), [286](#), [314](#).
- M_T*: [47](#), [53](#), [77](#), [78](#), [178](#), [179](#), [182](#), [184](#), [186](#), [198](#), [200](#), [202](#), [204](#), [205](#), [206](#).
- m_t*: [10](#), [11](#), [13](#), [18](#), [20](#), [25](#), [37](#), [47](#), [51](#), [52](#), [53](#), [57](#), [58](#), [67](#), [69](#), [74](#), [82](#), [90](#), [100](#), [101](#), [155](#), [160](#), [204](#), [205](#), [206](#), [208](#), [210](#), [212](#), [214](#), [216](#), [218](#), [220](#), [222](#), [224](#), [226](#), [227](#), [228](#), [237](#), [244](#), [246](#), [287](#), [314](#).
- m_u*: [18](#), [20](#), [37](#), [53](#), [57](#), [67](#), [69](#), [74](#), [79](#), [80](#), [90](#), [100](#), [101](#), [143](#), [152](#), [167](#), [250](#), [255](#), [287](#), [322](#), [323](#), [325](#).
- magic*: [107](#).
- main*: [2](#).
- malloc*: [5](#), [27](#).
- mask*: [330](#), [331](#).
- max_a*: [168](#), [173](#), [175](#).
- MAX_ABS_G*: [6](#), [35](#), [165](#), [166](#), [168](#), [171](#), [173](#), [175](#), [295](#).
- MAX_COLUMNS*: [91](#), [122](#).
- Max_Light_Loss*: [229](#).
- max_log_b*: [162](#), [171](#).
- max_log_ba*: [173](#).
- max_log_bs*: [175](#).
- MAX_MC_iterations*: [4](#), [6](#), [10](#), [11](#), [16](#), [17](#).
- max_possible_m_r*: [82](#).
- maxloss*: [227](#), [230](#).
- MC_iterations*: [17](#), [25](#), [38](#), [47](#), [63](#).
- MC_Lost*: [17](#), [71](#), [78](#), [84](#).
- MC_NONE*: [10](#), [11](#), [36](#), [47](#), [78](#).
- MC_REDO*: [10](#), [11](#), [36](#), [78](#).
- mc_runs*: [71](#), [73](#), [84](#), [89](#).
- MC_tolerance*: [15](#), [17](#), [38](#), [63](#), [101](#), [115](#).
- mc_total*: [13](#), [14](#), [17](#).
- MC_USE_EXISTING*: [36](#), [47](#).
- measure_OK*: [45](#), [49](#).
- measure_type**: [4](#), [23](#), [25](#), [37](#), [43](#), [49](#), [56](#), [60](#), [67](#), [68](#), [71](#), [77](#), [78](#), [79](#), [81](#), [84](#), [94](#), [99](#), [101](#), [108](#), [123](#), [127](#), [129](#), [131](#), [133](#), [135](#), [138](#), [139](#), [140](#), [141](#), [142](#), [148](#), [155](#), [157](#), [161](#), [167](#), [170](#), [172](#), [174](#), [176](#), [228](#), [229](#), [230](#), [234](#), [243](#), [245](#), [247](#), [249](#), [251](#), [254](#), [259](#), [265](#), [270](#), [283](#), [310](#), [315](#), [334](#), [338](#).
- measured_m_r*: [82](#).
- measurement*: [83](#), [90](#).
- measurements*: [70](#), [74](#).
- memcpy*: [139](#), [141](#).
- method*: [15](#), [18](#), [37](#), [38](#), [45](#), [60](#), [65](#), [67](#), [69](#), [73](#), [89](#), [95](#), [101](#), [115](#), [143](#), [160](#), [179](#), [183](#), [337](#), [339](#).
- metric*: [38](#), [46](#), [63](#), [204](#), [205](#).
- MGRID*: [123](#), [143](#), [152](#), [153](#), [177](#).
- min_a*: [168](#), [173](#), [175](#), [314](#).
- min_b*: [314](#).
- min_g*: [314](#).
- min_log_b*: [162](#), [171](#).
- min_log_ba*: [173](#).
- min_log_bs*: [175](#).
- min_possible_m_r*: [82](#).
- MinMax_MR_MT*: [50](#), [81](#).
- MM*: [123](#), [125](#), [138](#), [139](#), [141](#), [160](#), [178](#), [179](#), [181](#), [183](#), [184](#), [185](#), [186](#), [193](#), [198](#), [202](#), [204](#), [205](#), [206](#), [228](#), [231](#).
- mnbrak*: [244](#), [246](#), [248](#), [250](#), [253](#).
- mr*: [51](#), [79](#), [80](#).
- MR_IS_ONLY_RD*: [3](#).
- mt*: [51](#), [79](#), [80](#).
- MT_IS_ONLY_TD*: [3](#).
- mu_a*: [10](#), [13](#), [14](#), [17](#), [25](#), [51](#).
- mu_in_slab*: [278](#), [279](#).
- mu_s*: [10](#).
- mu_sp*: [10](#), [13](#), [14](#), [17](#), [25](#).
- mua*: [23](#), [47](#), [334](#), [335](#).
- mus*: [23](#), [47](#), [334](#), [335](#).
- musp*: [23](#), [47](#), [334](#), [335](#).

- my_strtod*: [6](#), [28](#).
n: [6](#), [12](#), [30](#), [66](#), [122](#), [158](#).
n_bottom: [231](#).
n_bottom_slide: [52](#), [65](#), [143](#), [159](#), [179](#), [181](#), [231](#), [279](#), [337](#).
n_photons: [4](#), [6](#), [16](#), [17](#), [78](#).
n_slab: [52](#), [65](#), [143](#), [179](#), [181](#), [231](#), [279](#), [337](#).
n_top: [231](#).
n_top_slide: [52](#), [65](#), [143](#), [159](#), [179](#), [181](#), [231](#), [279](#), [337](#).
Near_Grid_Points: [156](#), [182](#), [237](#), [314](#).
nfluxes: [231](#).
NO_SLIDES: [3](#), [6](#), [18](#).
NO_UNSCATTERED_LIGHT: [3](#).
nslide: [66](#), [67](#).
num_measures: [20](#), [37](#), [67](#), [69](#), [74](#), [90](#), [98](#), [143](#), [152](#), [255](#), [283](#), [316](#), [339](#).
num_photons: [71](#), [72](#), [84](#), [89](#).
num_spheres: [10](#), [17](#), [18](#), [19](#), [37](#), [50](#), [52](#), [59](#), [69](#), [72](#), [78](#), [86](#), [95](#), [101](#), [113](#), [114](#), [115](#), [183](#), [339](#).
NUMBER_OF_GUESSES: [232](#), [237](#).
old_mm: [78](#), [155](#), [157](#).
old_rr: [78](#), [155](#), [157](#).
ONE_SLIDE_NEAR_SPHERE: [3](#), [6](#), [18](#).
ONE_SLIDE_NOT_NEAR_SPHERE: [3](#), [6](#), [18](#).
ONE_SLIDE_ON_BOTTOM: [3](#), [6](#), [18](#).
ONE_SLIDE_ON_TOP: [3](#), [6](#), [18](#).
optarg: [6](#).
optind: [5](#), [6](#).
overwrite_defaults: [60](#), [64](#).
P: [193](#).
p: [236](#).
P_ss: [193](#).
P_std: [193](#).
P_su: [193](#).
P_0: [193](#).
params: [2](#), [4](#), [16](#), [20](#), [94](#), [98](#), [99](#), [100](#), [101](#), [108](#), [115](#).
parse_string_into_array: [6](#), [30](#).
phase_function: [65](#), [143](#).
points: [33](#).
pow: [15](#).
print_dot: [13](#), [17](#), [33](#).
print_error_legend: [2](#), [26](#).
Print_Invert_Type: [336](#).
print_long_error: [13](#), [32](#).
print_maybe: [110](#), [112](#), [113](#), [114](#), [120](#).
Print_Measure_Type: [338](#).
print_optical_property_result: [10](#), [13](#), [17](#), [25](#).
print_results_header: [10](#), [16](#), [17](#), [24](#).
print_usage: [6](#), [22](#).
print_version: [6](#), [21](#).
printf: [28](#), [110](#), [111](#), [112](#), [113](#), [114](#), [115](#), [120](#).
process_command_line: [2](#), [4](#), [6](#), [12](#).
p1: [232](#).
p2: [232](#).
qsort: [237](#).
quad_Dif_Calc_R_and_T: [231](#).
quad_pts: [15](#), [45](#), [60](#), [65](#), [67](#), [73](#), [89](#), [101](#), [115](#), [143](#), [160](#), [179](#), [337](#).
quick_guess: [232](#), [275](#), [315](#).
r: [4](#), [23](#), [25](#), [30](#), [43](#), [49](#), [56](#), [60](#), [67](#), [71](#), [77](#), [79](#), [81](#), [84](#), [99](#), [101](#), [108](#), [138](#), [140](#), [142](#), [148](#), [156](#), [161](#), [167](#), [170](#), [172](#), [174](#), [176](#), [229](#), [234](#), [243](#), [245](#), [247](#), [249](#), [251](#), [254](#), [259](#), [265](#), [270](#), [283](#), [310](#), [315](#), [334](#), [336](#).
r_first: [193](#), [198](#).
r_old: [228](#), [230](#).
r_third: [198](#).
R_0: [202](#).
rate: [33](#).
rc: [57](#), [283](#), [285](#), [286](#), [288](#), [316](#).
rd: [57](#), [58](#), [283](#), [286](#), [288](#), [316](#), [317](#), [318](#).
Rd: [248](#), [250](#), [252](#).
rd_r: [37](#), [54](#), [69](#), [75](#), [87](#), [113](#), [128](#), [339](#).
rd_t: [37](#), [55](#), [69](#), [76](#), [88](#), [114](#), [128](#), [339](#).
Read_Data_Legend: [98](#), [121](#).
Read_Data_Line: [2](#), [99](#).
Read_Data_Line_Per_Labels: [100](#), [101](#).
Read_Header: [2](#), [94](#).
read_number: [95](#), [96](#), [97](#), [100](#), [101](#), [104](#).
REFLECTION_SPHERE: [123](#), [128](#), [130](#), [132](#), [136](#), [193](#).
RELATIVE: [35](#), [40](#), [46](#), [204](#), [205](#).
remove_comment: [118](#), [122](#).
remove_first_char: [119](#).
remove_whitespace: [117](#), [122](#).
result: [120](#).
results: [70](#), [71](#).
RGRID: [123](#), [153](#), [177](#).
rp: [231](#).
RR: [123](#), [125](#), [138](#), [139](#), [141](#), [143](#), [155](#), [160](#), [162](#), [164](#), [165](#), [166](#), [168](#), [171](#), [173](#), [175](#), [178](#), [179](#), [181](#), [203](#), [204](#), [205](#), [206](#), [208](#), [210](#), [212](#), [214](#), [216](#), [218](#), [220](#), [222](#), [224](#), [226](#), [228](#).
rs: [231](#).
rstd_r: [18](#), [37](#), [54](#), [69](#), [72](#), [75](#), [82](#), [90](#), [95](#), [100](#), [101](#), [113](#), [193](#), [202](#), [339](#).
rstd_t: [18](#), [37](#), [52](#), [54](#), [55](#), [69](#), [76](#), [100](#), [101](#), [114](#), [198](#), [339](#).
rt: [57](#), [59](#), [283](#), [286](#), [288](#).
Rt: [248](#), [250](#), [252](#).
RT: [158](#), [159](#).
RT_Flip: [158](#), [160](#), [179](#).
rt_name: [12](#).
rt_total: [13](#), [14](#), [16](#), [17](#).

- ru*: [50](#), [52](#).
- Ru*: [179](#), [181](#), [182](#), [185](#), [193](#), [248](#), [250](#), [252](#).
- rw_r*: [18](#), [37](#), [54](#), [55](#), [69](#), [72](#), [75](#), [87](#), [96](#), [100](#), [101](#), [113](#), [128](#), [134](#), [136](#), [193](#), [339](#).
- rw_t*: [18](#), [37](#), [55](#), [69](#), [72](#), [76](#), [88](#), [97](#), [100](#), [101](#), [114](#), [128](#), [134](#), [136](#), [198](#), [339](#).
- r1*: [278](#), [279](#), [280](#), [281](#), [282](#).
- r2*: [278](#), [279](#), [280](#), [281](#), [282](#).
- s*: [27](#), [30](#), [144](#), [149](#), [156](#), [231](#).
- Same_Calc_State*: [142](#), [177](#).
- sample*: [83](#), [85](#).
- search*: [13](#), [15](#), [38](#), [44](#), [46](#), [51](#), [57](#), [58](#), [59](#), [63](#), [82](#), [115](#), [143](#), [149](#), [162](#), [168](#), [171](#), [173](#), [175](#), [177](#), [203](#), [237](#), [311](#), [314](#), [321](#), [337](#).
- search_type**: [39](#), [56](#), [144](#), [156](#).
- seconds_elapsed*: [29](#), [33](#).
- SEEK_SET**: [116](#).
- Set_Calc_State*: [78](#), [138](#), [155](#), [157](#), [162](#), [168](#), [171](#), [173](#), [175](#), [228](#), [230](#), [235](#), [241](#), [244](#), [246](#), [248](#), [250](#), [252](#), [253](#), [255](#), [260](#), [266](#), [271](#), [312](#), [313](#).
- Set_Debugging*: [6](#), [328](#).
- setup*: [70](#), [72](#).
- skip*: [13](#), [14](#).
- skip_white*: [102](#), [105](#), [122](#).
- slab*: [10](#), [11](#), [38](#), [47](#), [52](#), [65](#), [80](#), [143](#), [153](#), [155](#), [158](#), [159](#), [160](#), [162](#), [164](#), [165](#), [166](#), [168](#), [171](#), [173](#), [175](#), [179](#), [181](#), [206](#), [208](#), [210](#), [212](#), [214](#), [216](#), [218](#), [220](#), [222](#), [224](#), [226](#), [228](#), [231](#), [235](#), [237](#), [240](#), [241](#), [243](#), [244](#), [245](#), [246](#), [248](#), [250](#), [252](#), [253](#), [255](#), [258](#), [260](#), [264](#), [269](#), [271](#), [274](#), [277](#), [279](#), [281](#), [282](#), [311](#), [312](#), [313](#), [314](#), [318](#), [322](#), [323](#), [325](#), [337](#).
- slab_bottom_slide_b*: [18](#), [37](#), [65](#), [69](#), [101](#).
- slab_bottom_slide_index*: [18](#), [37](#), [65](#), [67](#), [69](#), [72](#), [85](#), [95](#), [101](#), [110](#), [153](#), [339](#).
- slab_bottom_slide_thickness*: [18](#), [37](#), [69](#), [72](#), [85](#), [95](#), [101](#), [110](#), [339](#).
- slab_cos_angle*: [18](#), [37](#), [65](#), [67](#), [69](#), [72](#), [115](#), [153](#).
- slab_index*: [18](#), [37](#), [65](#), [67](#), [69](#), [72](#), [85](#), [95](#), [101](#), [110](#), [153](#), [339](#).
- slab_thickness*: [15](#), [18](#), [37](#), [69](#), [71](#), [72](#), [85](#), [95](#), [101](#), [110](#), [335](#), [339](#).
- slab_top_slide_b*: [18](#), [37](#), [65](#), [69](#), [101](#).
- slab_top_slide_index*: [18](#), [37](#), [65](#), [67](#), [69](#), [72](#), [85](#), [95](#), [101](#), [110](#), [153](#), [339](#).
- slab_top_slide_thickness*: [18](#), [37](#), [69](#), [72](#), [85](#), [95](#), [101](#), [110](#), [339](#).
- slabtype**: [231](#).
- slide_bottom*: [231](#).
- slide_top*: [231](#).
- slow_guess*: [310](#).
- SMALL_A_CALC_VALUE**: [35](#), [293](#).
- smallest*: [157](#).
- Sp_mu_RT*: [52](#).
- Sp_mu_RT_Flip*: [52](#), [179](#), [181](#).
- sphere*: [127](#), [128](#).
- sphere_area*: [69](#).
- sphere_r*: [70](#), [75](#), [83](#), [87](#).
- sphere_t*: [70](#), [76](#), [83](#), [88](#).
- Spheres_Inverse_RT*: [70](#).
- Spheres_Inverse_RT2*: [83](#).
- sqr*: [96](#), [97](#), [332](#).
- sqrt*: [113](#), [114](#), [282](#), [293](#), [339](#).
- sscanf*: [6](#), [30](#).
- start_time*: [2](#), [4](#), [13](#), [17](#), [29](#), [33](#).
- stderr*: [2](#), [5](#), [6](#), [11](#), [12](#), [13](#), [17](#), [18](#), [19](#), [26](#), [27](#), [28](#), [30](#), [32](#), [33](#), [46](#), [47](#), [52](#), [57](#), [72](#), [82](#), [98](#), [101](#), [105](#), [107](#), [122](#), [150](#), [151](#), [152](#), [153](#), [157](#), [160](#), [162](#), [168](#), [171](#), [173](#), [175](#), [179](#), [181](#), [183](#), [193](#), [198](#), [206](#), [230](#), [235](#), [237](#), [238](#), [244](#), [246](#), [248](#), [250](#), [252](#), [255](#), [256](#), [260](#), [262](#), [271](#), [288](#), [318](#), [337](#), [339](#).
- stdin*: [2](#), [12](#).
- stdout*: [10](#), [12](#), [13](#), [16](#), [21](#), [22](#).
- str*: [28](#), [117](#), [118](#), [119](#).
- strcat*: [5](#), [27](#).
- strchr*: [5](#), [98](#), [120](#).
- strcmp*: [12](#).
- strcpy*: [5](#), [27](#).
- strdup*: [6](#), [12](#), [27](#).
- strdup_together*: [12](#), [27](#).
- strlen*: [5](#), [12](#), [27](#), [30](#), [100](#), [119](#), [122](#).
- strstr*: [12](#).
- strtod*: [28](#).
- SUBSTITUTION**: [18](#), [36](#), [95](#), [115](#).
- swap*: [123](#).
- SWAP**: [123](#), [159](#).
- t*: [27](#), [30](#), [156](#).
- T_TRUST_FACTOR**: [123](#), [205](#).
- T_0**: [202](#).
- tc*: [57](#), [283](#), [285](#), [287](#), [288](#), [316](#).
- td*: [57](#), [283](#), [287](#), [288](#), [316](#).
- Td*: [248](#), [250](#), [252](#), [283](#).
- tdiffuse*: [129](#), [130](#), [131](#), [132](#).
- temp_m_t*: [82](#).
- temp_mua*: [7](#), [8](#), [9](#).
- temp_mus*: [7](#), [8](#), [9](#).
- The_Grid*: [123](#), [143](#), [145](#), [147](#), [150](#), [160](#), [162](#), [168](#), [171](#), [173](#), [175](#), [181](#).
- The_Grid_Initialized*: [123](#), [143](#), [145](#), [150](#), [162](#), [168](#), [171](#), [173](#), [175](#).
- The_Grid_Search*: [123](#), [151](#), [162](#), [168](#), [171](#), [173](#), [175](#).
- tmp*: [317](#).
- tmp_str*: [6](#).

- tolerance*: [15](#), [38](#), [63](#), [101](#), [115](#), [235](#), [241](#), [244](#), [246](#),
[248](#), [250](#), [253](#), [255](#), [260](#), [266](#), [271](#).
tp: [231](#), [283](#).
TRANSMISSION_SPHERE: [123](#), [130](#), [132](#), [134](#), [198](#).
TRUE: [2](#), [10](#), [11](#), [13](#), [35](#), [47](#), [67](#), [71](#), [80](#), [84](#), [143](#),
[149](#), [162](#), [168](#), [171](#), [173](#), [175](#).
ts: [231](#).
tt: [57](#), [58](#), [59](#), [283](#), [287](#), [288](#).
Tt: [248](#), [250](#), [252](#).
tu: [50](#), [52](#).
Tu: [66](#), [67](#), [179](#), [181](#), [182](#), [185](#), [198](#), [248](#), [250](#),
[252](#), [277](#), [280](#), [281](#), [282](#).
TWO_IDENTICAL_SLIDES: [3](#), [6](#).
Two_Sphere_R: [133](#), [202](#).
Two_Sphere_T: [135](#), [202](#).
twoprime: [302](#), [307](#).
twounprime: [304](#), [307](#).
t1: [278](#), [279](#), [280](#), [281](#), [282](#).
t2: [278](#), [279](#), [280](#), [281](#), [282](#).
U_Find_A: [46](#), [244](#), [246](#), [247](#).
U_Find_AB: [46](#), [234](#).
U_Find_AG: [46](#), [254](#).
U_Find_B: [46](#), [81](#), [82](#), [251](#).
U_Find_Ba: [46](#), [245](#).
U_Find_BaG: [46](#), [265](#).
U_Find_BG: [46](#), [259](#), [265](#), [270](#).
U_Find_Bs: [46](#), [243](#).
U_Find_BsG: [46](#), [270](#).
U_Find_G: [46](#), [249](#).
ungetc: [103](#).
UNINITIALIZED: [2](#), [4](#), [6](#), [7](#), [8](#), [9](#), [11](#), [13](#), [15](#), [18](#),
[36](#), [51](#), [57](#), [58](#), [59](#), [64](#), [80](#), [115](#), [235](#), [244](#), [246](#),
[248](#), [250](#), [252](#), [255](#), [260](#), [271](#), [335](#).
UNKNOWN: [18](#), [36](#), [69](#), [115](#).
URU: [129](#), [130](#), [131](#), [132](#), [133](#), [134](#), [135](#), [136](#), [158](#),
[159](#), [182](#), [184](#), [231](#).
uru: [14](#), [17](#), [71](#), [78](#), [84](#), [160](#), [179](#), [181](#).
URU_calc: [184](#), [193](#), [198](#), [202](#), [205](#).
URU_COLUMN: [123](#), [160](#), [181](#).
uru_lost: [11](#), [13](#), [17](#), [25](#), [37](#), [69](#), [71](#), [77](#), [78](#), [84](#), [184](#).
uru_sample: [126](#), [127](#), [128](#).
uru_third: [126](#), [127](#), [128](#).
UR1: [66](#), [67](#), [133](#), [134](#), [135](#), [136](#), [158](#), [159](#), [182](#),
[185](#), [193](#), [200](#), [231](#), [316](#), [318](#).
ur1: [14](#), [17](#), [71](#), [78](#), [84](#), [160](#), [179](#), [181](#).
UR1_calc: [184](#), [185](#), [186](#), [193](#), [200](#), [202](#).
UR1_COLUMN: [123](#), [160](#), [181](#).
ur1_loss: [229](#), [230](#).
ur1_lost: [11](#), [13](#), [17](#), [25](#), [37](#), [47](#), [69](#), [71](#), [78](#), [84](#),
[185](#), [186](#), [193](#), [228](#), [230](#), [339](#).
UTU: [133](#), [134](#), [135](#), [136](#), [158](#), [159](#), [182](#), [184](#), [231](#).
utu: [14](#), [17](#), [71](#), [78](#), [84](#), [160](#), [179](#), [181](#).
UTU_calc: [184](#), [202](#), [205](#).
UTU_COLUMN: [123](#), [160](#), [181](#).
utu_lost: [11](#), [13](#), [17](#), [25](#), [37](#), [69](#), [71](#), [78](#), [84](#), [184](#), [339](#).
UT1: [66](#), [67](#), [133](#), [134](#), [135](#), [136](#), [158](#), [159](#), [182](#), [185](#),
[198](#), [200](#), [231](#), [316](#), [317](#), [318](#).
ut1: [14](#), [17](#), [71](#), [78](#), [84](#), [160](#), [179](#), [181](#).
UT1_calc: [184](#), [185](#), [186](#), [198](#), [200](#), [202](#).
UT1_COLUMN: [123](#), [160](#), [181](#).
ut1_loss: [229](#), [230](#).
ut1_lost: [11](#), [13](#), [17](#), [25](#), [37](#), [47](#), [69](#), [71](#), [78](#), [84](#),
[185](#), [186](#), [198](#), [228](#), [230](#), [339](#).
val: [28](#).
Valid_Grid: [148](#), [237](#), [314](#).
verbosity: [21](#), [33](#).
Version: [21](#), [22](#), [110](#).
VersionShort: [21](#).
what_char: [25](#), [31](#), [33](#).
What_Is_B: [80](#), [250](#), [255](#), [277](#), [285](#), [318](#), [322](#),
[323](#), [325](#).
Write_Header: [10](#), [16](#), [108](#).
x: [95](#), [101](#), [104](#), [120](#), [134](#), [136](#), [162](#), [164](#), [165](#), [166](#),
[171](#), [173](#), [175](#), [207](#), [209](#), [211](#), [213](#), [215](#), [217](#), [219](#),
[221](#), [223](#), [225](#), [236](#), [248](#), [250](#), [253](#), [311](#), [332](#).
xx: [110](#), [115](#), [165](#), [166](#).
y: [236](#).
zbrent: [230](#).

- ⟨ Allocate local simplex variables 236 ⟩ Used in sections 235, 255, 260, 266, and 271.
- ⟨ Calc M_R and M_T for dual beam sphere 200 ⟩ Used in section 183.
- ⟨ Calc M_R and M_T for no spheres 186 ⟩ Used in section 183.
- ⟨ Calc M_R and M_T for single beam sphere 193, 198 ⟩ Used in section 183.
- ⟨ Calc M_R and M_T for two spheres 202 ⟩ Used in section 183.
- ⟨ Calculate and Print the Forward Calculation 7, 8, 9, 10 ⟩ Used in section 2.
- ⟨ Calculate and write optical properties 13 ⟩ Used in section 2.
- ⟨ Calculate specular reflection and transmission 279 ⟩ Used in section 278.
- ⟨ Calculate the deviation 203 ⟩ Used in section 183.
- ⟨ Calculate the unscattered transmission and reflection 285 ⟩ Used in section 284.
- ⟨ Check MU 53 ⟩ Used in section 50.
- ⟨ Check MR for zero or one spheres 51 ⟩ Used in section 50.
- ⟨ Check MT for zero or one spheres 52 ⟩ Used in section 50.
- ⟨ Check for bad values of Tu 280 ⟩ Used in section 278.
- ⟨ Check sphere parameters 54, 55 ⟩ Used in section 50.
- ⟨ Choose the best node of the a and b simplex 240 ⟩ Used in section 235.
- ⟨ Choose the best node of the a and g simplex 258 ⟩ Used in section 255.
- ⟨ Choose the best node of the ba and g simplex 269 ⟩ Used in section 266.
- ⟨ Choose the best node of the bs and g simplex 274 ⟩ Used in section 271.
- ⟨ Choose the best node of the b and g simplex 264 ⟩ Used in section 260.
- ⟨ Clean up guesses 326 ⟩ Used in section 316.
- ⟨ Command-line changes to m 18 ⟩ Used in section 2.
- ⟨ Command-line changes to r 15 ⟩ Used in sections 2 and 13.
- ⟨ Count command-line measurements 20 ⟩ Used in section 2.
- ⟨ Debug info for estimate RT 288 ⟩ Used in section 284.
- ⟨ Declare variables for main 4 ⟩ Used in section 2.
- ⟨ Definition for Allocate_Grid 145 ⟩ Used in section 123.
- ⟨ Definition for Calculate_Distance_With_Corrections 183 ⟩ Used in section 123.
- ⟨ Definition for Calculate_Distance 179 ⟩ Used in section 123.
- ⟨ Definition for Calculate_Grid_Distance 181 ⟩ Used in section 123.
- ⟨ Definition for Calculate_MR_MT 78 ⟩ Used in section 40.
- ⟨ Definition for Calculate_Minimum_MR 80 ⟩ Used in section 40.
- ⟨ Definition for Calculate_Mua_Musp 335 ⟩ Used in section 275.
- ⟨ Definition for Debug 331 ⟩ Used in section 275.
- ⟨ Definition for Estimate_RT 284 ⟩ Used in section 275.
- ⟨ Definition for Fill_AB_Grid 162 ⟩ Used in section 123.
- ⟨ Definition for Fill_AG_Grid 168 ⟩ Used in section 123.
- ⟨ Definition for Fill_BG_Grid 171 ⟩ Used in section 123.
- ⟨ Definition for Fill_BaG_Grid 173 ⟩ Used in section 123.
- ⟨ Definition for Fill_BsG_Grid 175 ⟩ Used in section 123.
- ⟨ Definition for Fill_Grid 177 ⟩ Used in section 123.
- ⟨ Definition for Find_AB_fn 210 ⟩ Used in section 123.
- ⟨ Definition for Find_AG_fn 208 ⟩ Used in section 123.
- ⟨ Definition for Find_A_fn 216 ⟩ Used in section 123.
- ⟨ Definition for Find_BG_fn 222 ⟩ Used in section 123.
- ⟨ Definition for Find_B_fn 218 ⟩ Used in section 123.
- ⟨ Definition for Find_BaG_fn 224 ⟩ Used in section 123.
- ⟨ Definition for Find_Ba_fn 212 ⟩ Used in section 123.
- ⟨ Definition for Find_BsG_fn 226 ⟩ Used in section 123.
- ⟨ Definition for Find_Bs_fn 214 ⟩ Used in section 123.
- ⟨ Definition for Find_G_fn 220 ⟩ Used in section 123.
- ⟨ Definition for Gain_11 130 ⟩ Used in section 123.

- ⟨ Definition for *Gain_22* 132 ⟩ Used in section 123.
- ⟨ Definition for *Gain* 128 ⟩ Used in section 123.
- ⟨ Definition for *Get_Calc_State* 141 ⟩ Used in section 123.
- ⟨ Definition for *Grid_ABG* 147 ⟩ Used in section 123.
- ⟨ Definition for *Initialize_Measure* 69 ⟩ Used in section 40.
- ⟨ Definition for *Initialize_Result* 61 ⟩ Used in section 40.
- ⟨ Definition for *Inverse_RT* 44 ⟩ Used in section 40.
- ⟨ Definition for *Max_Light_Loss* 230 ⟩ Used in section 123.
- ⟨ Definition for *MinMax_MR_MT* 82 ⟩ Used in section 40.
- ⟨ Definition for *Near_Grid_Points* 157 ⟩ Used in section 123.
- ⟨ Definition for *Print_Invert_Type* 337 ⟩ Used in section 275.
- ⟨ Definition for *Print_Measure_Type* 339 ⟩ Used in section 275.
- ⟨ Definition for *RT_Flip* 159 ⟩ Used in section 123.
- ⟨ Definition for *Read_Data_Legend* 122 ⟩ Used in section 91.
- ⟨ Definition for *Read_Data_Line_Per_Labels* 101 ⟩ Used in section 91.
- ⟨ Definition for *Read_Data_Line* 100 ⟩ Used in section 91.
- ⟨ Definition for *Read_Header* 95 ⟩ Used in section 91.
- ⟨ Definition for *Same_Calc_State* 143 ⟩ Used in section 123.
- ⟨ Definition for *Set_Calc_State* 139 ⟩ Used in section 123.
- ⟨ Definition for *Set_Debugging* 329 ⟩ Used in section 275.
- ⟨ Definition for *Spheres_Inverse_RT2* 84 ⟩ Used in section 40.
- ⟨ Definition for *Spheres_Inverse_RT* 71 ⟩ Used in section 40.
- ⟨ Definition for *Two_Sphere_R* 134 ⟩ Used in section 123.
- ⟨ Definition for *Two_Sphere_T* 136 ⟩ Used in section 123.
- ⟨ Definition for *U_Find_AB* 235 ⟩ Used in section 232.
- ⟨ Definition for *U_Find_AG* 255 ⟩ Used in section 232.
- ⟨ Definition for *U_Find_A* 248 ⟩ Used in section 232.
- ⟨ Definition for *U_Find_BG* 260 ⟩ Used in section 232.
- ⟨ Definition for *U_Find_BaG* 266 ⟩ Used in section 232.
- ⟨ Definition for *U_Find_Ba* 246 ⟩ Used in section 232.
- ⟨ Definition for *U_Find_BsG* 271 ⟩ Used in section 232.
- ⟨ Definition for *U_Find_Bs* 244 ⟩ Used in section 232.
- ⟨ Definition for *U_Find_B* 252 ⟩ Used in section 232.
- ⟨ Definition for *U_Find_G* 250 ⟩ Used in section 232.
- ⟨ Definition for *Valid_Grid* 149 ⟩ Used in section 123.
- ⟨ Definition for *What_Is_B* 278 ⟩ Used in section 275.
- ⟨ Definition for *Write_Header* 109 ⟩ Used in section 91.
- ⟨ Definition for *a2acalc* 291 ⟩ Used in section 275.
- ⟨ Definition for *abg_distance* 155 ⟩ Used in section 123.
- ⟨ Definition for *abgb2ag* 309 ⟩ Used in section 275.
- ⟨ Definition for *abgg2ab* 307 ⟩ Used in section 275.
- ⟨ Definition for *acalc2a* 293 ⟩ Used in section 275.
- ⟨ Definition for *b2bcalc* 299 ⟩ Used in section 275.
- ⟨ Definition for *bcalc2b* 301 ⟩ Used in section 275.
- ⟨ Definition for *check_magic* 107 ⟩ Used in section 91.
- ⟨ Definition for *determine_search* 57 ⟩ Used in section 40.
- ⟨ Definition for *ez_Inverse_RT* 67 ⟩ Used in section 40.
- ⟨ Definition for *fill_grid_entry* 160 ⟩ Used in section 123.
- ⟨ Definition for *g2gcalc* 295 ⟩ Used in section 275.
- ⟨ Definition for *gcalc2g* 297 ⟩ Used in section 275.
- ⟨ Definition for *get_current_line_number* 116 ⟩ Used in section 91.
- ⟨ Definition for *maxloss* 228 ⟩ Used in section 123.

- ⟨ Definition for *measure_OK* 50 ⟩ Used in section 40.
- ⟨ Definition for *print_maybe* 120 ⟩ Used in section 91.
- ⟨ Definition for *quick_guess* 316 ⟩ Used in section 275.
- ⟨ Definition for *read_number* 105 ⟩ Used in section 91.
- ⟨ Definition for *remove_comment* 118 ⟩ Used in section 91.
- ⟨ Definition for *remove_first_char* 119 ⟩ Used in section 91.
- ⟨ Definition for *remove_whitespace* 117 ⟩ Used in section 91.
- ⟨ Definition for *skip_white* 103 ⟩ Used in section 91.
- ⟨ Definition for *slow_guess* 311 ⟩
- ⟨ Definition for *sqr* 333 ⟩ Used in section 275.
- ⟨ Definition for *twoprime* 303 ⟩ Used in section 275.
- ⟨ Definition for *twounprime* 305 ⟩ Used in section 275.
- ⟨ Determine calculated light to be used 184, 185 ⟩ Used in section 183.
- ⟨ Estimate the backscattered reflection 286 ⟩ Used in section 284.
- ⟨ Estimate the scattered transmission 287 ⟩ Used in section 284.
- ⟨ Estimate *aprime* 317 ⟩ Used in section 316.
- ⟨ Estimate *bprime* 318 ⟩ Used in sections 320, 324, and 325.
- ⟨ Evaluate the *BaG* simplex at the nodes 268 ⟩ Used in section 266.
- ⟨ Evaluate the *BsG* simplex at the nodes 273 ⟩ Used in section 271.
- ⟨ Evaluate the *bg* simplex at the nodes 263 ⟩ Used in section 260.
- ⟨ Evaluate the *a* and *b* simplex at the nodes 239 ⟩ Used in section 235.
- ⟨ Evaluate the *a* and *g* simplex at the nodes 257 ⟩ Used in section 255.
- ⟨ Exit with bad input data 45 ⟩ Used in section 44.
- ⟨ Fill *r* with reasonable values 62, 63, 64, 65 ⟩ Used in section 61.
- ⟨ Find the optical properties 46 ⟩ Used in section 44.
- ⟨ Find thickness when multiple internal reflections are present 282 ⟩ Used in section 278.
- ⟨ Free simplex data structures 242 ⟩ Used in sections 235, 255, 260, 266, and 271.
- ⟨ Generate and write grid 11 ⟩ Used in section 2.
- ⟨ Generate next albedo using *j* 164 ⟩ Used in sections 162 and 168.
- ⟨ Generate next anisotropy using *i* 165 ⟩ Used in section 168.
- ⟨ Generate next anisotropy using *j* 166 ⟩ Used in sections 171, 173, and 175.
- ⟨ Get the initial *a*, *b*, and *g* 237 ⟩ Used in sections 235, 255, 260, 266, and 271.
- ⟨ Guess when all three measurements are known 321 ⟩ Used in section 316.
- ⟨ Guess when finding albedo 322 ⟩ Used in section 321.
- ⟨ Guess when finding anisotropy and albedo 325 ⟩ Used in section 321.
- ⟨ Guess when finding optical depth 323 ⟩ Used in section 321.
- ⟨ Guess when finding the albedo and optical depth 324 ⟩ Used in section 321.
- ⟨ Guess when only reflection is known 319 ⟩ Used in section 316.
- ⟨ Guess when reflection and transmission are known 320 ⟩ Used in section 316.
- ⟨ Handle options 6 ⟩ Used in section 2.
- ⟨ Improve result using Monte Carlo 17 ⟩ Used in section 13.
- ⟨ Include files for *main* 3 ⟩ Used in section 2.
- ⟨ Initialize the nodes of the *a* and *b* simplex 238 ⟩ Used in section 235.
- ⟨ Initialize the nodes of the *a* and *g* simplex 256 ⟩ Used in section 255.
- ⟨ Initialize the nodes of the *ba* and *g* simplex 267 ⟩ Used in section 266.
- ⟨ Initialize the nodes of the *bs* and *g* simplex 272 ⟩ Used in section 271.
- ⟨ Initialize the nodes of the *b* and *g* simplex 262 ⟩ Used in section 260.
- ⟨ Iteratively solve for *b* 253 ⟩ Used in section 252.
- ⟨ Local Variables for Calculation 14 ⟩ Used in section 13.
- ⟨ One parameter deviation 204 ⟩ Used in section 203.
- ⟨ One parameter search 58 ⟩ Used in section 57.
- ⟨ Print basic sphere and MC effects 47 ⟩ Used in section 44.

- ⟨Print diagnostics 206⟩ Used in section 183.
- ⟨Print results function 25⟩ Used in section 2.
- ⟨Prototype for *Allocate_Grid* 144⟩ Used in sections 124 and 145.
- ⟨Prototype for *Calculate_Distance_With_Corrections* 182⟩ Used in sections 124 and 183.
- ⟨Prototype for *Calculate_Distance* 178⟩ Used in sections 124 and 179.
- ⟨Prototype for *Calculate_Grid_Distance* 180⟩ Used in sections 124 and 181.
- ⟨Prototype for *Calculate_MR_MT* 77⟩ Used in sections 41 and 78.
- ⟨Prototype for *Calculate_Minimum_MR* 79⟩ Used in sections 41 and 80.
- ⟨Prototype for *Calculate_Mua_Musp* 334⟩ Used in sections 276 and 335.
- ⟨Prototype for *Debug* 330⟩ Used in sections 276 and 331.
- ⟨Prototype for *Estimate_RT* 283⟩ Used in sections 276 and 284.
- ⟨Prototype for *Fill_AB_Grid* 161⟩ Used in sections 123 and 162.
- ⟨Prototype for *Fill_AG_Grid* 167⟩ Used in sections 123 and 168.
- ⟨Prototype for *Fill_BG_Grid* 170⟩ Used in sections 124 and 171.
- ⟨Prototype for *Fill_BaG_Grid* 172⟩ Used in sections 124 and 173.
- ⟨Prototype for *Fill_BsG_Grid* 174⟩ Used in sections 124 and 175.
- ⟨Prototype for *Fill_Grid* 176⟩ Used in sections 124 and 177.
- ⟨Prototype for *Find_AB_fn* 209⟩ Used in sections 124 and 210.
- ⟨Prototype for *Find_AG_fn* 207⟩ Used in sections 124 and 208.
- ⟨Prototype for *Find_A_fn* 215⟩ Used in sections 124 and 216.
- ⟨Prototype for *Find_BG_fn* 221⟩ Used in sections 124 and 222.
- ⟨Prototype for *Find_B_fn* 217⟩ Used in sections 124 and 218.
- ⟨Prototype for *Find_BaG_fn* 223⟩ Used in sections 124 and 224.
- ⟨Prototype for *Find_Ba_fn* 211⟩ Used in sections 124 and 212.
- ⟨Prototype for *Find_BsG_fn* 225⟩ Used in sections 124 and 226.
- ⟨Prototype for *Find_Bs_fn* 213⟩ Used in sections 124 and 214.
- ⟨Prototype for *Find_G_fn* 219⟩ Used in sections 124 and 220.
- ⟨Prototype for *Gain_11* 129⟩ Used in sections 124 and 130.
- ⟨Prototype for *Gain_22* 131⟩ Used in sections 124 and 132.
- ⟨Prototype for *Gain* 127⟩ Used in sections 124 and 128.
- ⟨Prototype for *Get_Calc_State* 140⟩ Used in sections 124 and 141.
- ⟨Prototype for *Grid_ABG* 146⟩ Used in sections 124 and 147.
- ⟨Prototype for *Initialize_Measure* 68⟩ Used in sections 41 and 69.
- ⟨Prototype for *Initialize_Result* 60⟩ Used in sections 41 and 61.
- ⟨Prototype for *Inverse_RT* 43⟩ Used in sections 41 and 44.
- ⟨Prototype for *Max_Light_Loss* 229⟩ Used in sections 124 and 230.
- ⟨Prototype for *MinMax_MR_MT* 81⟩ Used in sections 41 and 82.
- ⟨Prototype for *Near_Grid_Points* 156⟩ Used in sections 124 and 157.
- ⟨Prototype for *Print_Invert_Type* 336⟩ Used in sections 276 and 337.
- ⟨Prototype for *Print_Measure_Type* 338⟩ Used in sections 276 and 339.
- ⟨Prototype for *RT_Flip* 158⟩ Used in sections 124 and 159.
- ⟨Prototype for *Read_Data_Legend* 121⟩ Used in section 122.
- ⟨Prototype for *Read_Data_Line* 99⟩ Used in sections 92 and 100.
- ⟨Prototype for *Read_Header* 94⟩ Used in sections 92 and 95.
- ⟨Prototype for *Same_Calc_State* 142⟩ Used in sections 124 and 143.
- ⟨Prototype for *Set_Calc_State* 138⟩ Used in sections 124 and 139.
- ⟨Prototype for *Set_Debugging* 328⟩ Used in sections 276 and 329.
- ⟨Prototype for *Spheres_Inverse_RT2* 83⟩ Used in sections 41, 42, and 84.
- ⟨Prototype for *Spheres_Inverse_RT* 70⟩ Used in sections 42 and 71.
- ⟨Prototype for *Two_Sphere_R* 133⟩ Used in sections 124 and 134.
- ⟨Prototype for *Two_Sphere_T* 135⟩ Used in sections 124 and 136.
- ⟨Prototype for *U_Find_AB* 234⟩ Used in sections 233 and 235.

- ⟨Prototype for *U_Find_AG* 254⟩ Used in sections 233 and 255.
- ⟨Prototype for *U_Find_A* 247⟩ Used in sections 233 and 248.
- ⟨Prototype for *U_Find_BG* 259⟩ Used in sections 233 and 260.
- ⟨Prototype for *U_Find_BaG* 265⟩ Used in sections 233 and 266.
- ⟨Prototype for *U_Find_Ba* 245⟩ Used in sections 233 and 246.
- ⟨Prototype for *U_Find_BsG* 270⟩ Used in sections 233 and 271.
- ⟨Prototype for *U_Find_Bs* 243⟩ Used in sections 233 and 244.
- ⟨Prototype for *U_Find_B* 251⟩ Used in sections 233 and 252.
- ⟨Prototype for *U_Find_G* 249⟩ Used in sections 233 and 250.
- ⟨Prototype for *Valid_Grid* 148⟩ Used in sections 124 and 149.
- ⟨Prototype for *What_Is_B* 277⟩ Used in sections 276 and 278.
- ⟨Prototype for *Write_Header* 108⟩ Used in sections 92 and 109.
- ⟨Prototype for *a2acalc* 290⟩ Used in sections 276 and 291.
- ⟨Prototype for *abg_distance* 154⟩ Used in sections 124 and 155.
- ⟨Prototype for *abgb2ag* 308⟩ Used in sections 276 and 309.
- ⟨Prototype for *abgg2ab* 306⟩ Used in sections 276 and 307.
- ⟨Prototype for *acalc2a* 292⟩ Used in sections 276 and 293.
- ⟨Prototype for *b2bcalc* 298⟩ Used in sections 276 and 299.
- ⟨Prototype for *bcalc2b* 300⟩ Used in sections 276 and 301.
- ⟨Prototype for *check_magic* 106⟩ Used in section 107.
- ⟨Prototype for *determine_search* 56⟩ Used in sections 41 and 57.
- ⟨Prototype for *ez_Inverse_RT* 66⟩ Used in sections 41, 42, and 67.
- ⟨Prototype for *g2gcalc* 294⟩ Used in sections 276 and 295.
- ⟨Prototype for *gcalc2g* 296⟩ Used in sections 276 and 297.
- ⟨Prototype for *maxloss* 227⟩ Used in sections 124 and 228.
- ⟨Prototype for *measure_OK* 49⟩ Used in sections 41 and 50.
- ⟨Prototype for *quick_guess* 315⟩ Used in sections 276 and 316.
- ⟨Prototype for *read_number* 104⟩ Used in section 105.
- ⟨Prototype for *skip_white* 102⟩ Used in section 103.
- ⟨Prototype for *slow_guess* 310⟩ Used in section 311.
- ⟨Prototype for *sqr* 332⟩ Used in sections 276 and 333.
- ⟨Prototype for *twoprime* 302⟩ Used in sections 276 and 303.
- ⟨Prototype for *twounprime* 304⟩ Used in sections 276 and 305.
- ⟨Put final values in result 241⟩ Used in sections 235, 244, 246, 248, 250, 252, 255, 260, 266, and 271.
- ⟨Read coefficients for reflection sphere 96⟩ Used in section 95.
- ⟨Read coefficients for transmission sphere 97⟩ Used in section 95.
- ⟨Read info about measurements 98⟩ Used in section 95.
- ⟨Save the command line for use later 5⟩ Used in section 2.
- ⟨Slow guess for *a* alone 312⟩ Used in section 311.
- ⟨Slow guess for *a* and *b* or *a* and *g* 314⟩ Used in section 311.
- ⟨Slow guess for *b* alone 313⟩ Used in section 311.
- ⟨Solve if multiple internal reflections are not present 281⟩ Used in section 278.
- ⟨Structs to export from IAD Types 37, 38, 39⟩ Used in section 34.
- ⟨Tests for invalid grid 150, 151, 152, 153⟩ Used in section 149.
- ⟨Two parameter deviation 205⟩ Used in section 203.
- ⟨Two parameter search 59⟩ Used in section 57.
- ⟨Unused diffusion fragment 231⟩
- ⟨Warn and quit for bad options 19⟩ Used in section 13.
- ⟨Write Header 16⟩ Used in section 13.
- ⟨Write first sphere info 113⟩ Used in section 109.
- ⟨Write general sphere info 112⟩ Used in section 109.
- ⟨Write irradiation info 111⟩ Used in section 109.

<Write measure and inversion info 115> Used in section 109.
 <Write second sphere info 114> Used in section 109.
 <Write slab info 110> Used in section 109.
 <Zero GG 169> Used in sections 162, 168, 171, 173, and 175.
 <calculate coefficients function 23> Used in section 2.
 <handle analysis 73> Used in section 71.
 <handle measurement 74> Used in section 71.
 <handle reflection sphere 75> Used in section 71.
 <handle setup 72> Used in section 71.
 <handle transmission sphere 76> Used in section 71.
 <handle2 analysis 89> Used in section 84.
 <handle2 illumination 86> Used in section 84.
 <handle2 measurement 90> Used in section 84.
 <handle2 reflection sphere 87> Used in section 84.
 <handle2 sample 85> Used in section 84.
 <handle2 transmission sphere 88> Used in section 84.
 <iad_calc.c 123>
 <iad_calc.h 124>
 <iad_find.c 232>
 <iad_find.h 233>
 <iad_io.c 91>
 <iad_io.h 92>
 <iad_main.c 2>
 <iad_pub.c 40>
 <iad_pub.h 41>
 <iad_type.h 34>
 <iad_util.c 275>
 <iad_util.h 276>
 <lib_iad.h 42>
 <mystrtod function 28> Used in section 2.
 <parse string into array function 30> Used in section 2.
 <prepare file for reading 12> Used in section 2.
 <print dot function 33> Used in section 2.
 <print error legend function 26> Used in section 2.
 <print long error function 32> Used in section 2.
 <print results header function 24> Used in section 2.
 <print usage function 22> Used in section 2.
 <print version function 21> Used in section 2.
 <seconds elapsed function 29> Used in section 2.
 <stringdup together function 27> Used in section 2.
 <what_char function 31> Used in section 2.